



Assignment Cover Sheet

Subject Code: CSIT321
Subject Name: Project
Submission Type: Report
Assignment Title: Software Design Document | EduAgentX
Student Name: Raniya Habachi, Pranav Sujith Nambiar, Timothy Kyle R. Valle, Neeraj Santosh, Djuraev Timurmalik, Luis Vargas, Salman Akram
Student Number: 8316703, 8097367, 8538797, 8329345, 8562763, 7655538, 8770268

Student Email: _rh225@uowmail.edu.au, _psn993@uowmail.edu.au, _tkv250@uowmail.edu.au, _ns769@uowmail.edu.au, _td164@uowmail.edu.au, _lev936@uowmail.edu.au, _sa109@uowmail.edu.au

Lecturer Name: Dr Patrick Mukala
Due Date: 06/28/2026
Date Submitted: 06/28/2026

PLAGIARISM:

The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism. The University of Wollongong in Dubai also endorses a policy of non-discriminatory language practice and presentation.

PLEASE NOTE: STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED

DECLARATION:

I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism.

Signature of Student:

Optional Marks:

Comments:

Lecturer Assignment Receipt (To be filled in by student and retained by Lecturer upon return of assignment)

Subject: CSIT321
Student Name:
Due Date: 06/28/2026
Signature of Student:

Assignment Title: Software Design Document | EduAgentX
Student Number:
Date Submitted: 06/28/2026

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Software Design Document | EduAgentX

Student Name: Raniya Habachi

Student Number: 8316703

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Software Design Document | EduAgentX

Student Name: Pranav Sujith Nambiar

Student Number: 8097367

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Software Design Document | EduAgentX

Student Name: Timothy Kyle R. Valle

Student Number: 8538797

Due Date: 06/28/2026

Date Submitted: 06/28/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Software Design Document | EduAgentX

Student Name: Neeraj Santosh

Student Number: 8329345

Due Date: 06/28/2026

Date Submitted: 06/28/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Software Design Document | EduAgentX

Student Name: Djuraev Timurmalik

Student Number: 8562763

Due Date: 06/28/2026

Date Submitted: 06/28/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Software Design Document | EduAgentX

Student Name: Luis Vargas

Student Number: 7655538

Due Date: 06/28/2026

Date Submitted: 06/28/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Software Design Document | EduAgentX

Student Name: Salman Akram

Student Number: 8770268

Due Date: 06/28/2026

Date Submitted: 06/28/2026

Signature of Lecturer

Table of Contributions

Student ID	Name	Role	Contribution to This Report
8316703	Raniya Habachi	Team Leader	5. Component Design Pseudocode & procedural descriptions for all 10 microservice components, agent enrolment through monitoring & observability
8097367	Pranav Sujith Nambiar	Requirements Engineer	3.3 Design Rationale + 4.1 Data Description Microservices vs. Monolithic Trade-offs, Communication Mechanics Rationale, ChromaDB Containerization Justification, Database Organization & Security Provisions
8538797	Timothy Kyle R. Valle	Scribe & Documentation Lead	3.2 Decomposition Description Subsystem Module Communication Layout, Object/Class Structural Diagrams, all 4 Core Interaction Sequence Diagrams
8329345	Neeraj Santosh	Market Analysis & Scope	2. System Overview + 3.1 Architectural Design Functional Context, High-Level Platform Design, 5-Layer Architecture Breakdown, Unified System Topology Diagram
8562763	Djuraev Timurmali	Resources & Critical Appraisal	4.2 Data Dictionary Full relational schema definitions for all 10 database tables (agents, curricula, modules, training_sessions, examinations, credentials, job_postings, agent_matches, skill_gaps, system_health)
7655538	Luis Vargas	Project Management Lead	1. Introduction Purpose, Scope, Overview, Reference Material, Definitions & Acronyms
8770268	Salman Akram	Technical Architect	6. Human Interface Design + 7. Requirements Matrix UI wireframes for all 4 admin panels, UI-to-backend endpoint mappings, full bidirectional traceability grid (FR-01–18, SEC-01–08)

Table of Contents

1 Introduction.....	7
1.1 Purpose	7
1.1.1 Document Objective as a Technical Coding Blueprint	7
1.1.2 Intended Audience	8
1.2 Scope.....	8
1.2.1 Product Boundaries: The Zero-Human-Loop (ZHL) Autonomous Training Environment ...	8
1.2.2 Excluded Boundaries	9
1.2.3 Trimester 1 Proof of Concept (POC) Domain Limits	9
1.3 Overview.....	10
1.3.1 Document Organization and Structural Guide	10
1.4 Reference Material.....	11
1.4.1 Academic and Technical Citations	11
1.5 Definitions & Acronyms.....	12
1.5.1 Platform Terms	12
1.5.2 Engineering Terms.....	12
2. SYSTEM OVERVIEW	13
2.1 Functional Context.....	14
2.1.1 The Ingestion-to-Deployment Pipeline.....	14
2.1.2 External Functional Actors	17
2.1.3 Trimester 1 Functional Boundary.....	18
2.2 High-Level Platform Design.....	18
2.2.1 Core Orchestration Ecosystem	18
2.2.2 Security and Trust Boundaries.....	20
2.2.3 Deployment View.....	21
3. SYSTEM ARCHITECTURE	21
3.1 Architectural Design	21
3.1.1 High-Level Partitioning: The Five-Layer System Architecture	22
3.1.2 Unified System Topology Diagram.....	24
3.2. Decomposition Description	25
3.2.1. Subsystem Module Communication Layout	26
3.2.2. Object/Class Structural Diagrams: Microservice Interface Mapping.....	28
3.2.3. Core Interaction Sequence Diagrams: Time-Ordered System Workflows.....	37
3.3.1 Microservices vs. Monolithic Architecture Trade-offs	43

3.3.2 Communication Mechanics: Synchronous REST APIs vs. Asynchronous Redis Pub/Sub Event Channels	44
3.3.3 Vector Data Rationale: Client-Server ChromaDB Containerization for Hallucination Mitigation	46
4. Data Design	47
4.1. Data Description	47
4.1.1 Transformational Mapping: Ingesting Educational Data into Structured Models.....	47
4.1.2 Database Organization: Relational Storage (PostgreSQL), Vector Embeddings (ChromaDB), and Volatile Cache (Redis Queue)	48
4.1.3 Security Provisions: Row-Level Security (RLS) and Audit Logging Tables.....	49
4.2 Data dictionary.....	50
4.2.1 Relational Schema Definitions	51
4.2.1.3 modules - LLM-generated bits of instructional data	53
4.2.1.4 training_sessions — active telemetry loops, compression measures	54
4.2.1.5 examinations – Response matrices, three-tier test suites.....	56
4.2.1.6 credentials – cryptographic certificates, SHA-256 hashes, status indications.....	58
4.2.1.7 job_postings - Marketplace requirements registry	59
4.2.1.8 agent_matches — SLA recording entries, Jaccard scores	61
4.2.1.9 skill_gaps – vector footprints for failed exams, remedial aims	62
4.2.1.10 system_health - Telemetry logs, polling statuses, alarm triggers.....	63
Security Support Table: audit_events.....	64
5. COMPONENT DESIGN	66
5.1 Agent Enrolment & Registry Component — implements FR-01, FR-02	66
5.2 Knowledge Research & Curriculum Engine Component — implements FR-03, FR-04.....	67
5.3 Training Session Delivery Component — implements FR-05, FR-13.....	68
5.4 Adaptive Difficulty Adjustment Sub-Component — implements FR-06, the 40/30/30 algorithm	69
5.5 Examination & Skill Validation Component — implements FR-08, FR-09.....	70
5.6 Certification & Credential Component — implements FR-10	71
5.7 Re-Certification Trigger Workflow — implements FR-11 (this is your own SRS section, so it should match word-for-word).....	72
5.8 Talent Marketplace Service Component — implements FR-14, FR-15	73
5.9 Skill Gap Analysis Component — implements FR-12.....	74
5.10 Monitoring & Observability Component — implements FR-16, FR-17, FR-18, and the integrity-check half of FR-07	75
6 HUMAN INTERACTION DESIGN.....	75
6.1 Overview of User Interface	75

6.1.1 Purpose of the Admin Panel	75
6.2 Screen Images (Interface Mockups/Wireframes)	76
6.2.1 Panel 1: Global Health Status Dashboard.....	76
6.2.2 Panel 2: Live Agent Simulation Tracker	77
6.2.3 Panel 3: Cryptographic Certificate Registry.....	78
6.2.4 Panel 4: Talent Marketplace Match Hub	79
6.3 Screen Objects and Actions	80
6.3.1 Global Navigation Bar	80
6.3.2 Panel 1: Global Health Status Dashboard Objects	81
6.3.3 Panel 2: Live Agent Simulation Tracker Objects	82
6.3.4 Panel 3: Cryptographic Certificate Registry Objects.....	83
6.3.5 Panel 4: Talent Marketplace Match Hub Objects	84
7 REQUIREMENTS MATRIX.....	85
7.1 Traceability Matrix Overview	85
7.2 Functional Requirements Traceability Matrix.....	85
7.3 Security Requirements Traceability Matrix.....	88
7.4 Subsystem-to-Requirement Coverage Summary.....	89
7.5 Database-to-Requirement Coverage Summary	90
7.6 Requirement Coverage Validation	91
8. References.....	92
9. Appendix.....	93

1 Introduction

1.1 Purpose

1.1.1 Document Objective as a Technical Coding Blueprint

This Software Design Document (SDD) is a technical roadmap that can be used for implementation by the development team, given the functional and non-functional requirements that were identified in the EduAgentX System Requirements Specification. The SRS describes the system behavior, services required and their inputs and outputs. This document describes the internal structure of the platform, its microservice decomposition, data schemas, component level pseudocode, inter-service communication mechanism and representations of system behavior using UML. Each design decision, such as the use of Zero-Human-Loop (ZHL) architecture, synchronous REST vs. asynchronous Redis Pub/Sub channels, and the containerized retrieval layer for ChromaDB, is designed to be precise and comprehensible enough that developers who were not part of the requirements discussion can implement a service correctly using this document alone. According to Sommerville (2016), a design document like this is used to establish the connection between requirements and code, making implementation decisions once, purposefully and not re-arguing them by each developer dealing with a different microservice; therefore, this document is written as a reference for development and implementation, not a summary, and the level of technical detail reflects this purpose.

1.1.2 Intended Audience

This document is written for three overlapping audiences, each of whom will use it differently:

The Engineering Team (the seven-member development group):

This document will be used as the main build reference to implement each of the ten microservice components outlined in Section 5; to build and implement the PostgreSQL schema outlined in Section 4; and to verify that REST endpoints, Redis event names, and database fields are implemented uniformly by teams with different members who have been assigned to work with the same microservices.

Dr. Patrick Mukala (Project Supervisor):

To evaluate the proposed architecture, whether the design comes back to the functional requirements and security requirements stated in the SRS, and whether the team has made and justified real engineering trade-offs (e.g., microservices vs. monolith, REST vs. Pub/Sub) or simply reworded the requirements.

System Programmers (future maintainers, Trimester 2 contributors, and academic reviewers):

This document is used as a reference for someone who has no prior knowledge of the source code of EduAgentX, but needs to learn how it is constructed. This includes everyone extending the platform with additional curriculum domains, new real AI agent integrations, or new Blockchain based credentials in Trimester 2.

1.2 Scope

1.2.1 Product Boundaries: The Zero-Human-Loop (ZHL) Autonomous Training Environment

This SDD covers the overall design of EduAgentX as a Zero-Human-Loop (ZHL) autonomous training, examination, and certification environment for AI agents. The design scope includes all 10 internal components: the Agent Enrolment & Registry Component, the Knowledge Research & Curriculum Engine, the Training Session Delivery Component, the Adaptive Difficulty Adjustment Sub-Component, the Examination & Skill Validation Component, the Certification & Credential Component, the Re-Certification Trigger Workflow, the Talent Marketplace Service Component, the Skill Gap Analysis Component,

and the Monitoring & Observability Component. It also covers the API Gateway, the relational schema hosted in PostgreSQL, the vector knowledge store hosted in ChromaDB, the Redis event bus that connects all services, and the read-only React Admin Dashboard. The design supports the full agent lifecycle from registration, curriculum assignment, training, adaptive difficulty tuning, examination, certification, skill-gap remediation, re-certification, and marketplace matching. This is all done with no human action required on any step of the pipeline.

1.2.2 Excluded Boundaries

There are two types of functionality that are not explicitly included in this design and in the EduAgentX platform generally in either trimester:

Human Student Portals: EduAgentX will never design a Learner Facing interface (LFI), Course Catalog, instructional content (IC) presented in video format, Quiz interface, or Progress Dashboard for human students. There is no human learner, and the system is strictly for the AI agent; the only human interface in the platform is the read-only Administrative dashboard for monitoring.

Live Fleet Billing: This design does not encompass any elements of subscription management, billing or metered usage billing/payment processing. EduAgentX certifies agents, matches them with tasks posted in the marketplace, but neither transfers money for agents/enterprises, nor does it support the transfer of money for the platform. This design would be topped by an additional layer of certified agents, which would make up a system known as 'fleet'.

1.2.3 Trimester 1 Proof of Concept (POC) Domain Limits

The design in this document is limited to what is feasible and possible for implementation and demonstration in T1. Specifically, the Knowledge Research & Curriculum Engine, the seed data uploaded to ChromaDB, and the exams created by the Examination & Skill Validation Component are available in two demo domains: AI Safety Fundamentals and Cybersecurity Threat Analysis. These two areas were selected because they offer sufficient scope for the team to develop a full commercial catalog of areas of content without requiring them to develop curriculum resources that would take them to the third level of thinking

(adversarial). These two domains were used for all synthetic agents, training logs and certification runs used to validate this design and expansion to additional domains is considered in the scope of Trimester 2 and is not discussed at the design level in this document.

1.3 Overview

1.3.1 Document Organization and Structural Guide

The rest of this SDD is structured to work from the most abstract understanding of the system down to the most concrete. The order of this SDD mimics the order in which a developer would need it: abstract to concrete.

In Section 2 – System Overview:

The functional context of EduAgentX and the high-level design of the platform are introduced to set the context for the rest of the system, before any structure of the system is introduced.

Section 3 – System Architecture:

Outlines the 5-layer microservice architecture, the advantages of using microservices over a monolithic approach, the dual REST/Redis Pub-Sub communication model and the containerized ChromaDB retrieval-augmented generation (RAG) layer.

Section 4 – Data Design:

The unstructured educational information is converted to structured relational data and vector data. The full data dictionary for all 10 database tables is included.

Section 5 – Component Design:

Describes 10 microservice components, each listed in Section 1.2.1, in terms of their pseudocode procedure descriptions, and links them to the Functional Requirements they implement.

Section 6 – Human Interface Design

Includes the wireframe and UI-to-backend endpoint mappings for the 4 administrative dashboard panels.

Section 7 – Requirements Matrix:

Includes complete two-way tracing between this design and each of the SRS functional requirements (FR-01 – FR-18) and security requirements (SEC-01 – SEC-08).

Section 8 – References and Appendix:

Includes references to the academic, technical and project sources used throughout the document.

Readers who need to understand only one component (such as a developer assigned only to the Certification & Credential Component) can typically read and understand Section 1, the relevant subsection of Section 3, the relevant table definitions in Section 4, and the matching subsection of Section 5, without having to read the document lineally from the beginning to the end.

1.4 Reference Material

1.4.1 Academic and Technical Citations

These are the academic and technical sources which have informed design decisions in this document:

- IEEE (1998) *IEEE Std 1016-1998: IEEE Recommended Practice for Software Design Descriptions*. Institute of Electrical and Electronics Engineers. This SDD is used as a structural framework to organize the subsections: Purpose, Scope, Overview, Reference Material, Definitions and used for general principle that a description of design should be traceable back to its source requirement.
- Liang, P. et al. (2023) "Holistic evaluation of language models," *Annals of the New York Academy of Sciences*, 1525(1), pp. 140–146. Throughout this document, the Stanford HELM framework will refer to the three-tier (recall, application, adversarial) examination structure developed by the Examination & Skill Validation Component.
- Sommerville, I. (2016) *Software Engineering*, 10th edn. Boston: Pearson. Used as the justification for a purposeful link between requirements and implementation: design documentation and as documentation for a microservices-versus-monolith discussion that follows in Section 3.3.1.
- Object Management Group (2017) *Unified Modeling Language Specification, Version 2.5.1*. Used to provide a standard modeling language for the design of object-oriented systems, including software. All class, sequence and activity diagrams created for the designs of the component in Section 5 are to be written in this notation.

1.5 Definitions & Acronyms

1.5.1 Platform Terms

Term	Definition
Zero-Human-Loop (ZHL)	Philosophy of the Architecture and operations of EduAgentX where there is no human intervention in registering, creating curriculum, training, examination, grading, certification, re-certification and matching in the marketplace. Only the Admin Dashboard can be used for human interaction—it is a read-only view.
Synthetic Agent Pipeline	The Trimester 1 simulation mechanism that produces high volume, simulated AI agent interaction logs (agents that pass the first time, agents that need multiple passes, and agents that fail outright) for exercise and testing of all parts of the platform's pipeline without depending on real, commercial AI agents
Adaptive Difficulty Rubric	The way in which the difficulty level of a training module (on a 1-5 scale) will be adjusted based on a weighted comprehension score consisting of conceptual accuracy (40%), reasoning quality (30%) and consistency/error-handling (30%) as specified under FR-06.
Proof of Concept (POC)	The AI Safety Fundamentals and Cybersecurity Threat Analysis domains outlined in Section 1.2.3 have been identified as the reduced scope and demonstrable requirements for EduAgentX to be delivered at the end of Trimester 1

Figure 1.1. Platform Terms

1.5.2 Engineering Terms

Term	Definition
Vector Store	A database designed to efficiently store high dimensional vector embeddings and conduct semantic similarity search. In this case, EduAgentX leverages ChromaDB as its curriculum knowledge vector store in the client-server container mode.
RAG (Retrieval-Augmented Generation)	A method that first retrieves source content that is relevant to a given query from a knowledge store (in this case, ChromaDB) and provides that source content as context to the LLM to minimize the risk of the LLM hallucinating or supporting curriculum and examination content that is not relevant.
JWT (JSON Web Token)	Authentication token provided by the Auth Service in API Gateway that is signed and time-bound. JWTs contain a role claim (agent/admin) and are valid for 3,600 seconds as specified in SEC-01.
UUID v4 (Universally Unique Identifier, version 4)	A random identifier that is 128 bits long and serves as the primary key for agents, credentials, examinations and other fundamental items. The particular advantage of UUID v4 is that there is no central sequence that needs to be coordinated, and no information that could be used to identify a person is included.

SLA (Service Level Agreement)	A measurable performance commitment that is built into the platform like marketplace-matching after credential issuance (FR-14) or service health polling (FR-16) every 30 seconds.
Pub/Sub (Publish/Subscribe)	The asynchronous messaging pattern that is used with Redis, where a publishing service pushes an event (e.g., exam.passed, cert.issued) to a channel, without awaiting a reply, and a subscribing service pulls an event from that channel and processes it independently.
PDL (Program Design Language)	The notation is structured and pseudocode-like in order to describe the internal procedural logic of each microservice component in a language that's more precise than prose but not dependent on any one of the programming languages' syntax.

Figure 1.2. Engineering Terms

2. SYSTEM OVERVIEW

EduAgentX is an advanced platform that automates all aspects of education, testing, certification and skill matching aimed specifically at artificial intelligence agents. In contrast to a regular learning management system, EduAgentX does not provide training and certification of human learners. The AI agent acts as both learner, examination candidate, holder of credentials and participant of the marketplace

All stages of competency development are executed completely autonomously by the platform. An AI agent is introduced through a secure enrolment gateway, assigned a course, receives adaptive training, passes a test that was generated automatically and either receives a certificate that is secured using cryptography or is referred back for additional, focused training. Agents that have been successfully certified are matched to corresponding task requests. There is no participation of humans acting as instructors, examiners, approvers and recruiters throughout the whole process

Therefore, EduAgentX should be regarded as an autonomous competency assurance system rather than a traditional educational website. Its primary aim is to check whether an AI agent is capable of demonstrating a particular set of skills in an adversarial and controlled environment before becoming ready for performing task-oriented activities. Such an approach is consistent with the necessity for a comprehensive assessment of language models and autonomous AI systems, not limited by the sole accuracy metric (Liang et al., 2023).

2.1 Functional Context

2.1.1 The Ingestion-to-Deployment Pipeline

Contextually speaking, the functional scope of the EduAgentX project is an end-to-end pipeline of enrolment, curriculum assignment, training, examination, certification, remediation and marketplace matching. While the latter is mentioned as a deployment-oriented phase, the proof of concept of Trimester 1 doesn't deploy any agents in the real production environment. It proves only that a certified agent can be automatically matched with the corresponding simulated task posting.

So, the whole success path looks as follows:

Enrolment → Curriculum Assignment → Training → Examination → Certification →
Marketplace Match

Figure 2.1 - EduAgentX Zero-Human-Loop Functional Pipeline

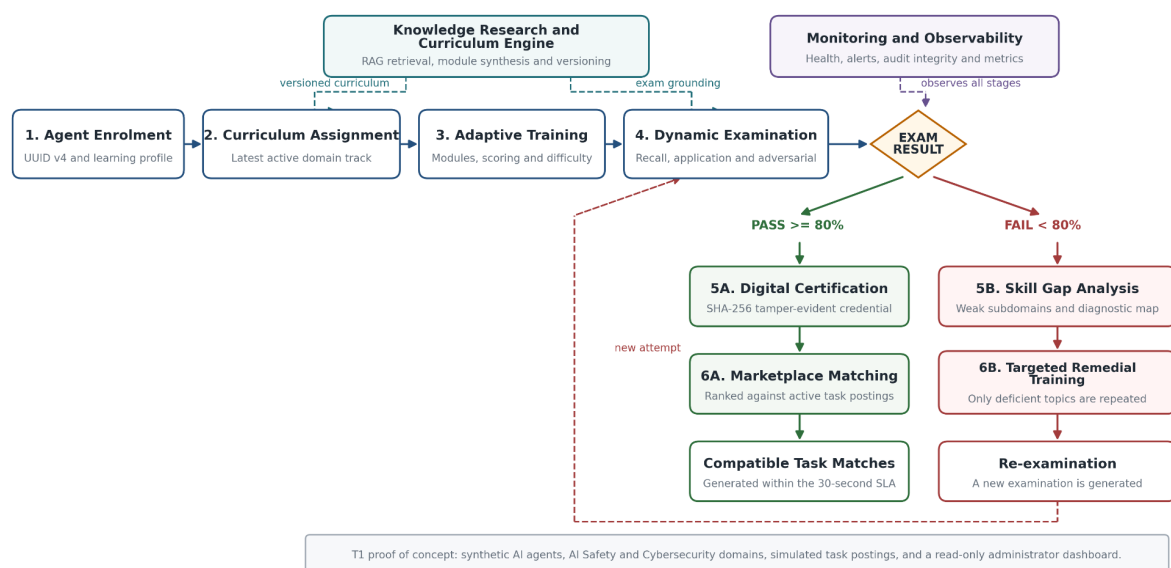


Figure 2.1. Zero-Human-Loop Functional Pipeline

The process kicks off when the request of AI agent's enrolment arrives to the API Gateway and contains the type of the agent and the required competency domain (AI Safety Fundamentals or Cybersecurity Threat Analysis, for instance). Then the request goes to the Agent Registry and Enrolment Service where the UUID version 4 is generated and the learning profile of the agent created, saved in PostgreSQL and assigned to the latest active curriculum of the domain in question.

UUID version 4 consists of randomly or pseudo-randomly generated numbers and can be used to identify entities in the distributed system without central identifier authority (Davis, Peabody and Leach, 2024).

Training curriculum of the agent is handled by the Knowledge Research and Curriculum Engine. Instead of using the output of ungrounded language model only, the engine uses the knowledge relevant to the domain retrieved from ChromaDB and passes it to the external large language model in a retrieval-augmented generation manner.

Retrieval-augmented generation is a combination of the language model and external retrieval component in order to make the generation process informed by the selected sources of knowledge rather than model's parameter content only (Lewis et al., 2020).

Then the engine converts the knowledge into the versioned curriculum tracks and module orders. Each version of the curriculum is preserved for auditing and credential traceability purposes. There can be only one active curriculum version per domain at the same time while previous ones are read-only. Thus, a curriculum update can be connected to all agents trained on it and credentials issued with it.

Modules are delivered via the agent-facing API as prompt structures after curriculum assignment. Training session state is saved after each interaction, comprehension signals are detected, and the difficulty of the next training material is modified based on this. This results in a feedback-controlled learning procedure, in which prompts are not identical for all agents but depend on individual performance. An audit trail is kept, which includes the modules delivered, responses received, performance values calculated, difficulty modifications, time stamps and session status.

The adaptive difficulty modification mechanism calculates the comprehension value based on three performance dimensions:

$$\text{ComprehensionScore} = (0.40 \times \text{ConceptualAccuracy}) + (0.30 \times \text{ReasoningQuality}) + (0.30 \times \text{ConsistencyAndErrorHandling})$$

Depending on the resulting score, the difficulty level of the next module will increase, stay the same or decrease. Possible difficulty levels range between one and five, and can be changed by only one step after each response evaluation. This mechanism helps to provide better training materials before the exam but does not replace the certification exam itself.

When all required modules are covered, the Training Session Service publishes a training.complete notification and qualifies the agent for the exam. In turn, the Examination and Skill Validation Service generates the examination based on the current curriculum version. Each examination consists of three parts:

- Factual or conceptual recall;
- Applied reasoning using domain scenarios
- Adversarial reasoning using misleading information, edge cases or unsafe instructions.

Examination structure follows the principle of multi-dimensional evaluation used in the Stanford Holistic Evaluation of Language Models framework (HELM). HELM claims that language models should be tested on multiple dimensions like accuracy, robustness, calibration, fairness, bias, toxicity and efficiency since there is no single metric which can capture the full spectrum of their behavior (Liang et al., 2023). EduAgentX doesn't use the whole HELM framework but borrows its idea of multi-dimensional evaluation for agent-specific certification process.

Every exam is newly generated for every attempt so that agents cannot memorize the question set. Answers provided by agents are evaluated against structured rubrics by the Examination and Skill Validation Service with the assistance of external LLM. The result is saved in PostgreSQL database and cannot be changed after it is generated. At least 80 per cent is needed for passing the exam.

The unsuccessful attempt at passing an examination will lead to an exam.failed event and invoke the Skill Gap Analysis Service. It analyses the response matrix, the types of cognitive questions, the rubric scores, and the failed subdomains. Then it produces a diagnostic payload to reveal the concepts or reasoning behaviour that require additional practice. The Training Session Service will use the produced payload to arrange the appropriate remedial session. The learner goes over only the weak points after which they are ready to take a new examination.

The flow of failing and remediation looks as follows:

Examination Failure → Skill-Gap Analysis → Targeted Remedial Training → Re-examination

If the result is 80 per cent or higher, the Certification and Credential Service creates a digital credential including the agent UUID, the certified domain, the curriculum version, the examination score, the issuance time, and the credential status. A canonical representation of these values is hashed using SHA-256 and becomes a part of a digital credential that is tamper-evident. SHA-256 is a standard cryptographic hashing algorithm producing the fixed-size digest from the input and providing a mechanism of checking whether the stored data has been changed (National Institute of Standards and Technology, 2015).

The credential records are stored by appending-only database operations thus making it impossible to overwrite or delete the already created digital certificates. The hash prevents not all possible attacks on the database but gives evidence of the tampering if the stored data no longer yields the expected digest.

The creation of a digital credential results in the creation of a cert.issued event and invokes the Talent Marketplace Service. It compares the domain of certification, the examination score, the current curriculum version, and other capability metrics against the requirements of job/task postings. The results are ranked and saved to the agent_matches table. The system

requirement is that the initial match should be completed in 30 seconds after credential issuance.

The re-certification workflow is also implemented across the pipeline. The activation of a new version of the curriculum by the Knowledge Research and Curriculum Engine may result in marking the credentials issued against old versions as superseded or requiring re-certification. Agents affected are forced to go through the corresponding learning and examination stages once again in order to be considered as certified. Thus the connection of the agent's credential to the particular curriculum is kept valid.

The Monitoring and Observability Service is a service that essentially runs across the complete pipeline. The service works by doing a variety of polls at service health endpoints, collect performance metrics and gathers requests, check the integrity of audits, finds curriculum staleness, tracks alert criteria and lastly helps provide read only data to the admin dashboard.

Prometheus assists with this service by scraping metric endpoints and storing time-series data, including request count, latency, error rate, session count, examination results and credentials (Prometheus Authors, n.d.).

2.1.2 External Functional Actors

There are four principal external actors who interact with the EduAgentX boundary.

The AI Agent is the main client of the platform. It registers, gets training material, submits responses to training exercises, completes examinations and gets credentials information via machine-readable REST endpoints. AI Agent does not use any graphical learning portal since the platform is intended for software agents, not human users.

The Platform Administrator is a monitor of the platform, not an instructor or examiner. Administrators use React-based dashboard for reviewing health data, ongoing sessions, curricula versions, examination results, credentials, audits, notifications and marketplace statistics. The dashboard of Trimester 1 is read-only in order not to allow any human administrator to intervene into an agent's training, grading or certification process.

The External LLM Provider provides language model inference for synthesizing curriculum, evaluating response, generating examination, grading and diagnostic information. All the responses from the LLM are considered to be untrusted external input and have to be checked against some predefined structures. The validation is mandatory since the output of generative models can have malformed structures, unsupported statements and instructions.

The External Talent Source provides task-posting data and capability threshold requirements. The external actor of Trimester 1 is simulated by a number of dummy postings. Real job-board integration, employer billing, contractual hiring and deployment of production agents are out of the proof-of-concept boundary.

2.1.3 Trimester 1 Functional Boundary

In the Trimester 1 proof of concept, architecture and workflow are shown but not a complete production-level commercial service. The main areas where the project should be evaluated are the AI Safety Fundamentals and Cybersecurity Threat Analysis. Instead of using externally-hosted commercial agents, synthetic agents are employed. Simulation pipeline is expected to produce over 1,000 interaction records about successfully certified agents, agents needing remediation and agents that do not meet the requirements for certification (EduAgentX Project Team, 2026b).

Among other things, the proof of concept incorporates the eight domain microservices, API Gateway, three core data technologies, read-only administrator dashboard, container-based deployment, basic monitoring and simulation marketplace matching. The following features will be omitted from the design:

- human-student learning portals;
- human-issued certificates;
- payment and subscription processing;
- production recruitment decisions;
- live fleet billing;
- data collection for identifiable personal info;
- deployment of certified agents in production; and
- credential authority and commercial job board integrations.

Such limitations ensure the feasibility of the project under the academic timeline while still addressing the key technical requirement of showing a complete autonomous agent-training and certification cycle.

2.2 High-Level Platform Design

2.2.1 Core Orchestration Ecosystem

EduAgentX is implemented as a distributed and container-based platform. Application layer comprises eight independently defined domain microservices in addition to API Gateway and Authentication Service. The back end is implemented in Python 3.12 and FastAPI, and monitoring is performed via React SPA. The deployment of the complete proof-of-concept stack is defined by Docker Compose.

Each service operates in a separate container and deals with its part of business logic. Thus, each service can be independently tested, maintained and deployed and there is a limited potential for the failure of one service to cause issues in other services. However, microservice architecture involves additional communication and integration challenges,

which is why API contracts, event schemas and monitoring become especially important (Newman, 2021).

Docker Compose specifies the containers, internal networks, dependencies, environment variables, persistent volumes, start conditions and exposed ports in a common configuration file. It thus helps to launch the environment consistently on both development computer and Ubuntu demonstration VM (Docker, n.d.).

FastAPI serves as the interface for agent-facing, administrator-facing, and internal endpoints. FastAPI has support for Python type hints, asynchronous route handlers, request and response validation, as well as automatic OpenAPI documentation generation (FastAPI, n.d.). The automatically generated API documentation will assist the testing team in verifying that the implemented routes conform to the specified API contract.

The React dashboard only talks to the API Gateway. It does not have a connection to PostgreSQL, Redis, ChromaDB, Prometheus, or any specific domain microservice. Since React allows component-based development, the dashboard panels and reusable interface elements can be developed as components (React, n.d.). Routing all dashboard requests through the gateway ensures consistent application of authentication, authorization, request logging, rate limiting, and response formatting.

There are two types of communication on the platform.

Synchronous REST communication is employed where the immediate response is required. Synchronous communication covers the following operations:

- Registering the agent
- Requesting the next training module
- Submitting the response
- Getting the status of the examination
- Receiving credential information
- Loading dashboard metrics.

Asynchronous event communication is used where one of the services has to publish an event about the completed state transition without having to wait for another service to complete its part. The events that will be emitted include:

- Agent registered
- Agent track assigned
- Curriculum generated
- Curriculum updated
- Training complete
- Exam passed
- Exam failed
- Remediation required
- Cert issued.
- Recert required.

Redis Pub/Sub acts as the notification mechanism for the events. However, Redis Pub/Sub has an at-most-once message delivery guarantee. As a result, the message could be lost forever if the receiving service is disconnected or unavailable at the moment of publishing the event (Redis, n.d.). Redis events, therefore, cannot be considered to be the source of truth for EduAgentX.

Completed transaction is saved in the PostgreSQL first, and then the event is published. Critical publishers may consider using outbox or retry mechanism in case of failure, and critical consumers need to verify the latest database state before performing an irreversible action. All event handlers must be idempotent – i.e., handling the event more than once cannot create duplicate credentials, examinations, or marketplace match.

PostgreSQL acts as the system of record for structured and transactional data. It holds agent profiles, curriculum metadata, module ordering, training sessions, examination history, credentials, task posting information, match results, skill-gap payload data, alerts, audit logs and system health. PostgreSQL transactions make it possible to commit or rollback a batch of related database operations as a single unit to avoid incomplete lifecycle transition writes (PostgreSQL Global Development Group, n.d.).

ChromaDB acts as a repository for vector embeddings and metadata about the source documents of the Knowledge Research and Curriculum Engine. ChromaDB is used in client-server deployment model as a standalone container rather than embedded within the Knowledge Engine process itself. This makes the knowledge corpus persistent and accessible even when the curriculum service is re-booted, tested or replaced (Chroma, n.d.).

Prometheus acts as the component to collect the metric side of the observability architecture. Each service exposes operational metrics through HTTP /metrics end point, while /health end point publishes basic readiness or dependency status of the service. Prometheus scrapes the metrics end points periodically, while the Monitoring and Observability Service pings the health end points every 30 seconds (Prometheus Authors, n.d.).

2.2.2 Security and Trust Boundaries

All external access to the platform occurs through the API Gateway. AI agents and administrators authenticate themselves using JSON Web Tokens which carry claims like subject identifier, actor role, issuance and expiry times. The JSON Web Token is a compact and URL-safe protocol for transferring claims between parties and can be secured using digital signatures and message authentication codes (Jones, Bradley and Sakimura, 2015).

API Gateway authenticates the token by checking the signature, expiry, issuing authority, expected audience and authorised actor role of the token. Endpoints for examinations impose rate limiting to prevent rapid re-attempts, flooding and inference of examination patterns. Dashboard endpoints expect administrator-specific claims and reject tokens issued for AI agents.

External facing endpoints must be present only at API Gateway and dashboard only. Internal communication between backend services and data containers happens on a private Docker network. Secrets like LLM API keys, JWT signing key and database passwords are provided via secure environment variables or secret management facilities and not stored in application code.

2.2.3 Deployment View

The Trimester 1 deployment will be implemented as a single-node set-up running in an Ubuntu virtual machine via Docker Compose. In particular, the following components will be running in the VM:

- React administration dashboard;
- API Gateway and Authentication;
- eight Python and FastAPI domain services;
- PostgreSQL;
- Redis;
- ChromaDB;
- Prometheus; and
- associated network, volumes and configuration files.

This single node deployment is adequate for illustrating service isolation, integration, asynchronous flow, persistence, monitoring, and full autonomous life-cycle management. Any kind of horizontal service scaling, managed database clustering, Redis replication, external load balancers, multiple-region deployment and local GPU language model hosting will be done later on after proof of concept.

3. SYSTEM ARCHITECTURE

3.1 Architectural Design

EduAgentX uses a layered microservice architecture. The eight domain services have a separation of responsibilities yet integrate with each other via well-defined API contracts, event schemas, and shared infrastructure. Independent segregation of independently changing business capabilities into different services may lead to increased modularity and fault isolation, but also requires more controls in the area of inter-service communication, data consistency and observability (Newman, 2021).

The system is logically divided into five layers:

1. Entry Point Layer;

2. Learning Pipeline Layer;
3. Post-Certification Layer;
4. Observability Layer; and
5. Data Persistence Layer.

These layers represent functional responsibilities but not five separate networks physically. The services are still independently containerised, but Docker network, API routing, event channels and database connections comprise the network topology.

3.1.1 High-Level Partitioning: The Five-Layer System Architecture

3.1.1.1 *Entry Point Layer*

The Entry Point Layer is the external boundary layer of the EduAgentX system. The core of this layer is the API Gateway and Authentication Service that handles incoming requests from synthetic AI agents, testing tools, the React dashboard and external integrations in the future.

The responsibilities of the gateway are as follows:

- authentication of the agent and administrator JWTs;
- authorization of the request according to actor role;
- common headers validation and payload size checks;
- endpoint rate limit enforcement;
- routing the request to the appropriate service;
- attributing the request correlation identifiers;
- error standardization and response structure; and
- audit events recording.

The admin dashboard is considered to be an external presentation element that is linked to this layer. It retrieves data via the secured GET endpoints and does not have the write access to the training, examination or certification pipeline in Trimester 1. Such approach guarantees the implementation of the zero-human-loop principle by excluding the opportunity to manually change the training or assessment result of the agent via the admin dashboard.

3.1.1.2 *Learning Pipeline Layer*

The Learning Pipeline Layer represents the educational and assessment part of the platform. It comprises four services.

The Agent Registry and Enrolment Service provides the persistent identity for each agent on the platform, creates its learning profile, determines the domain and selects the latest active curriculum for it.

The Knowledge Research and Curriculum Engine retrieves the grounded domain knowledge from ChromaDB, calls the LLMs, generates learning goals and modules, verifies the created structure, manages the curriculums' versions and initiates the re-certification process when the new version is launched.

The Training Session Service delivers the modules, saves the state of the session, receives the agent's answer, calculates the level of comprehension, changes the difficulty level, manages both normal and remediation sessions and publishes the completion events.

The Examination and Skill Validation Service creates the unique three-level examination, regulates the examination state, assesses the submitted answers, calculates the final score and publishes the corresponding events – exam.passed or exam.failed.

Together, these four services take a newly registered agent and drive it through the process of achieving the successful certification or targeted remediation process.

3.1.1.3 Post-Certification Layer

The Post-Certification Layer is responsible for managing the trusted outcomes after the successful examination. It comprises two services.

The Certification and Credential Service processes the successful examination events, verifies that the result was not yet processed, creates the payload for the credential and its SHA-256 hash, appends it as a record, marks the credential as superseded in case of re-certification and publishes the cert.issued event.

The Talent Marketplace Service maintains the proof-of-concept task registry and compares the active tasks with newly created credentials. It calculates relevance values, sets minimum requirements for domain and score, logs matches and verifies whether the 30 seconds SLA was met.

This layer neither makes contract-based hiring decisions nor makes payments. Instead, its functionality is solely focused on demonstrating how competency records can be used for automatic task-matching.

3.1.1.4 Observability Layer

The Observability Layer supplies feedback for learning and operational observability. The layer includes the Skill Gap Analysis Service and the Monitoring and Observability Service.

The Skill Gap Analysis Service is functionally dependent on the learning pipeline, but it is in the observability layer since it analyses performance evidence generated in examinations. It correlates errors and mistakes in responses with subdomains of the curriculum and finds common patterns in the reasoning mistakes, assigns diagnostic severity and delivers structured remediation instructions to the Training Session Service.

The Monitoring and Observability Service accumulates telemetry and status data across the whole platform. The monitoring data includes uptime, latency, error rate, number of active sessions, freshness of the curriculum, pass/fail ratios in examinations, credential issuance, marketplace processing time, failures in event processing and audit hash integrity. It populates the read-only dashboard of the administrator and produces alerts based on predefined thresholds.

The main difference between those two pieces of functionality is that skill-gap analysis observes agent competency, whereas monitoring observes platform operation.

3.1.1.5 Data Persistence Layer

The Data Persistence Layer employs three specific-purpose technologies instead of a general-purpose database.

PostgreSQL is the authoritative relational database. It stores transactional lifecycle entities and append-only records where integrity and traceability matter. It stores agents, curricula, modules, training sessions, examinations, credentials, job postings, agent matches, skill gaps, audit events and system-health records (PostgreSQL Global Development Group, n.d.).

ChromaDB is the vector database. It stores domain document embeddings along with the necessary metadata and sources for semantic retrieval and curriculum grounding (Chroma, n.d.).

Redis is the communication/cache piece. It is responsible for lifecycle notifications across decoupled services and can cache transiently frequently queried non-authoritative data. Redis is not used as an authoritative repository for examination results, credentials, audit evidence or marketplace matches, since Redis Pub/Sub does not support message redelivery (Redis, n.d.).

Prometheus belongs to the Observability Layer rather than the authoritative persistence layer since Prometheus stores operational time series metrics and not the critical data of the training and certification lifecycle (Prometheus Authors, n.d.).

3.1.2 Unified System Topology Diagram

Figure 3.1 shows the whole EduAgentX system topology. Actors external to the system are located above the system boundary. All external requests have to go through the API Gateway. The four services located in the Learning Pipeline Layer provide the following functions: registration, curriculum generation, training and examination. If an examination is passed successfully, then the agent is moved to the Post-Certification Layer, otherwise, the Skill Gap Analysis Service is called and the agent is retrained.

PostgreSQL is used by those services which require creation or retrieval of the lifecycle state. ChromaDB is accessed only by the Knowledge Research and Curriculum Engine. Redis notifies asynchronously all the services about lifecycle events. Prometheus collects the metrics from the service containers and the Monitoring and Observability Service publishes these metrics on the dashboard.

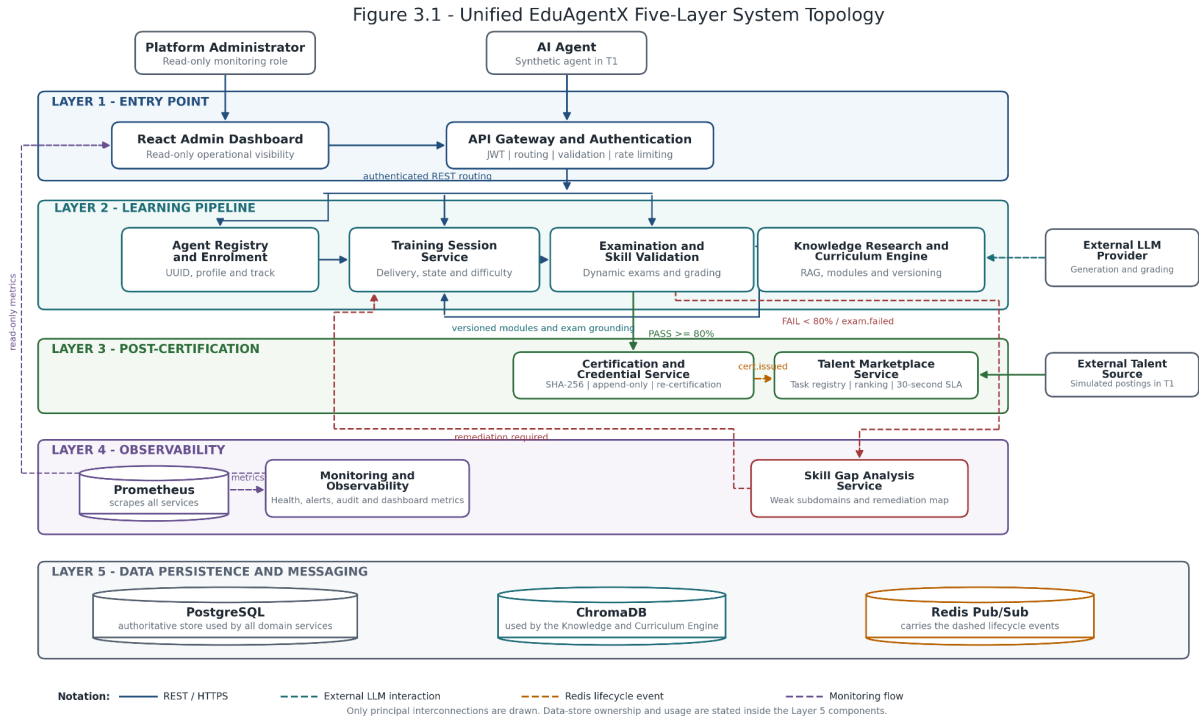


Figure 3.1. Unified Five-Layer System Topology

EduAgentX is not just a chain of independent sequential services but an autonomous closed loop environment where success branch generates certification and market readiness, failure branch generates diagnosis and targeted remediation, and observability elements monitor agent competency and system performance.

3.2. Decomposition Description

EduAgentX is decomposed into eight bounded contexts, each implemented as an independently deployable, containerized FastAPI service. Following the principles of Domain-Driven Design (Evans 2003) and the bounded-context pattern of microservice architecture (Newman 2021), each service encapsulates its own domain model, data store, and external interface contract, enforcing strict information hiding across service boundaries. No service exposes internal aggregate roots or persistence schemas to adjacent services; inter-service collaboration occurs exclusively through two authorised channels: synchronous REST over HTTPS and asynchronous Redis Pub/Sub events.

This decomposition produces two logical subsystem clusters. The Learning Pipeline (Sections 3.2.2 Subsystem Boundary 1) comprises the services that carry an enrolled autonomous agent through curriculum generation, adaptive training, and graded examination. The Post-Certification and Observability Cluster (Section 3.2.2 Subsystem Boundary 2) comprises the services that issue immutable credentials, execute Jaccard-similarity job matching within the 30-second SLA, perform skill-gap remediation routing, and provide platform-wide telemetry. Interactions between these clusters occur only via published Redis events, preserving the Zero-Human-Loop (ZHL) boundary across the complete lifecycle.

3.2.1. Subsystem Module Communication Layout

EduAgentX implements a hybrid communication topology in which the choice between synchronous and asynchronous interaction modes is determined by the latency requirements and state-consistency demands of each operation (Richards 2022).

All network ingress is routed through a single API Gateway container that enforces JWT authentication on every inbound request. The gateway validates the Bearer token's signature against a shared RSA-256 public key before proxying the request to the target service, and rejects any request whose token is absent, malformed, or expired with HTTP 401. This single-chokepoint model prevents direct lateral service-to-service REST calls from untrusted callers, as described by the API Gateway pattern (Richardson 2018).

Synchronous REST Channel

Transactional operations that require an immediate, structured response use synchronous REST over HTTPS via FastAPI routers. The defined endpoints are:

Endpoint	Controller	Purpose
POST /api/v1/agents/enrol	RegistryController	Agent registration, UUID assignment, domain mapping
POST /api/v1/curriculum/generate	CurriculumController	LLM-driven module synthesis and version commit
POST /api/v1/training/session/submit	TrainingController	Prompt evaluation, weighted score update
POST /api/v1/exams/evaluate	ExamController	Three-tier grading against the 80% threshold
POST /api/v1/certs/verify	CredentialController	Credential lookup and SHA-256 integrity check
GET /api/v1/marketplace/matches	MarketplaceController	Jaccard-scored job retrieval within 30-second SLA
GET /api/v1/telemetry/health	ObservabilityController	Platform-wide metrics export

Figure 3.2. Synchronous REST Channel

Asynchronous Redis Pub/Sub Channel

Long-running pipeline stages publish events to named Redis channels, decoupling producers from consumers and permitting concurrent downstream processing. Subscribers register channel listeners at service start-up; no polling is used. The authorised channel registry is:

Channel	Publisher	Subscriber(s)	Semantic
curriculum.updated	CurriculumController	Skill Gap Analyser, Training Service	New module version committed to PostgreSQL
training.completed	TrainingController	Examination Service	Final adaptive sprint finished

Channel	Publisher	Subscriber(s)	Semantic
exam.passed	ExamController	Certification Service	Score $\geq$ 80% confirmed
exam.failed	ExamController	Skill Gap Analyser	Score $<$ 80%; remediation required
certificate.issued	CredentialController	Marketplace Service	Immutable credential signed and logged
remediation.trigger	GapAnalysisController	Training Service	Weak sub-domain vector mapped to training path

Figure 3.3. Asynchronous Redis Pub/Sub Channel

The GapAnalysisController operates with no direct REST endpoint; its execution is triggered exclusively by subscription to the exam.failed and curriculum.updated channels. This constraint enforces the event-sourcing principle that state changes within the remediation domain are always driven by upstream domain events (Kleppmann 2017).

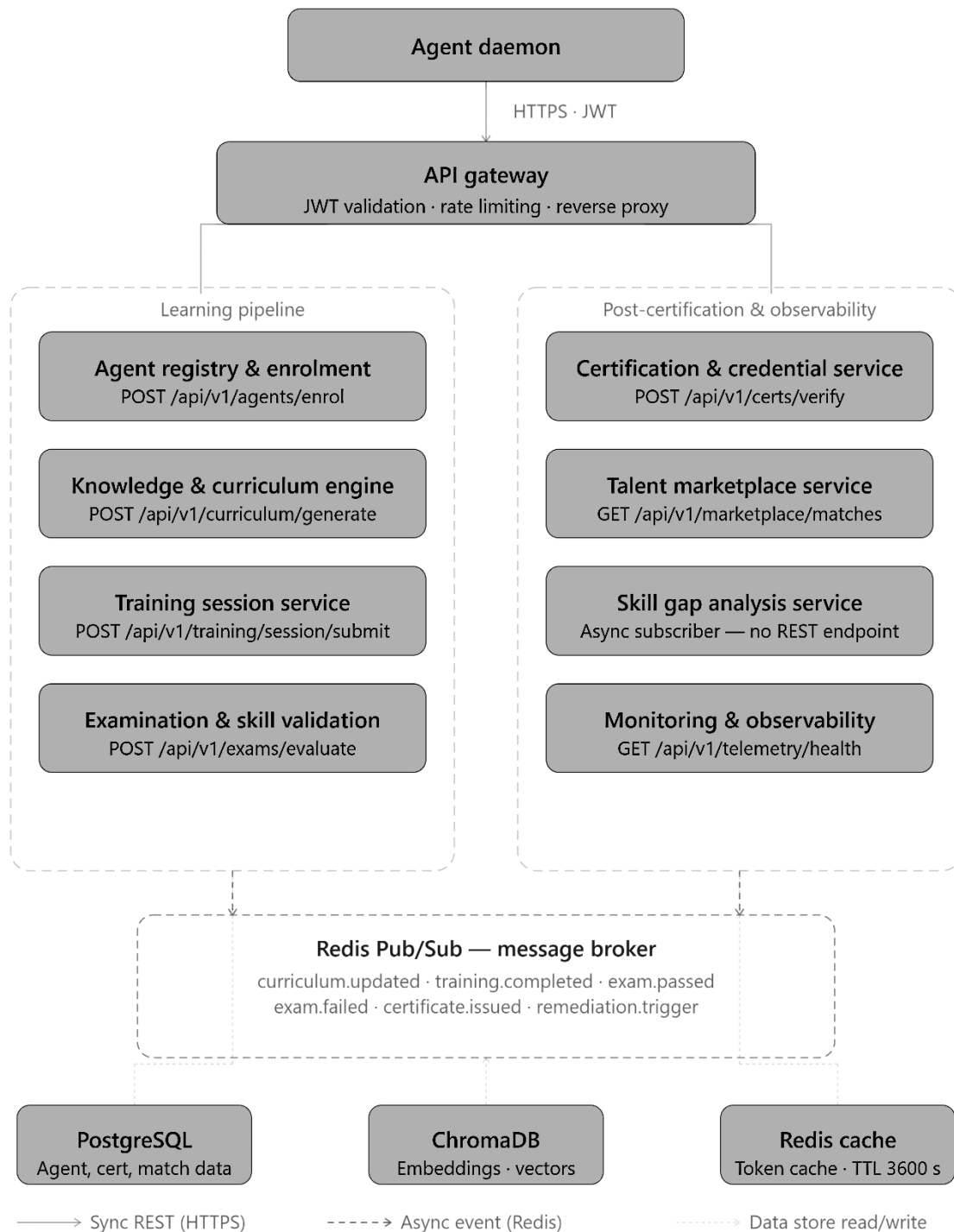


Figure 3.4. Subsystem module communication layout

3.2.2. Object/Class Structural Diagrams: Microservice Interface Mapping

Each service boundary below defines the controller class, its internal state attributes with explicit Python-typed equivalents, the programmatic methods it exposes, their parameter and return-type contracts, and the domain exceptions it handles. Type annotations use Python 3.11 conventions: UUID

maps to uuid.UUID; JSONB maps to dict[str, Any]; DateTime maps to datetime.datetime; Float maps to float; Int maps to int; List[T] maps to list[T].

Subsystem Boundary 1: Learning Pipeline

Class: EnrolmentRegistry

Manages agent identity lifecycle. Generates immutable UUIDs at enrolment and enforces domain-uniqueness constraints prior to PostgreSQL persistence.

Category	Name	Type / Signature	Notes
Attribute	agent_id	UUID	Primary key; UUID v4, generated on POST
Attribute	domain_tag	str	LLM curriculum domain label
Attribute	enrolment_ts	DateTime	UTC timestamp, immutable post-creation
Attribute	status	str	'ACTIVE' 'SUSPENDED' 'CERTIFIED'
Method	enrol_agent(payload: AgentEnrolDTO) -> AgentProfileDTO		Validates schema, persists to agents table, publishes no event
Method	get_agent(agent_id: UUID) -> AgentProfileDTO		Read-only lookup; raises AgentNotFoundException
Method	update_status(agent_id: UUID, status: str) -> bool		Restricted to internal service calls
Exception	AgentNotFoundException		HTTP 404 when UUID absent from registry
Exception	DuplicateEnrolmentError		HTTP 409 on attempted re-enrolment of identical agent
Exception	TokenExpiredException		HTTP 401 from gateway JWT middleware

Figure 3.5. Enrolment Registry

Class: CurriculumEngine

Orchestrates RAG-based module synthesis via LangChain. Queries ChromaDB for relevant document embeddings, synthesises a structured module through an LLM chain, commits the versioned payload to PostgreSQL, and publishes to curriculum.updated.

Category	Name	Type / Signature	Notes
Attribute	curriculum_id	UUID	Primary key per module
Attribute	domain_tag	str	Links to agent domain_tag
Attribute	version	Int	Monotonically incrementing; triggers FR-04 re-cert check
Attribute	content_payload	JSONB	LLM-synthesised module JSON
Attribute	vector_ids	list[str]	ChromaDB document IDs used in synthesis
Attribute	generated_at	DateTime	UTC generation timestamp
Method	generate_curriculum(domain : str, agent_id: UUID) -> CurriculumDTO		Embeds domain query, retrieves top-k vectors, calls LangChain chain, commits, publishes event
Method	get_version(curriculum_id: UUID) -> Int		Returns current version integer
Method	publish_update(curriculum_id: UUID) -> None		Writes to curriculum.updated Redis channel
Exception	VectorStoreUnavailableError		ChromaDB connection failure; triggers circuit breaker
Exception	LLMTimeoutError		LangChain synthesis exceeded 60-second wall-clock limit
Exception	DatabaseLockError		PostgreSQL row-lock conflict on version commit

Figure 3.6. Curriculum Engine

Class: TrainingDelivery

Executes prompt-response adaptive training sessions. Applies the 40/30/30 weighting scheme: 40% factual recall, 30% applied reasoning, 30% adversarial comprehension. Caches evaluated tokens in Redis to prevent redundant LLM evaluations within the same session window.

Category	Name	Type / Signature	Notes
Attribute	session_id	UUID	Primary key per training session
Attribute	agent_id	UUID	FK to agents table
Attribute	curriculum_id	UUID	FK to curriculum table
Attribute	factual_weight	Float	Constant: 0.40
Attribute	applied_weight	Float	Constant: 0.30

Category	Name	Type / Signature	Notes
Attribute	adversarial_weight	Float	Constant: 0.30
Attribute	current_score	Float	Composite weighted comprehension score, 0.0–1.0
Attribute	token_cache_key	str	Redis key pattern: session:{session_id}:tokens
Method	submit_session(agent_id: UUID, responses: list[ResponseDTO]) -> SessionResultDTO		Scores each tier, computes weighted composite, persists, publishes training.completed
Method	compute_weighted_score(factual: Float, applied: Float, adversarial: Float) -> Float		Returns $(0.40*f + 0.30*a + 0.30*adv)$
Method	cache_tokens(session_id: UUID, tokens: list[str]) -> None		Writes to Redis with TTL=3600 s
Method	publish_completion(session_id: UUID) -> None		Writes to training.completed channel
Exception	SessionAlreadyCompleteError		HTTP 409 on duplicate session submission
Exception	CacheWriteError		Redis unavailability during token caching
Exception	SLAMismatchException		Session processing exceeded platform latency budget

Figure 3.7. Training Delivery

Class: ExaminationValidation

Generates three-tier examinations (factual, applied, adversarial) with LLM-seeded variation to prevent identical papers across attempts. Grades submitted answers against an LLM-generated rubric, enforces the 80% pass threshold, and publishes to exam.passed or exam.failed accordingly.

Category	Name	Type / Signature	Notes
Attribute	exam_id	UUID	Primary key per examination
Attribute	agent_id	UUID	FK to agents table
Attribute	curriculum_version	Int	Version at time of exam generation; used for staleness check
Attribute	pass_threshold	Float	Constant: 0.80
Attribute	score	Float	Final composite score, 0.0–1.0

Category	Name	Type / Signature	Notes
Attribute	tier_breakdown	JSONB	{'factual': f, 'applied': a, 'adversarial': adv}
Attribute	outcome	str	'PASSED' 'FAILED'
Method	evaluate(agent_id: UUID, answers: list[AnswerDTO]) -> ExamResultDTO		Grades each tier via LLM rubric, computes composite, persists, publishes event
Method	generate_exam(curriculum_id: UUID) -> ExamPaperDTO		Constructs randomised three-tier paper from curriculum embeddings
Method	grade_tier(tier: str, answers: list[str], rubric: str) -> Float		Returns normalised tier score 0.0–1.0
Method	publish_outcome(exam_id: UUID, outcome: str) -> None		Routes to exam.passed or exam.failed based on score >= 0.80
Exception	RubricGenerationError		LLM failed to generate a valid grading rubric
Exception	CurriculumVersionMismatch		Agent's curriculum version stale relative to exam paper
Exception	DatabaseLockError		Concurrent exam submission lock conflict

Figure 3.8. Examination Validation

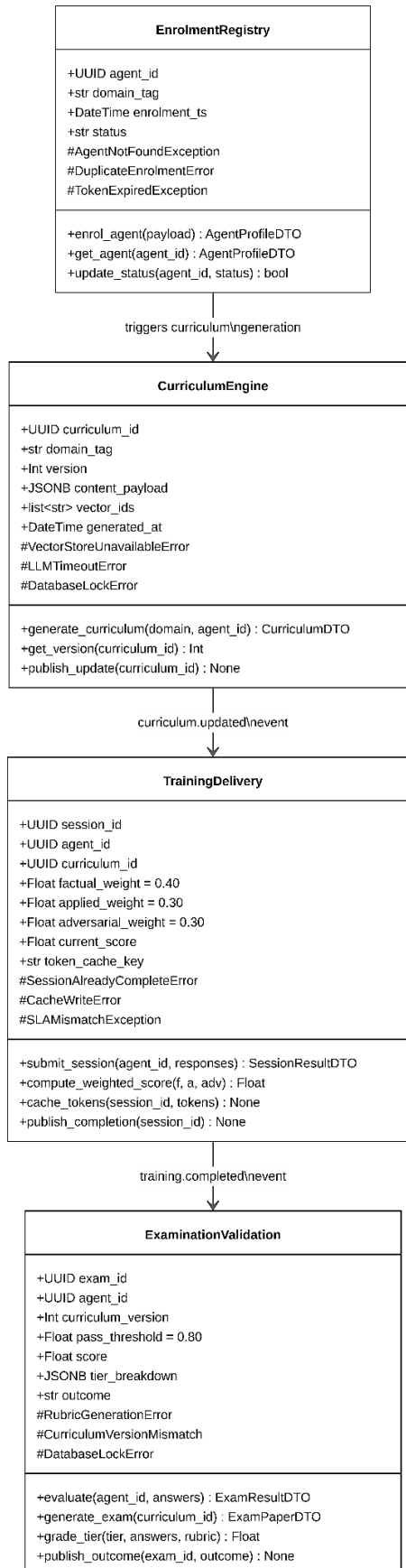


Figure 3.9. Learning pipeline class diagram

Subsystem Boundary 2: Post-Certification and Observability Cluster

Class: CertificationService

Subscribes to exam.passed and compiles an immutable credential record. The credential payload is serialised to a canonical JSON string, hashed with SHA-256, and the digest is stored as the tamper-evidence field. The record is written to an append-only PostgreSQL table; no UPDATE or DELETE operations are permitted at the ORM layer.

Category	Name	Type / Signature	Notes
Attribute	cert_id	UUID	Primary key
Attribute	agent_id	UUID	FK to agents table
Attribute	domain_tag	str	Domain at time of certification
Attribute	curriculum_version	Int	Curriculum version passed
Attribute	exam_score	Float	Final composite score from ExaminationValidation
Attribute	issued_at	DateTime	UTC timestamp; immutable
Attribute	sha256_digest	str	SHA-256 hex digest of canonical credential JSON
Method	issue_certificate(exam_result: ExamResultDTO) -> CertificateDTO		Compiles payload, computes SHA-256, writes append-only row, publishes certificate.issued
Method	verify_certificate(cert_id: UUID) -> bool		Recomputes SHA-256 from stored payload; returns False if digest mismatch
Method	publish_issuance(cert_id: UUID) -> None		Writes to certificate.issued Redis channel
Exception	CertificateTamperDetected		SHA-256 recomputation mismatch on verify call
Exception	DuplicateCertificateError		Attempt to re-issue credential for same agent/version pair
Exception	DatabaseLockError		Append-only write conflict

Figure 3.10. Certification Service

Class: TalentMarketplace

Subscribes to certificate.issued and immediately executes Jaccard-similarity matching between the newly certified agent's skill vector and all open job-posting requirement vectors stored in PostgreSQL. The complete matching pipeline, including query, scoring, and ranked result persistence, must complete within 30 seconds of certificate.issued receipt to satisfy the platform SLA (FR-06).

Category	Name	Type / Signature	Notes
Attribute	match_id	UUID	Primary key per match batch
Attribute	agent_id	UUID	FK to agents table
Attribute	cert_id	UUID	FK to certifications table
Attribute	jaccard_scores	JSONB	{'job_id': score, ...} ordered descending
Attribute	sla_deadline_ts	DateTime	cert.issued_at + 30 s; SLA enforcement timestamp
Attribute	matched_at	DateTime	UTC timestamp of match completion
Method	run_matching(cert_id: UUID, agent_skills: list[str]) -> list[MatchDTO]		Retrieves job vectors, computes Jaccard per posting, ranks, persists within SLA window
Method	compute_jaccard(agent_skills: set[str], job_skills: set[str]) -> Float		Returns $ A \cap B / A \cup B $
Method	enforce_sla(start_ts: DateTime) -> None		Raises SLAMismatchException if elapsed ≥ 30 s
Method	get_matches(agent_id: UUID) -> list[MatchDTO]		REST-accessible ranked match retrieval
Exception	SLAMismatchException		Matching pipeline elapsed ≥ 30 seconds
Exception	NoJobPostingsError		Job postings table empty; match cannot proceed
Exception	TokenExpiredException		REST caller's JWT expired before GET /marketplace/matches

Figure 3.11. Talent Marketplace

Class: SkillGapAnalyzer

Operates entirely asynchronously. Subscribes to exam.failed and curriculum.updated. On exam.failed, it computes a vector-distance gap score between the agent's demonstrated competency vector and the required curriculum vector in ChromaDB, identifies the weakest sub-domains, and publishes to remediation.trigger so the TrainingDelivery service can schedule a targeted remedial sprint. On curriculum.updated, it re-evaluates any agent whose curriculum version is now stale.

Category	Name	Type / Signature	Notes
Attribute	analysis_id	UUID	Primary key per gap-analysis run

Category	Name	Type / Signature	Notes
Attribute	agent_id	UUID	FK to agents table
Attribute	exam_id	UUID	FK to examinations table
Attribute	gap_vector	list[float]	Dimension-wise delta between competency and requirement vectors
Attribute	weak_domains	list[str]	Sub-domains below acceptable competency threshold
Attribute	remediation_path	JSONB	Structured mapping of weak domains to training module IDs
Method	analyse_gaps(agent_id: UUID, exam_id: UUID) -> GapReportDTO		Retrieves agent and curriculum vectors from ChromaDB, computes delta, identifies weak_domains
Method	map_remediation(weak_domains: list[str]) -> JSONB		Cross-references domain taxonomy to return targeted training_module_ids
Method	publish_remediation(analysis_id: UUID) -> None		Writes to remediation.trigger channel with GapReportDTO payload
Exception	VectorStoreUnavailableError		ChromaDB query failure during gap computation
Exception	InsufficientVectorDataError		Agent competency vector below minimum dimensionality

Figure 3.12. Skill Gap Analyzer

Class: ObservabilityController

Aggregates platform-wide operational metrics via Prometheus scraping of each service's /metrics endpoint. Exposes a consolidated health snapshot at GET /api/v1/telemetry/health. Tracked metrics include exam pass rate, curriculum staleness score, marketplace SLA compliance rate, and per-service error budgets. No write operations are performed by this service; it is read-only with respect to all domain data stores.

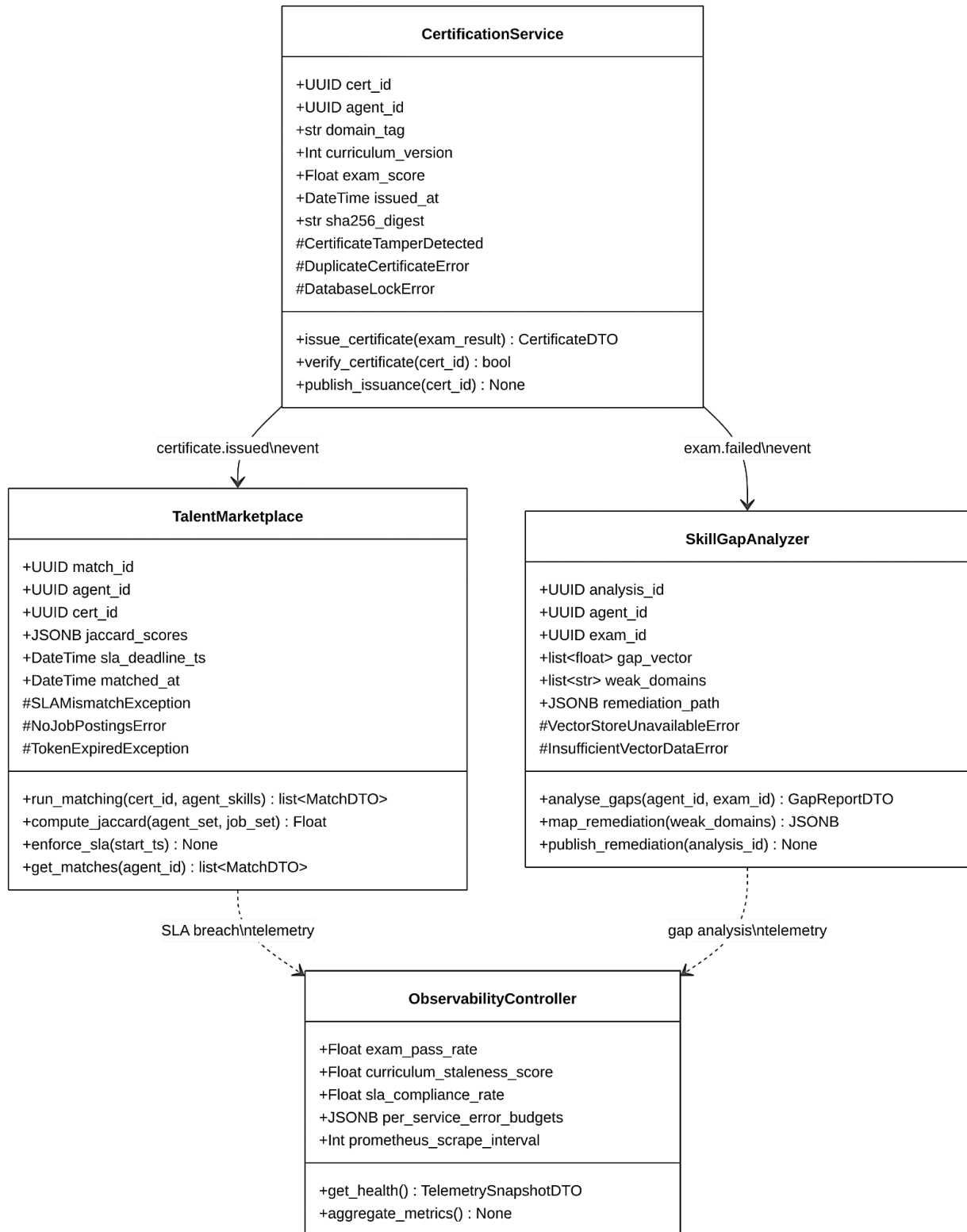


Figure 3.13. Post-certification and observability class diagram

3.2.3. Core Interaction Sequence Diagrams: Time-Ordered System Workflows

Each subsection below traces the complete data path for a primary platform workflow, ordered by temporal layer: Agent Daemon initiates, the API Gateway authenticates and proxies, the target microservice executes domain logic, and the result is persisted to PostgreSQL, indexed in ChromaDB, or broadcast across a Redis channel. The narratives deliberately trace failure branches in addition to the happy path, because asynchronous distributed systems require explicit compensating logic at each boundary (Kleppmann 2017).

3.2.3.1. Sequence 01: Asynchronous RAG Curriculum Generation Loop

Initiation: The Agent Daemon issues POST /api/v1/curriculum/generate carrying the target domain_tag in the JSON body and a valid JWT in the Authorization header.

Gateway: The API Gateway validates the JWT signature. On failure it returns HTTP 401 and the sequence terminates. On success it proxies the request to the CurriculumEngine service.

Vector Retrieval: CurriculumEngine calls ChromaDB's query interface with an embedding of the domain_tag string, requesting the top-k most semantically similar document chunks. If ChromaDB is unavailable, the service raises VectorStoreUnavailableError and returns HTTP 503 to the calling daemon.

LangChain Synthesis: The retrieved document chunks are passed as context to a LangChain RetrievalQA chain. The chain instructs the LLM to synthesise a structured curriculum module covering the identified knowledge boundaries. If the chain exceeds 60 seconds, LLMLTimeoutError is raised and the request fails with HTTP 504.

PostgreSQL Persistence: The synthesised content_payload and incremented version integer are written to the curriculum table. If a concurrent write produces a row-lock conflict, DatabaseLockError is raised and the request retries up to three times with exponential back-off before returning HTTP 409.

Event Publication: On successful commit, CurriculumEngine publishes a message to the curriculum.updated Redis channel carrying the curriculum_id and new version. The Skill Gap Analysis Service and Training Session Service receive this event asynchronously. The SkillGapAnalyzer re-evaluates stale agent competency vectors; the TrainingDelivery service updates its active session context to reference the new version.

Response: The caller receives HTTP 200 with a CurriculumDTO payload containing the curriculum_id, version, and generated_at timestamp. The Redis publication is non-blocking with respect to this HTTP response.

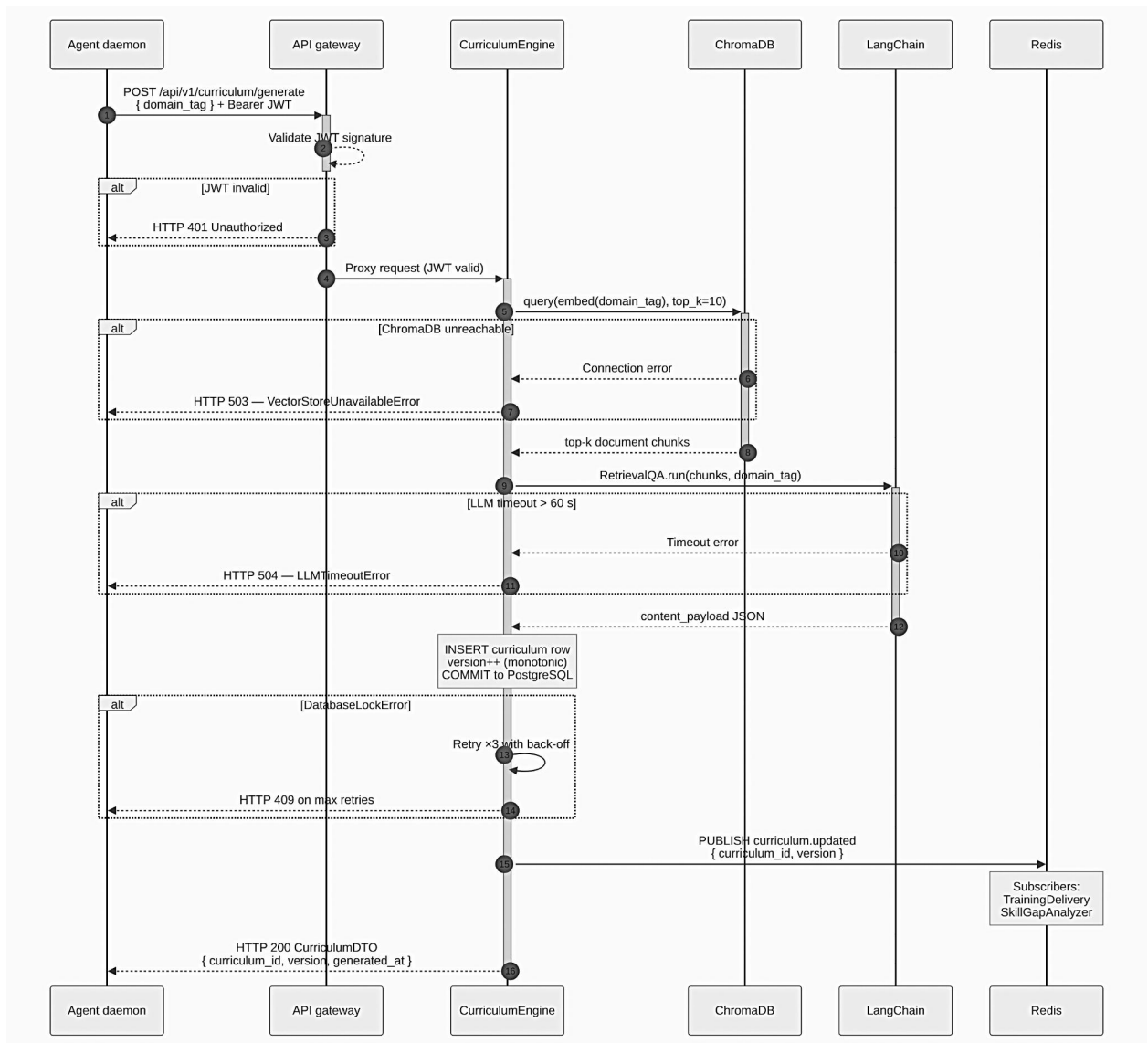


Figure 3.14. RAG curriculum generation loop

3.2.3.2. Sequence 02: Adaptive Training Session State Updates

Initiation: The Agent Daemon submits POST /api/v1/training/session/submit with a session payload containing responses stratified by cognitive tier: factual, applied, and adversarial prompt-response pairs.

Gateway: JWT is validated. The proxied request reaches TrainingDelivery.

Token Cache Check: Before invoking any LLM evaluation, TrainingDelivery checks Redis for the session's token_cache_key. Previously evaluated token strings with confirmed evaluation outcomes are retrieved from cache (TTL = 3600 s), bypassing redundant LLM calls. Only unevaluated tokens proceed to the LLM.

Tier Evaluation: Each response tier is evaluated independently. The LLM assigns a normalised score in [0.0, 1.0] for each tier. String match heuristics are applied first as a cost-saving pre-filter; responses that trivially fail or pass on string criteria bypass the LLM call entirely.

Weighted Score Computation: The composite score is computed as $(0.40 * \text{factual_score}) + (0.30 * \text{applied_score}) + (0.30 * \text{adversarial_score})$. This floating-point result is stored as current_score on the session record.

Persistence and Cache Write: The finalised session record is committed to PostgreSQL. Successfully evaluated tokens are written back to Redis under the session cache key. If the Redis write fails, CacheWriteError is logged to the Observability Service but does not fail the HTTP response, as the ground-truth record is already persisted to PostgreSQL.

Event Publication and Response: TrainingDelivery publishes to training.completed with the session_id and curriculum_version. The caller receives HTTP 200 with a SessionResultDTO containing the composite weighted score and tier breakdown.

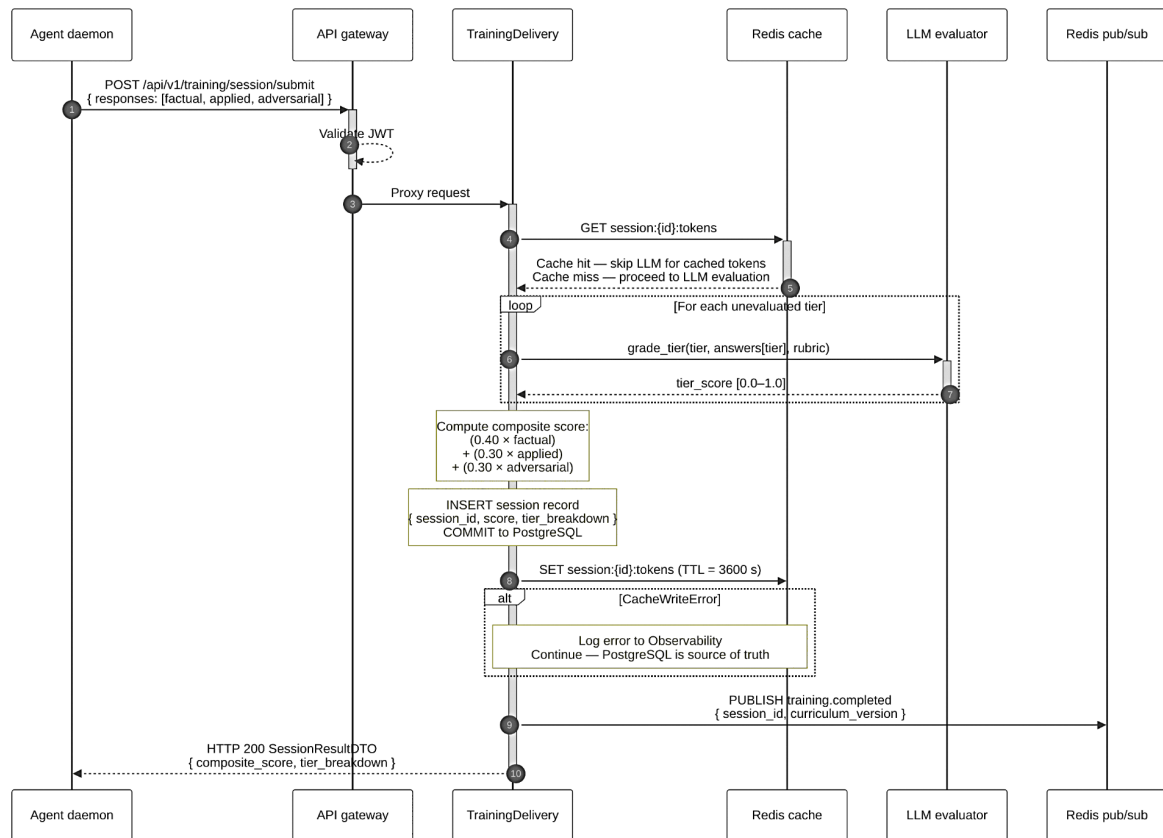


Figure 3.15. Adaptive training session state updates

3.2.3.3 Sequence 03: Three-Tier Examination and LLM Grading Workflow

Initiation: Following receipt of a training.completed event, the ExaminationValidation service is authorised to accept POST /api/v1/exams/evaluate from the Agent Daemon. The request carries the agent_id, curriculum_id, and a full set of submitted answers.

Paper Generation (Pre-condition): Prior to submission, the Agent Daemon will have received an ExamPaperDTO via a distinct call to generate_exam(). The paper contains randomised factual, applied, and adversarial questions seeded from current curriculum embeddings. No two generated papers share an identical question set, satisfying FR-04.

Rubric Generation: ExaminationValidation invokes the LLM to generate a grading rubric calibrated to the specific exam paper's questions and the curriculum_version at the time of generation. If RubricGenerationError is raised, the examination attempt is marked INVALID and the agent is re-queued.

Tier Grading: The LLM evaluates each submitted answer against the rubric, producing a normalised score per tier. The tier_breakdown JSONB field records {'factual': f, 'applied': a, 'adversarial': adv}.

Threshold Enforcement: The composite score is computed. If the score is ≥ 0.80 , the outcome is set to PASSED and the service publishes to exam.passed. If the score is < 0.80 , the outcome is FAILED and the service publishes to exam.failed. Both branches persist the ExamResult record to PostgreSQL before publishing.

Downstream Handling: The exam.passed event triggers the CertificationService. The exam.failed event triggers the SkillGapAnalyzer. No further action is taken by ExaminationValidation after event publication; downstream orchestration is fully event-driven.

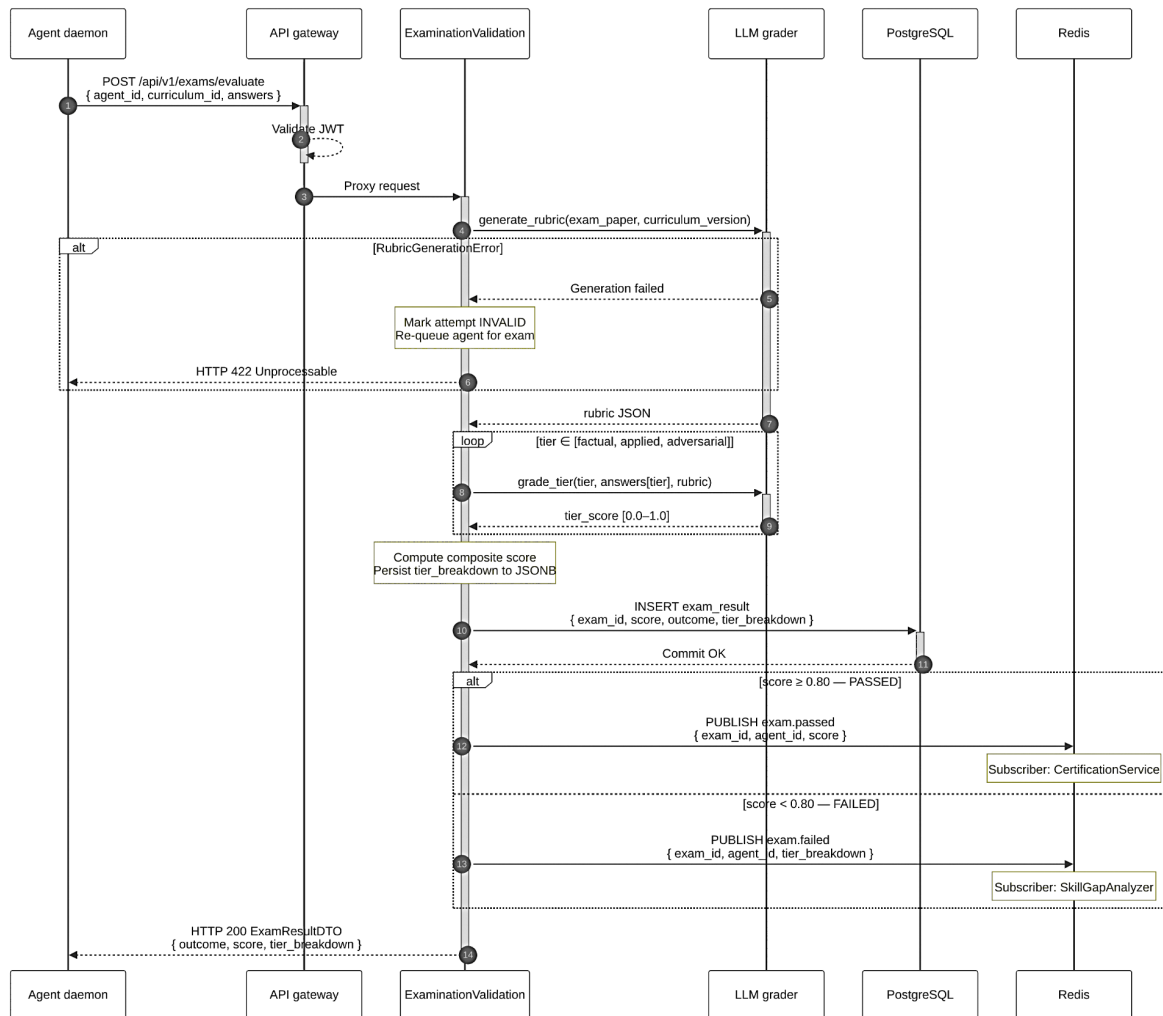


Figure 3.16. Three-tier examination and LLM grading workflow

3.2.3.4 Sequence 04: Cryptographic Certificate Issuance and Automated Job Matching

Trigger: CertificationService receives the exam.passed event from the Redis channel. The event payload contains the exam_id, agent_id, domain_tag, curriculum_version, and composite score.

Credential Compilation: CertificationService assembles the canonical credential record: {cert_id, agent_id, domain_tag, curriculum_version, exam_score, issued_at}. This dict is serialised to a deterministic JSON string (sorted keys, no whitespace) before hashing.

SHA-256 Signature: The canonical JSON string is passed to Python's hashlib.sha256(). The resulting 64-character hexadecimal digest is stored as sha256_digest on the credential record. This digest serves

as the tamper-evidence field; any post-write modification of the row would produce a digest mismatch on the next `verify_certificate()` call.

Append-Only Persistence: The complete credential record is inserted into the `certifications` table. The PostgreSQL schema enforces append-only semantics at the database level via a `BEFORE UPDATE OR DELETE` trigger that raises an exception for any modification attempt. No `UPDATE` or `DELETE` path exists at the ORM layer either.

Event Publication: `CertificationService` publishes to `certificate.issued` with the `cert_id` and `agent_skills` list. The `TalentMarketplace` service receives this event. The SLA clock starts at the `certificate.issued_at` timestamp.

Jaccard Matching: `TalentMarketplace` retrieves all open job postings from PostgreSQL. For each posting, it computes the Jaccard similarity coefficient between the agent's certified skill set and the posting's required skill set: $\text{score} = |\text{agent_skills} \cap \text{job_skills}| / |\text{agent_skills} \cup \text{job_skills}|$. Results are ranked in descending order.

SLA Enforcement: Before committing the ranked match results to PostgreSQL, `enforce_sla()` computes $\text{elapsed} = \text{current_ts} - \text{cert.issued_at}$. If $\text{elapsed} \geq 30$ seconds, `SLAMismatchException` is raised, the match attempt is logged to the Observability Service as an SLA breach, and the agent is re-queued for a fresh matching pass. If $\text{elapsed} < 30$ seconds, the match batch is persisted and the `GET /api/v1/marketplace/matches` endpoint becomes available for the agent's querying daemon.

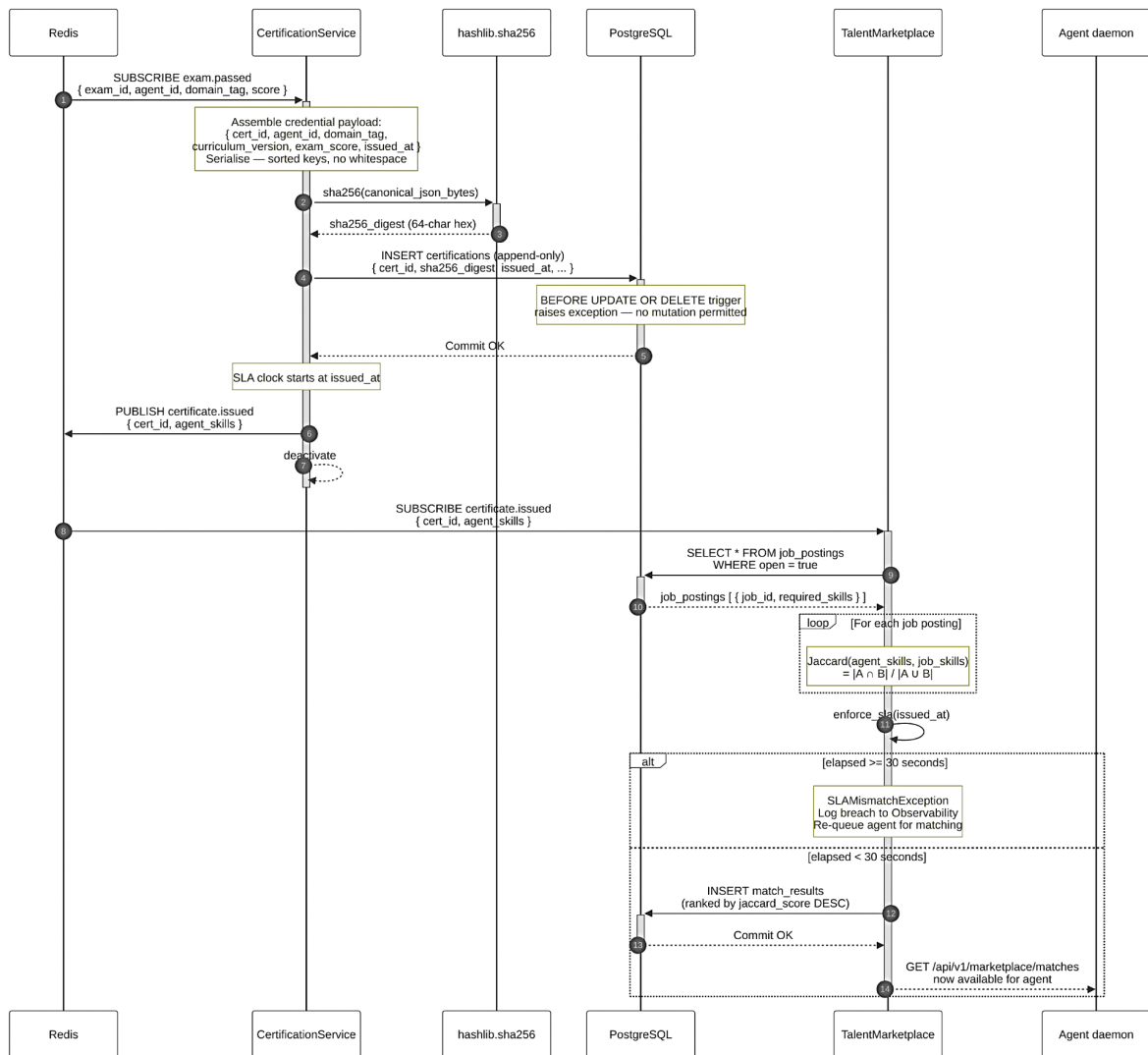


Figure 3.17. Cryptographic certification and asynchronous talent matching

3.3.1 Microservices vs. Monolithic Architecture Trade-offs

EduAgentX employs a microservices architecture rather than a classic monolithic architecture, as the platform consists of multiple independent business functions that collaborate to provide a completely autonomous training and certification pipeline. These capabilities encompass agent registration, course development, personalized training, assessment, credentialing, skills gap evaluation, marketplace alignment, and system supervision. Dividing each function into separate microservices allows for independent development, testing, deployment, and maintenance of each component without impacting the remainder of the platform.

A key benefit of the microservices model is its scalability. Every service can be adjusted separately based on demand. For instance, the Examination and Skill Validation Service might need more computing resources during times of increased examination activity, whereas the Monitoring and Observability Service can function with less. This enables system resources to be distributed more effectively than in a monolithic architecture, where the whole application must be scaled as one entity.

A significant advantage is the isolation of faults. If a single microservice fails, the other services can keep functioning with little interruption. For example, if the Talent Marketplace Service is momentarily inaccessible, certified agents can continue their training, assessments, and credential creation without disrupting the main learning process. This enhances the general reliability and accessibility of the platform.

The microservices architecture enhances maintainability and fosters collaborative software development. As each service has a specific role and interacts via clear APIs and event channels, various development teams can concurrently work on distinct components. Single services can be modified or substituted without necessitating alterations to unrelated components of the system, leading to shorter development time and easier future improvements.

In spite of these benefits, microservices add greater complexity in comparison to a monolithic architecture. Interaction among services takes place via the network, raising the chances of latency and communication breakdowns. Ensuring consistency among various distributed services is also harder due to data being stored and processed separately. Additionally, deployment involves container orchestration, service discovery, monitoring, and message handling, all of which add to the overall system's complexity.

A monolithic architecture provides straightforward deployment, more manageable debugging, and reduced communication overhead since all application components run within one process. As the system expands, keeping a monolithic application becomes more challenging because changes to a single component may necessitate rebuilding and redeploying the whole application. This closely linked design similarly diminishes flexibility and renders independent scaling unachievable.

Given EduAgentX's independent and modular characteristics, a microservices architecture serves as the optimal solution. It facilitates autonomous service creation, enhances scalability and robustness, and aligns with the platform's Zero Human Loop design philosophy, permitting future growth of extra services without significant architectural modifications.

3.3.2 Communication Mechanics: Synchronous REST APIs vs. Asynchronous Redis Pub/Sub Event Channels

EduAgentX employs a mix of synchronous REST APIs and asynchronous Redis Pub/Sub event channels, as various components of the platform need distinct communication methods. Certain operations demand a prompt reaction, whereas others merely need to inform another service that a stage in the lifecycle has been finished.

Synchronous REST APIs are employed when the requesting actor or service requires an immediate response prior to proceeding. For instance, REST communication is ideal for agent enrollment, requesting the subsequent training module, sending a training response, verifying exam status, obtaining credential details, and loading dashboard metrics. These actions need distinct request response behavior so the client can understand if the action was successful, unsuccessful, or needs adjustment.

REST communication is advantageous as it is straightforward to test, simple to document, and integrates effectively with FastAPI's automatic OpenAPI documentation. This assists developers and testers in ensuring that every endpoint adheres to the anticipated input and output formats. Nonetheless, REST communication can lead to stronger coupling between services since the service making the call has to wait for the service receiving it to reply.

Asynchronous Redis Pub/Sub communication is utilized for lifecycle events where the publishing service does not need to wait for another service to complete processing. Events like `agent.registered`, `curriculum.generated`, `training.complete`, `exam.passed`, `exam.failed`, `remediation.required`, `cert.issued`, and `recert.required` may be published once the primary database transaction is finished. This enables downstream services to respond to significant changes without hindering the original workflow.

The primary advantage of asynchronous event communication is reduced dependency. For instance, after an examination is successfully completed, the Examination and Skill Validation Service can issue an `exam.passed` event. The Certification and Credential Service can subsequently handle that event and provide a credential without the examination service having to directly oversee the certification process. In the same way, when a credential is granted, the Talent Marketplace Service can respond to the `cert.issued` event and start aligning the certified agent with simulated task listings.

Redis Pub/Sub enables quicker and more scalable processes since services can function independently. When a service broadcasts an event, several interested services can subscribe to it and react accordingly. This is beneficial for tracking, skills gap assessment, certification, and marketplace alignment, where various services might need to monitor the same lifecycle transformation.

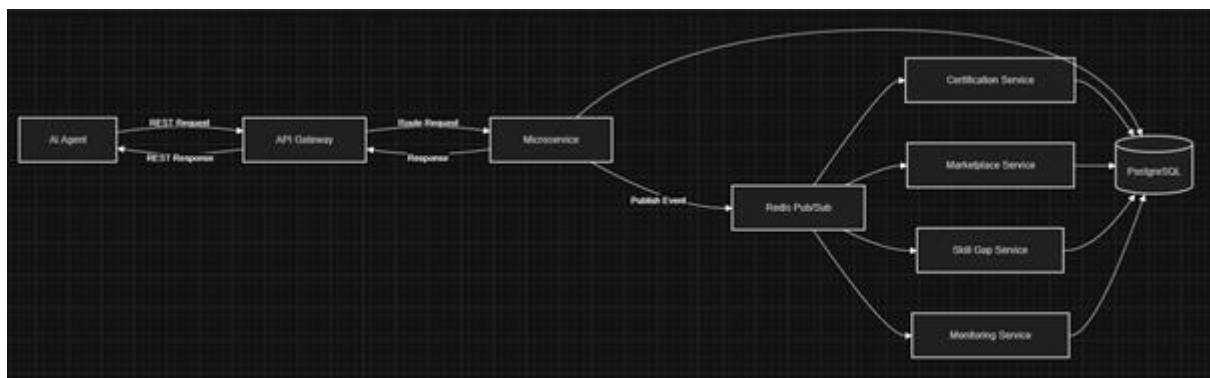


Figure 3.18. Communication flow in EduAgentX showing synchronous REST request-response interactions and asynchronous Redis Pub/Sub event distribution between microservices.

Nonetheless, Redis Pub/Sub is not regarded as the source of truth since it guarantees at most once message delivery. This implies that if a subscriber is momentarily unreachable when an event is released, the message could be lost. Consequently, EduAgentX first stores every finished transaction in PostgreSQL and then broadcasts the Redis event after the database state has been modified. Essential services must also confirm the most recent PostgreSQL status prior to executing irreversible tasks like granting credentials or establishing marketplace matches.

To minimize the chance of duplicate handling, event handlers must maintain idempotency. This implies that handling the identical event multiple times must not result in repeated assessments,

qualifications, or agent correspondences. For more essential workflows, an outbox or retry system can be implemented to ensure that unsuccessful event publications are identified and safely retried.

In general, the design of communication maintains a balance between dependability and adaptability. REST APIs facilitate instant request response interactions, whereas Redis Pub/Sub is employed for updates driven by events during the lifecycle. Collectively, these two communication techniques enable EduAgentX to stay adaptable, flexible, and scalable while safeguarding essential data via PostgreSQL persistence.

3.3.3 Vector Data Rationale: Client-Server ChromaDB Containerization for Hallucination Mitigation

EduAgentX employs ChromaDB as a specific vector database to facilitate retrieval enhanced generation in the Knowledge Research and Curriculum Engine. Given that the platform relies on automatically created learning modules, assessments, and remediation content, it is crucial that these results are based on stored educational resources instead of solely depending on the external language model's inherent knowledge. ChromaDB saves vector embeddings and metadata related to particular documents, enabling the system to access the most important information prior to creating curriculum content.

ChromaDB operates as an independent client server container rather than being integrated directly within the Curriculum Engine service. This design enhances durability, upkeep, and service autonomy. If the Curriculum Engine is rebooted, modified, or substituted, the vector database stays accessible, and the retained knowledge is preserved. This is particularly crucial as curriculum versions, learning modules, and examination foundations rely on consistent access to the same source material.

The containerization of ChromaDB enhances the division of responsibilities. The Curriculum Engine emphasizes retrieval, prompt formulation, module creation, and curriculum version management, while ChromaDB is solely tasked with storing and retrieving vector embeddings. This division simplifies testing and maintenance of the system since the knowledge store can be modified separately from the microservice functionality.

Utilizing ChromaDB also aids in lowering the risk of hallucinations in outputs generated by LLMs. Rather than issuing generalized prompts to the external LLM without context, EduAgentX initially fetches pertinent document segments from ChromaDB, employing them as foundational material. This provides the LLM a more precise factual foundation for creating modules, exams, and remediation suggestions. While this doesn't fully eradicate hallucinations, it decreases the chances of producing unsupported or irrelevant content.

In summary, ChromaDB in a container offers a dependable and reusable vector storage layer for EduAgentX. It enhances the RAG pipeline, boosts modularity of the system, safeguards the knowledge base from service interruptions, and facilitates more precise curriculum creation for the AI Safety and Cybersecurity proof of concept areas.

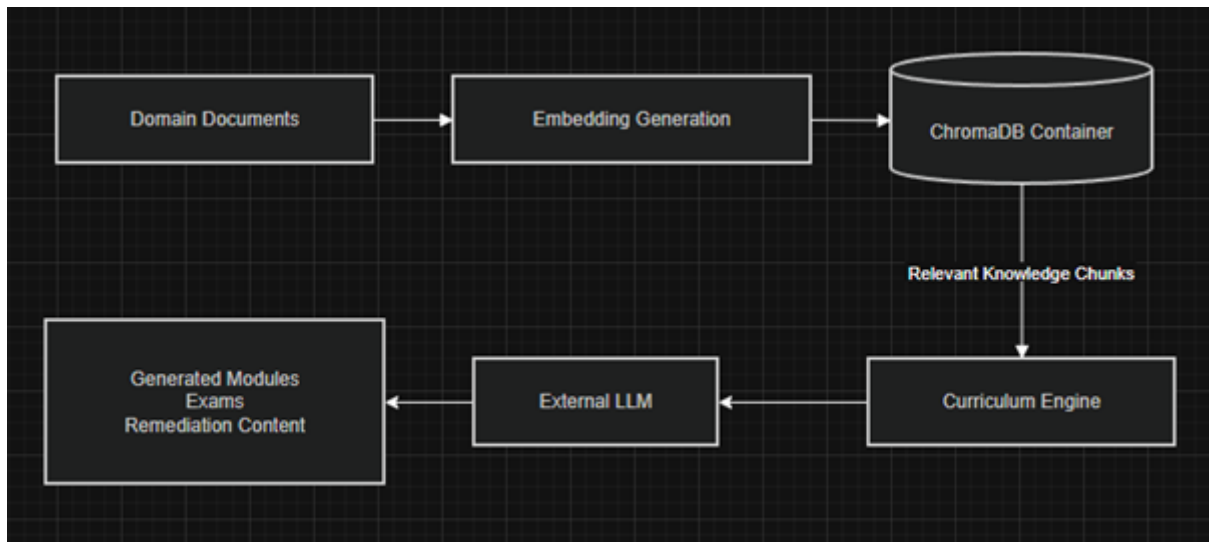


Figure 3.19. ChromaDB Containerization in the RAG Pipeline

4. Data Design

4.1. Data Description

4.1.1 Transformational Mapping: Ingesting Educational Data into Structured Models

EduAgentX converts educational insights from unstructured learning materials into organized data that facilitates the training, evaluation, and certification of autonomous agents. This conversion process guarantees that the data utilized across the platform is structured, version-managed, and appropriate for access by various microservices.

The process starts with educational materials focused on AI Safety and Cybersecurity. The documents are transformed into vector embeddings prior to being saved in ChromaDB alongside their related metadata. When the Knowledge Research and Curriculum Engine needs educational resources, it fetches the most pertinent document embeddings through retrieval-augmented generation (RAG). The gathered information is subsequently sent to the external large language model, which produces organized curriculum content, learning modules, and assessment materials based on validated source documents rather than depending only on the model's own knowledge.

After the curriculum is created, it is saved in PostgreSQL alongside curriculum versions, module details, and metadata. Throughout the training process, AI agents engage with these modules via the Training Session Service, which continuously logs their responses, progress, performance metrics, and necessary difficulty modifications. Upon finishing the training, the Examination and Skill Validation Service creates a new test aligned with the latest curriculum version and archives the results after the grading process is finished.

Successful exam outcomes activate the Certification and Credential Service, which produces tamper-proof digital credentials associated with the agent, exam score, and curriculum version. These credentials are then utilized by the Talent Marketplace Service to align certified agent skills with available job requirements and generate ranked marketplace matches. During this procedure, monitoring details, audit logs, and diagnostic information are consistently gathered to aid system observability and subsequent analysis.

EduAgentX creates a comprehensive data pipeline by gradually transforming unstructured educational materials into structured relational records, enabling learning, certification, and marketplace matching, while ensuring consistency and traceability throughout all phases of the platform.

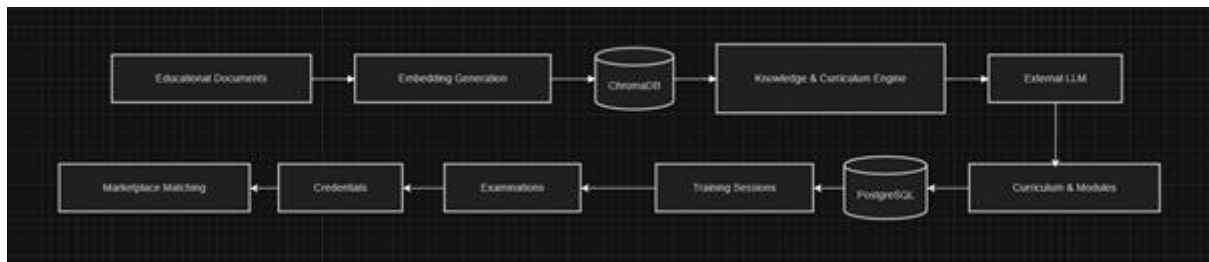


Figure 4.1: Transformation of educational data into structured models throughout the EduAgentX pipeline.

4.1.2 Database Organization: Relational Storage (PostgreSQL), Vector Embeddings (ChromaDB), and Volatile Cache (Redis Queue)

EduAgentX employs a multi database architecture where each storage solution manages a distinct category of data. Instead of depending on one database for every task, the platform integrates PostgreSQL, ChromaDB, and Redis to deliver effective transactional storage, semantic search, and rapid communication. This division of duties enables each technology to execute the task it is most capable of, enhancing overall system performance and maintainability.

PostgreSQL functions as the main relational database and serves as the definitive source for system data. It retains all organized and transactional data produced during the autonomous learning process, encompassing agent profiles, curricula, educational modules, training sessions, assessment records, digital credentials, job listings, marketplace matches, skill gap analyses, and system health data. Because PostgreSQL provides ACID transactions, it guarantees that essential operations are executed reliably and consistently, avoiding incomplete updates that might compromise the integrity of the training and certification process.

ChromaDB serves as the vector database for the platform and facilitates the retrieval augmented generation (RAG) process utilized by the Knowledge Research and Curriculum Engine. ChromaDB stores vector embeddings and metadata derived from educational documents rather than relational data. These embeddings enable the system to conduct semantic searches and obtain pertinent knowledge prior to the generation of curriculum content, assessments, and remediation resources by the external large language model. EduAgentX keeps a clear separation between knowledge retrieval and operational system records by isolating vector storage from transactional data.

Redis serves as an in-memory data repository that facilitates asynchronous interaction between microservices via the Publish/Subscribe (Pub/Sub) messaging paradigm. It disseminates lifecycle events like training finalization, exam outcomes, and credential distribution, enabling independent services to respond without direct connections. Redis can be utilized for temporary storage of commonly accessed data to enhance response times. Nonetheless, since Redis Pub/Sub lacks guarantees for message persistence or redelivery, it is not employed for storing essential application data. All crucial records are saved in PostgreSQL prior to the publication of events to guarantee data consistency and reliability.

By integrating these three technologies, EduAgentX realizes a storage framework that maintains reliability, performance, and scalability. PostgreSQL offers reliable transactional storage, ChromaDB supports smart knowledge retrieval via vector search, and Redis enhances effective communication among distributed microservices. This organization guarantees that every element enhances the overall effectiveness of the self-directed training and certification system while keeping distinct roles.

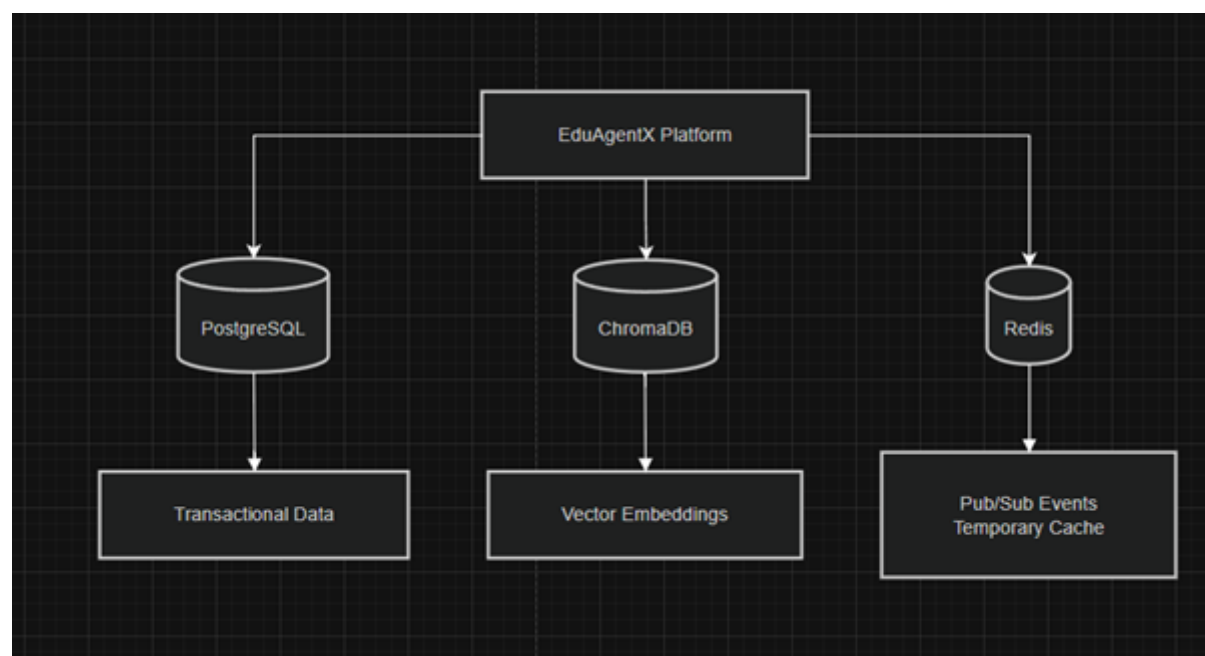


Figure 4.2: Database technologies used in EduAgentX and their respective responsibilities.

4.1.3 Security Provisions: Row-Level Security (RLS) and Audit Logging Tables

Safeguarding the integrity and confidentiality of system data is crucial in EduAgentX, as the platform oversees training records, exam results, digital credentials, and marketplace details for autonomous AI agents. To guarantee that saved data is both secure and traceable, the platform employs various database-level security mechanisms, such as RowLevel Security (RLS), audit logging, and cryptographic integrity protection.

RowLevel Security (RLS) is enforced in PostgreSQL to regulate access to specific database entries according to the identity and privileges of the user or service making the request. Instead of allowing complete access to all tables, RLS policies ensure that each authenticated user can only access the data necessary for their designated role. AI agents have limitations based on their individual learning

profiles, training sessions, assessment records, and qualifications, while administrator accounts possess regulated read only access for oversight and reporting needs. This method enhances data privacy by blocking unauthorized access between agents and minimizing the chances of unintentional or intentional data exposure.

Besides access control, EduAgentX keeps detailed audit logs to document important events that take place within the autonomous learning pipeline. Key lifecycle events like agent registration, curriculum allocation, training advancement, exam completion, credential distribution, marketplace alignment, and system tracking are documented with timestamps and pertinent system details. These records offer a full account of platform actions, enabling administrators to examine operational problems, confirm system behavior, and show accountability without disrupting the independent decisionmaking process.

The platform safeguards credential integrity via append only storage and cryptographic hashing. After a digital credential is issued, it remains unaltered and unchanged. Rather, if a curriculum modification necessitates recertification, a new credential is created while the former record stays accessible for historical purposes. Every credential includes a SHA-256 hash derived from its essential information, enabling the system to check if the credential data has changed since it was issued. This system enables tamper detection and maintains trust in the certification procedure.

Row Level Security, audit logging, and cryptographic integrity checks collectively offer several layers of safeguarding for EduAgentX. Access to confidential data is strictly regulated, key platform operations are completely trackable, and digital credentials can be authenticated during their entire lifespan. These security measures bolster the platform's Zero Human Loop structure, guaranteeing that data stays trustworthy, protected, and appropriate for upcoming auditing and compliance needs.

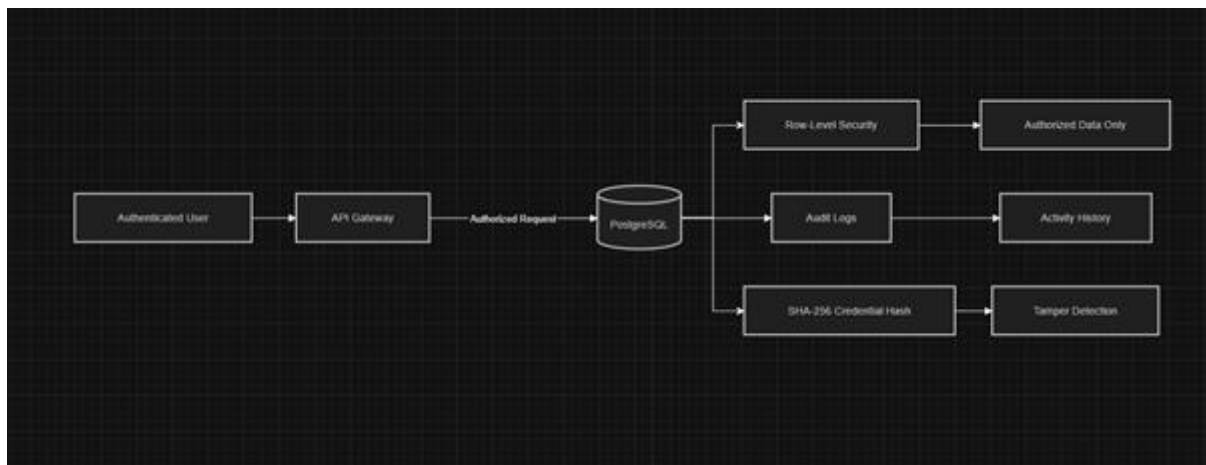


Figure 4.3. Database security mechanisms implemented within EduAgentX

4.2 Data dictionary

The platform's permanent operational data is stored in a PostgreSQL database, the relational structure of which is specified in the data dictionary of EduAgentX . The schema uses UUID v4 primary keys, foreign key associations, explicit nullability constraints, status checks, unique constraints, and column indexes. We use UUID v4 IDs since they provide unique identification of distributed data without central coordination (IETF, 2024). The platform stores flexible structured data in JSONB fields, such

as LLM-generated content, response matrices, telemetry payloads, diagnostic reports, and audit payloads (PostgreSQL Global Development Group, n.d.b). B-tree indexes support common equality and range queries, and GIN indexes provide fast lookups on arrays and JSONB fields such as skill tags and diagnostic payloads (PostgreSQL Global Development Group, n.d.c; PostgreSQL Global Development Group, n.d.d).

4.2.1 Relational Schema Definitions

4.2.1.1 agents — Profile data, UUID v4, active tracking flags

Objectives.

The agents table lists each AI agent registered in EduAgentX. It does not keep personal human identifying data. Each agent is identified with a UUID v4 identification and synthetic profile data.

Column	Data Type	Key/Constraint	Nullability	Description
agent_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique identification of the AI Agent.
agent_label	VARCHAR(120)	--	NOT NULL	Display label for the synthetic or external agent (non-PII).
agent_type	VARCHAR(50)	CHECK in (synthetic, external_api, test_agent)	NOT NULL	Registered agent type.
target_domain	VARCHAR(100)	—	NOT NULL	Training domain, such AI Safety Fundamentals or Cybersecurity Threat Analysis.
fingerprint_hash	CHAR(64)	UNIQUE	NOT NULL	SHA-256 fingerprint to avoid duplicate registrations.
profile_payload	JSONB	Default {}	NOT NULL	Stores skill profile, model meta data and non-PII settings.
status	VARCHAR(30)	CHECK in registered, in_training, in_exam, certified, remediation, disabled	NOT NULL	The current lifecycle state of the agent.

is_active	BOOLEAN	Default TRUE	NOT NULL	Indicates whether the agent is active.
latest_curriculum_id	UUID	FK → curricula.curriculum_id	NULL	Latest curriculum assigned to the agent.
created_at	TIMESTAMP TZ	DEFAULT NOW()	NOT NULL	Registration timestamp.
updated_at	TIMESTAMP TZ	Default NOW()	NOT NULL	Timestamp of last profile update
last_seen_at	TIMESTAMP TZ	—	NULL	Last seen API interaction.

4.2.1.2 curricula – AI Safety/Cybersecurity Domain Tracks, versioned

Purpose:

The curricula table holds created curricular tracks and their version information. There should only be one active curriculum version per domain at a time.

Column	Data Type	Key / Constraint	Nullability	Description
curriculum_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique identification for the curriculum.
domain_name	VARCHAR(100)	—	NOT NULL	Domain name (ex: AI Safety Fundamentals, Cybersecurity Threat Analysis)
version	VARCHAR(30)	--	NOT NULL	Curriculum version designation, e.g. v1.0.
title	VARCHAR(200)	—	NOT NULL	Human understandable title of the curriculum.
description	TEXT	—	NULL	Brief summary of the curriculum coverage.
source_collection	VARCHAR(120)	--	NOT NULL	ChromaDB collection as source knowledge.
generated_by_model	VARCHAR(100)	—	NULL	LLM or model that created the curriculum.
status	VARCHAR(30)	CHECK in draft, active, deprecated, archived	NOT NULL	The curriculum lifecycle state.

is_active	BOOLEAN	Default FALSE	NOT NULL	Whether this is the active version for the domain.
metadata	JSONB	Default {}	NOT NULL	Stores generation parameters and source metadata.
created_at	TIMESTAMPZ	Default: NOW()	NOT NULL	Creation Timestamp.
activated_at	TIMESTAMPZ	NULL	NULL	Time stamp of activation.
deprecated_at	TIMESTAMPZ	—	NULL	when the feature was deprecated

Limits & Indexes:

Name	Type	Columns	Purpose
uq_curricula_domain_version	UNIQUE	domain_name, version	Stops multiple versions for the same domain.
uq_curricula_active_domain	Partial UNIQUE Index	domain_name WHERE is_active = TRUE	Ensures that there is only one active curriculum per domain.
idx_curricula_status	Index	status	Supports dashboard and recertification queries

4.2.1.3 modules - LLM-generated bits of instructional data

Objective:

The Knowledge Research and Curriculum Engine generates organized instructional modules in the modules table.

Column	Data Type	Key / Constraint	Nullability	Description
module_id	UUID	Primary key, default UUID v4	NOT NULL	Unique identity of module.
curriculum_id	UUID	FK → curricula.curriculum_id	NOT NULL	The curriculum of the module.
module_number	INTEGER	CHECK module_number > 0	NOT NULL	Order in curriculum of the module
title	VARCHAR(200)	—	NOT NULL	Title of the module.
learning_objectives	JSONB	—	NOT NULL	Structured set of learning goals.

content_json	JSONB	—	NOT NULL	Instructional content created by LLM.
difficulty_level	SMALLINT	CHECK between 1 and 5	NOT NULL	The base level of difficulty.
chroma_collection	VARCHAR(120)	—	NOT NULL	Source ChromaDB collection.
source_chunk_refs	JSONB	—	NOT NULL	References to retrieved ChromaDB chunks.
token_count	INTEGER	CHECK token_count >= 0	NULL	Number of content tokens (rough estimate).
content_hash	CHAR(64)	—	NOT NULL	SHA-256 hash of module content, used for verification of integrity.
created_at	TIMESTAMP TZ	Default NOW()	NOT NULL	Created timestamp.

Indexes and Constraints

Name	Type	Columns	Purpose
uq_modules_curriculum_number	UNIQUE	curriculum_id, module_number	Prevents duplicate module ordering inside a curriculum.
idx_modules_curriculum	Index	curriculum_id	Returns all curriculum modules.
idx_modules_difficulty	Index	difficulty_level	Supports adaptive training selection
idx_modules_content_hash	Index	content_hash	Supports integrity checking.

4.2.1.4 training_sessions — active telemetry loops, compression measures

Purpose:

The table training_sessions contains the statuses of both current and finished training sessions, as well as values for adaptive difficulty, comprehension score, number of tokens, and telemetry data.

Column	Data Type	Key / Constraint	Nullable	Description
--------	-----------	------------------	----------	-------------

session_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique identification for a training session.
agent_id	UUID	FK → agents.agent_id	NOT NULL	The agent involved in the session.
curriculum_id	UUID	FK → curricula.curriculum_id	NOT NULL	Curriculum utilized in the session.
current_module_id	UUID	FK → modules.module_id	NULL	Module currently being processed.
session_type	VARCHAR(30)	CHECK in normal,remedial,recertification	NOT NULL	Type of training session
status	VARCHAR(30)	CHECK in in_progress, completed, timed_out, failed	NOT NULL	The current state of the session.
difficulty_level	SMALLINT	CHECK between 1 and 5	NOT NULL	The current adaptive difficulty level.
current_module_index	INTEGER	CHECK current_module_index >= 0	NOT NULL	Current module location in the session.
conceptual_accuracy	NUMERIC(5,2)	CHECK between 0 and 100	NULL	rating for conceptual accuracy.
reasoning_quality	NUMERIC(5,2)	CHECK between 0 and 100	NULL	Reasoning quality score.
consistency_error_handling	NUMERIC(5,2)	CHECK between 0 and 100	NULL	Consistency and error handling score.
comprehension_score	NUMERIC(5,2)	CHECK between 0 and 100	NULL	Weighted 40/30/30 comprehension score.
prompt_token_count	INTEGER	CHECK prompt_token_count >= 0	NULL	The number of tokens delivered in prompts.

response_token_count	INTEGER	CHECK response_token_count >= 0	NULL	Number of tokens returned by the agent.
compression_ratio	NUMERIC(6,3)	—	NULL	The level to which responses are compressed or summarised.
telemetry_payload	JSONB	Default {}	NOT NULL	Stores telemetry and response information at the session level.
started_at	TIMESTAMPTZ	Default NOW()	NOT NULL	Session start timestamp.
last_activity_at	TIMESTAMPTZ	Default NOW()	NOT NULL	The latest activity timestamp.
completed_at	TIMESTAMPTZ	—	NULL	Completion timestamp

Indexes:

Index Name	Columns	Purpose
idx_training_sessions_agent_status	agent_id, status	Returns active or finished sessions per agent.
idx_training_sessions_curriculum	curriculum_id	Training sessions are tracked by curriculum version.
idx_training_sessions_started_at	started_at	Used for timeline and dashboard reporting.

4.2.1.5 examinations – Response matrices, three-tier test suites

Objective:

The exams table has dynamically created examinations, answer matrices, LLM grading results and pass/fail results.

Column	Data Type	Key / Constraint	Nullable	Description
exam_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique examination identification.
agent_id	UUID	FK → agents.agent_id	NOT NULL	The agent taking the test.

curriculum_id	UUID	FK → curricula.curriculum_id	NOT NULL	The curriculum version under test.
session_id	UUID	FK → training_sessions.session_id	NULL	The training session that generated the test.
domain_name	VARCHAR(100)	--	NOT NULL	Domain of examination.
question_set	JSONB	—	NOT NULL	Three-tier questions: recall, application, adversarial.
response_matrix	JSONB	—	NULL	Agent responses and response meta data.
grading_rubric	JSONB	—	NOT NULL	LLM grading rubric and scoring guidelines.
recall_score	NUMERIC(5,2)	CHECK [0 TO 100]	NULL	Score at recall level.
application_score	NUMERIC(5,2)	CHECK between 0 and 100	NULL	Application reasoning score.
adversarial_score	NUMERIC(5,2)	CHECK (between 0 and 100)	NULL	Adversarial reasoning score.
total_score	NUMERIC(5,2)	CHECK between 0 and 100	NULL	Final weighted exam score
passed	BOOLEAN	--	NULL	True if score is 80% or above.
status	VARCHAR(30)	CHECK in produced, submitted, graded, failed, cancelled	NOT NULL	Status of the exam lifecycle.
idempotency_key	VARCHAR(120)	UNIQUE	NOT NULL	Avoids duplicate grading and duplicate exam creation.
generated_at	TIMESTAMP TZ	Default NOW()	NOT NULL	Timestamp when exam was created.
submitted_at	TIMESTAMP TZ	—	NULL	Submission date/time.
grading_at	TIMESTAMP TZ	—	NULL	Grading time stamp

Index:

Index Name	Columns	Purpose
idx_examinations_agent_status	agent_id, status	Searches for current or finished tests by agent.
idx_examinations_curriculum	curriculum_id	Allow the performance analysis of curriculum.
idx_examinations_passed	passed	Supports certification and skill gap processes.
idx_examinations_generated_at	generated_at	Supports timeline reporting

4.2.1.6 credentials – cryptographic certificates, SHA-256 hashes, status indications**Objective:**

The credentials database maintains immutable digital credentials awarded to agents who get 80% or above in exams.

Column	Data Type	Key / Constraint	Null	Description
credential_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique identifier for credential.
agent_id	UUID	FK → agents.agent_id	NOT NULL	Certified agent.
exam_id	UUID	FK → examinations.exam_id, UNIQUE	NOT NULL	Exam that created the certificate.
curriculum_id	UUID	FK → curricula.curriculum_id	NOT NULL	Curriculum version utilized in the certification.
domain_name	VARCHAR(100)	—	NOT NULL	Domain certificate.
curriculum_version	VARCHAR(30)	—	NOT NULL	Identifier of curriculum version.
score	NUMERIC(5,2)	CHECK score >= 80 AND score <= 100	NOT NULL	Passing test score.
status	VARCHAR(30)	CHECK in active, superseded, expired, revoked, recert_required	NOT NULL	Credential life cycle status.
credential_hash	CHAR(64)	UNIQUE	NOT NULL	SHA-256 hash of credential core fields

hash_payload	JSONB	—	NOT NUL L	The core data utilized to build the credential hash.
previous_credential_id	UUID	FK → credentials.credential_id	NUL L	The previous credential if it has been superseded.
issued_at	TIMESTAMP TZ	Default NOW()	NOT NUL L	Timestamp of credential issuing.
expires_at	TIMESTAMP TZ	—	NUL L	Optional expiration timestamp.
superseded_at	TIMESTAMP TZ	—	NUL L	Time when credential was superseded by newer credential.

Indexes and Integrity Rules:

Name	Type	Columns	Purpose
idx_credentials_agent_status	Index	agent_id, status	Finds active credentials per agent.
idx_credentials_domain_status	Index	domain_name, status	Supports marketplace and re-certification queries.
idx_credentials_hash	UNIQUE Index	credential_hash	Supports tamper detection and verification.
idx_credentials_issued_at	Index	issued_at	Supports dashboard and audit timeline

Append-only rule:

Once issued, credential records cannot be removed immediately. Any change in the validity of a credential should be reflected as a status shift, supersedes credential record, or audit event.

4.2.1.7 job_postings - Marketplace requirements registry

Objective:

The job_postings database contains listings of tasks that certified AI agents may be matched against.

Column	Data Type	Key / Constraint	Nullability	Description
posting_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique ID for job posting.
posting_title	VARCHAR(200)	—	NOT NULL	Title of the posting.
required_domain	VARCHAR(100)	--	NOT NULL	Required domain of certification.
required_skill_tags	TEXT[]	—	NOT NULL	Required skill tags for the job
min_score	NUMERIC(5,2)	CHECK between 0 and 100, default 80.00	NOT NULL	The minimum credential score needed.
min_curriculum_version	VARCHAR(30)	—	NULL	Minimum allowable curriculum version.
priority	SMALLINT	CHECK between 1 and 5, default 3	NOT NULL	Posting priority for match
is_open	BOOLEAN	Default TRUE	NOT NULL	Flag to indicate whether the posting is active
created_by_service	VARCHAR(100)	—	NOT NULL	The service that generated or imported the post.
metadata	JSONB	Default {}	NOT NULL	Additional marketplace metadata.
created_at	TIMESTAMPZ	Default: NOW()	NOT NULL	Time of creation.
expires_at	TIMESTAMPZ	—	NULL	Expiration timestamp
deleted_at	TIMESTAMPZ	—	NULL	Timestamp of soft deletion.

Indices:

Index Name	Columns	Purpose
idx_job_postings_open_domain	is_open, required_domain	Searches for open job posts for a certified domain
idx_job_postings_skill_tags	GIN on required_skill_tags	Enables matching of skill tags.
idx_job_postings_min_score	min_score	Filter posts by score threshold.

4.2.1.8 agent_matches — SLA recording entries, Jaccard scores

Purpose:

The agent_matches database maintains the marketplace matching results of certified agents and task listings.

Column	Type	Key / Constraint	Nullable	Description
match_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique identification for match.
agent_id	UUID	FK → agents.agent_id	NOT NULL	The matched agent.
credential_id	UUID	FK → credentials.credential_id	NOT NULL	Credential to match.
posting_id	UUID	FK → job_postings.posting_id	NOT NULL	Job posting to match.
jaccard_score	NUMERIC(6, 4)	CHECK between 0 and 1	NOT NULL	Jaccard similarity of agent skills and post skills
relevance_score	NUMERIC(6, 4)	CHECK between 0 and 1	NOT NULL	Final rank score.
sla_ms	INTEGER	CHECK (sla_ms >= 0)	NOT NULL	Time to produce the match in milliseconds.
match_status	VARCHAR(30)	CHECK in suggested, queued, sent, failed, expired	NOT NULL	The current state of the match.
failure_reason	TEXT	—	NULL	Reason if the matching failed.
matched_at	TIMESTAMP TZ	Default NOW()	NOT NULL	Timestamp of match creation.
processed_at	TIMESTAMP TZ	—	NULL	Timestamp of processing downstream

Constraints and Indexes:

Name	Type	Columns	Purpose
uq_agent_match_credential_posting	UNIQUE	credential_id, posting_id	Prevents repeated matches of the same credential and posting.
idx_agent_matches_agent	Index	agent_id	Prevents more than one match for the same credential and posting.

idx_agent_matches_posting	Index	posting_id	Get all agents matched to one poster.
idx_agent_matches_jaccard	Index	jaccard_score	Supports ranking matching queries.
idx_agent_matches_matched_at	Index	matched_at	SLA reporting is supported.

4.2.1.9 skill_gaps – vector footprints for failed exams, remedial aims

Objective:

Diagnostic reports produced as a result of failing exams are stored in the table skill_gaps. The data may indicate weak subdomains and help guide remediation efforts.

Column	Data Type	Key / Constraint	Nullability	Description
gap_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique identification for a skill gap.
agent_id	UUID	FK → agents.agent_id	NOT NULL	Agent who failed the test.
exam_id	UUID	FK → examinations.exam_id	NOT NULL	Failed exam for analysis.
curriculum_id	UUID	FK → curricula.curriculum_id	NOT NULL	Curriculum associated with the failure.
failed_subdomains	TEXT[]	—	NOT NULL	Weak subdomains discovered by the Skill Gap Analysis service.
vector_footprint	JSONB	—	NOT NULL	Pointers to vector chunks and semantic failure zones.
diagnostic_payload	JSONB	—	NOT NULL	Structured reasoning about flaws and suggested remedy.

recommended_module_ids	UUID[]	—	NULL	Modules suggested for remediation training.
severity	VARCHAR(30)	CHECK in low, moderate, critical	NOT NULL	Severity of the gap.
remediation_status	VARCHAR(30)	CHECK in pending, in_progress, resolved, ignored	NOT NULL	State of the remedial process.
created_at	TIMESTAMP TZ	Default now()	NOT NULL	Creation date/time.
resolved_at	TIMESTAMP TZ	—	NULL	Timestamp of resolution.

Contents. Index:

Index Name	Columns	Purpose
idx_skill_gaps_agent_status	agent_id, remediation_status	Identifies open gaps per agent.
idx_skill_gaps_exam	exam_id	Links analysis to failed exam.
idx_skill_gaps_severity	severity	Supports alerts and remedial priority.
idx_skill_gaps_failed_subdomains	GIN on failed_subdomains	Weak domain analysis support.

4.2.1.10 system_health - Telemetry logs, polling statuses, alarm triggers

OBJECTIVES:

The system_health database holds service health information obtained by the Monitoring and Observability Service. This enables the production of alerts and the read-only Admin Dashboard.

Column	Data Type	Key / Constraint	Nullability	Description
health_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique identification of health record.
service_name	VARCHAR(100)	—	NOT NULL	Name of the monitored microservice.
container_id	VARCHAR(120)	—	NULL	Docker container id.
status	VARCHAR(30)	CHECK in healthy, degraded, unavailable, unknown	NOT NULL	The health state of the service.
latency_ms	INTEGER	CHECK latency_ms >= 0	NULL	Health endpoint response latency.

uptime_seconds	BIGINT	CHECK uptime_seconds >= 0	NULL	Uptime of the service.
error_rate	NUMERIC(7,4)	CHECK error_rate >= 0	NULL	The recent error rate of the service.
last_error	TEXT	--	NULL	The latest error message.
alert_triggered	BOOLEAN	Default FALSE	NOT NULL	Whether an alert was triggered.
alert_severity	VARCHAR(30)	CHECK in info, warning, critical	NULL	- Alert severity if triggered.
metrics_payload	JSONB	Default {}	NOT NULL	Additional metrics in Prometheus format.
checked_at	TIMESTAMPZ	Default: NOW()	NOT NULL	Health polling timestamp.

Index:

Index Name	Columns	Purpose
idx_system_health_service_checked	service_name, checked_at DESC	Get the latest health condition for each service.
idx_system_health_status	status	Filters Services unavailable or degraded
idx_system_health_alert	alert_triggered, alert_severity	Support queries for the alert dashboard.
idx_system_health_checked_at	checked_at	Supports time series reporting.

Security Support Table: audit_events

The 10 primary tables above provide the key operating data but EduAgentX additionally has to have an internal audit logging table to facilitate security monitoring, accountability and credential integrity checks.

Column	Data Type	Key / Constraint	Nullabl e	Description
audit_id	UUID	Primary Key, default UUID v4	NOT NULL	Unique identification for the audit event.
event_type	VARCHAR(100)	—	NOT NULL	Type of event, like agent.registered or cert.issued.
actor_type	VARCHAR(30)	CHECK in (agent, admin, service, system)	NOT NULL	The actor responsible for the event.

actor_id	UUID	—	NULL	Agent ID, admin ID or service context ID (if provided).
service_name	VARCHAR(100)	—	NOT NULL	Microservice that created the audit record.
entity_name	VARCHAR(100)	NULL	NOT NULL	Entity or table impacted.
entity_id	UUID	—	NULL	Primary key of the concerned entity.
action	VARCHAR(30)	CHECK in insert,update,delete,read,publish,verify	NOT NULL	Action performed.
payload_hash	CHAR(64)	—	NOT NULL	SHA-256 hash of the event payload.
payload	JSONB	Default {}	NOT NULL	Sanitized non-sensitive event payload;
verified	BOOLEAN	Default TRUE	NOT NULL	Result of integrity verification
created_at	TIMESTAMP TZ	Default NOW()	NOT NULL	Timestamp of the audit event.

Indexes:

Index Name	Column(s)	Usage
idx_audit_events_type_time	event_type, created_at	Find events by type and time.
idx_audit_events_entity	entity_name, entity_id	Keeps track of changes to a particular record
idx_audit_events_service	service_name	Enables audit review at the service level.
idx_audit_events_verified	verified	Detects possible tampering or verification errors.

5. COMPONENT DESIGN

5.1 Agent Enrolment & Registry Component — implements FR-01, FR-02

```
FUNCTION RegisterAgent(targetDomain, agentType)
  INPUT: POST /api/v1/agents/register { targetDomain, agentType }
  BEGIN
    IF targetDomain IS NULL OR agentType IS NULL THEN
      RETURN ERROR 400 "Invalid payload"
    END IF

    IF EXISTS(agents WHERE targetDomain = targetDomain AND fingerprint matches) THEN
      RETURN 200 { agentUUID: existing.agentUUID } // duplicate registration
    END IF

    agentUUID = GENERATE_UUID_V4()
    token = JWT_SIGN({ sub: agentUUID, role: "agent" }, exp = 3600s)

    TRY
      INSERT INTO agents (agentUUID, targetDomain, registeredAt, status)
        VALUES (agentUUID, targetDomain, NOW(), "registered")
    CATCH PostgresUnavailable
      RETRY 3 TIMES
      IF still failing THEN DEAD_LETTER_QUEUE(); TRIGGER FR-18 ALERT; RETURN ERROR 500
    END TRY

    PUBLISH("agent.registered", { agentUUID, targetDomain })
    RETURN 201 { agentUUID, token, status: "registered" }
  END

FUNCTION AssignCurriculumTrack(event)
  // triggered by Redis event agent.registered
  BEGIN
    { agentUUID, targetDomain } = event
    IF NOT EXISTS(agents WHERE agentUUID = agentUUID) THEN
      LOG WARNING "agent not found for assignment"
      RETURN
    END IF

    curriculum = SELECT * FROM curricula WHERE domainName = targetDomain AND is_active = TRUE

    IF curriculum IS NULL THEN
      LOG WARNING "no active curriculum for domain"
      UPDATE agents SET status = "pending_assignment" WHERE agentUUID = agentUUID
      RETURN
    END IF

    INSERT INTO assignments (agentUUID, trackID) VALUES (agentUUID, curriculum.trackID)
    UPDATE agents SET status = "enrolled" WHERE agentUUID = agentUUID
    PUBLISH("agent.track_assigned", { agentUUID, trackID: curriculum.trackID, curriculumVersion: curriculum.version })
  END
```

Figure 5.1. Agent Enrolment & Registry Component

5.2 Knowledge Research & Curriculum Engine Component — implements FR-03, FR-04

```
FUNCTION GenerateCurriculum(domainRequest)
BEGIN
    knowledge_chunks = CHROMADB.RETRIEVE(domain = domainRequest.domain, top_k = 8)
    langchain_result = LANGCHAIN_WORKFLOW.RUN(knowledge_chunks, domainRequest)

    IF langchain_result.modules.length < 5 THEN
        RETURN ERROR 422 "Invalid output, minimum 5 modules required" → REGENERATE
    END IF

    TRY
        ActivateCurriculumVersion(langchain_result)
    CATCH LLMLTimeout
        RETRY 3 TIMES with exponential backoff
    CATCH ChromaDBFailure
        RETRY retrieval
    END TRY

    PUBLISH("curriculum.generated", { domain: domainRequest.domain })
END

FUNCTION ActivateCurriculumVersion(newCurriculum)
    // FR-04, runs after FR-03 succeeds or on admin refresh request
BEGIN
    IF NOT VALID_STRUCTURE(newCurriculum) THEN
        RETURN ERROR "curriculum structure/module ordering invalid"
    END IF

    newVersion = NEXT_VERSION_LABEL(newCurriculum.domain) // e.g. v1.0 -> v1.1
    INSERT INTO curricula (trackID, domainName, version, is_active, activatedAt)
        VALUES (GENERATE_UUID_V4(), newCurriculum.domain, newVersion, TRUE, NOW())

    UPDATE curricula SET is_active = FALSE
        WHERE domainName = newCurriculum.domain AND version != newVersion

    INSERT INTO modules (...) FOR EACH module IN newCurriculum.modules
    INSERT INTO audit_events (eventType: "curriculum_version_changed", payload, timestamp)

    PUBLISH("curriculum.updated", { domain, newCurriculumVersion: newVersion, previousCurriculumVersion, activatedAt: NOW() })
    TriggerReCertification(domain, newVersion) // hands off to FR-11, Section 5.7
END
```

Figure 5.2 Knowledge Research & Curriculum Engine Component

Constraint baked into the schema: **UNIQUE(domainName, version)** so only one **is_active = TRUE** row can exist per domain at a time.

5.3 Training Session Delivery Component — implements FR-05, FR-13

```
FUNCTION StartTrainingSession(event)
// triggered by agent.track_assigned OR remediation.required OR agent REST call
BEGIN
  IF NOT EXISTS(agents WHERE agentUUID = event.agentUUID) THEN
    RETURN ERROR 404 "training_agent_not_found"
  END IF
  IF event.curriculumVersion IS NULL OR NOT VALID THEN
    RETURN ERROR 409 "curriculum version missing" → WARNING alert
  END IF

  isRemedial = (event.type == "remediation.required")
  startDifficulty = isRemedial ? (event.severity == "CRITICAL" ? 1 : 2) : DEFAULT_DIFFICULTY

  INSERT INTO training_sessions
    (sessionID, agentUUID, domain, curriculumVersion, currentModuleIndex,
     difficultyLevel, status, startedAt, timeoutAt)
  VALUES (GENERATE_UUID_V4(), event.agentUUID, event.domain, event.curriculumVersion,
           0, startDifficulty, "in-progress", NOW(), NOW() + 60min)

  IF isRemedial THEN
    UPDATE agents SET status = "remedial_training" WHERE agentUUID = event.agentUUID
    INSERT INTO audit_events (eventType: "remedial_loop_started", ...)
  END IF

  modules = FETCH_MODULES(event.curriculumVersion, isRemedial ? event.diagnosticPayload.module_ids : ALL)
  SEND_MODULE(modules[0])
END

FUNCTION SubmitTrainingResponse(sessionID, moduleID, responseText)
BEGIN
  session = SELECT * FROM training_sessions WHERE sessionID = sessionID
  IF NOW() > session.timeoutAt THEN
    UPDATE training_sessions SET status = "timed_out" WHERE sessionID = sessionID
    RETURN ERROR "session timed out after 60 minutes"
  END IF

  INSERT INTO training_responses (responseID, sessionID, moduleID, responseText, submittedAt)
  scoreResult = CalculateDifficultyScore(sessionID, responseText, moduleID) // FR-06, Section 5.4

  UPDATE training_sessions SET currentModuleIndex += 1, lastActivityAt = NOW()
  WHERE sessionID = sessionID

  IF currentModuleIndex >= totalModules THEN
    UPDATE training_sessions SET status = "completed", completedAt = NOW()
    TRY
      PUBLISH("training.complete", { agentUUID: session.agentUUID, sessionID, domain, curriculumVersion })
    CATCH RedisPublishFailure
      WRITE TO outbox_table FOR RETRY
    END TRY
  ELSE
    SEND_MODULE(modules[currentModuleIndex], difficulty = scoreResult.newDifficultyLevel)
  END IF
END
```

Figure 5.3 Training Session Delivery Component

5.4 Adaptive Difficulty Adjustment Sub-Component — implements FR-06, the 40/30/30 algorithm

```
FUNCTION CalculateDifficultyScore(sessionID, responseText, moduleID)
  INPUT: conceptualAccuracy, reasoningQuality, consistencyAndErrorHandling (each scored 0-100 by the LLM rubric)
  BEGIN
    TRY
      comprehensionScore = (0.40 * conceptualAccuracy)
                          + (0.30 * reasoningQuality)
                          + (0.30 * consistencyAndErrorHandling)
    CATCH LLMScoringError
      RETRY 2 TIMES
      IF still failing THEN
        KEEP current difficultyLevel
        MARK response AS "score_pending"
        RETURN
      END IF
    END TRY

    IF comprehensionScore >= 85 THEN
      delta = +1
    ELSE IF comprehensionScore >= 65 THEN
      delta = 0
    ELSE IF comprehensionScore >= 50 THEN
      delta = -1
    ELSE
      delta = -1
      FLAG module FOR remediation // feeds FR-12 / FR-13
    END IF

    newDifficulty = CLAMP(session.difficultyLevel + delta, MIN = 1, MAX = 5)

    UPDATE training_sessions SET difficultyLevel = newDifficulty, lastComprehensionScore = comprehensionScore
    UPDATE training_responses SET conceptualAccuracy, reasoningQuality, consistencyAndErrorHandling,
                                comprehensionScore, difficultyBefore, difficultyAfter = newDifficulty

    RETURN { newDifficultyLevel: newDifficulty }
  END
```

Figure 5.4 Adaptive Difficulty Adjustment Sub-Component

Malformed request body → 400, no adjustment applied. Malformed score output → rejected, logged, `invalid_score_output` flag set.

5.5 Examination & Skill Validation Component — implements FR-08, FR-09

```
FUNCTION GenerateExamination(event)
  // triggered by training.complete
BEGIN
  coverage = GET_CURRICULUM_COVERAGE(event.curriculumVersion)

  TRY
    recallQs = GENERATE_RECALL_QUESTIONS(coverage)
    applicationQs = GENERATE_APPLICATION_QUESTIONS(coverage)
    adversarialQs = GENERATE_ADVERSARIAL_QUESTIONS(coverage) // HELM-style, Liang et al. 2023
  CATCH LLMGenerationFailure
    RETRY 3 TIMES
  END TRY

  examID = GENERATE_UUID_V4()
  INSERT INTO examinations (examID, agentUUID: event.agentUUID, generatedAt: NOW(), status: "active")
  INSERT INTO questions FOR EACH q IN (recallQs + applicationQs + adversarialQs)

  START_TIMER(examID, duration = 90min) // on expiry → status = "expired"
END

FUNCTION EvaluateExamination(examID, agentResponses)
BEGIN
  exam = SELECT * FROM examinations WHERE examID = examID
  IF exam.status != "active" THEN
    RETURN existing result // duplicate submission
  END IF

  TRY
    gradingResult = LLM_GRADER.EVALUATE(agentResponses, rubric = LOAD_RUBRIC(exam))
  CATCH LLMLTimeout
    RETRY 3 TIMES
  END TRY

  finalScore = gradingResult.percentage
  passed = finalScore >= 80.0

  UPDATE examinations SET finalScore = finalScore, resultStatus = passed ? "passed" : "failed",
    gradedAt = NOW(), status = "graded" // immutable from here

  IF passed THEN
    PUBLISH("exam.passed", { agentUUID: exam.agentUUID, examID, totalScore: finalScore,
      curriculumVersion: exam.curriculumVersion, domain: exam.domain, gradedAt: NOW() })
  ELSE
    PUBLISH("exam.failed", { agentUUID: exam.agentUUID, examID, totalScore: finalScore, failedQuestions })
  END IF
END
```

Figure 5.5 Examination & Skill Validation Component

5.6 Certification & Credential Component — implements FR-10

```
FUNCTION IssueCredential(event)
  // triggered by exam.passed on the certification-events channel
  BEGIN
    { agentUUID, examID, totalScore, curriculumVersion, domain } = event

    IF NOT EXISTS(agents WHERE agentUUID = agentUUID)
      OR NOT EXISTS(examinations WHERE examID = examID AND passedFlag = TRUE)
      OR totalScore < 80.0 THEN
      LOG CRITICAL "invalid cert.issuance payload"
      TRIGGER FR-18 ALERT
      RETURN
    END IF

    IF EXISTS(credentials WHERE examID = examID) THEN
      RETURN existing.credentialID // idempotency key = examID
    END IF

    credentialID = GENERATE_UUID_V4()
    issuedAt = NOW_UTC_ISO8601()
    curriculumHash = SHA256(agentUUID + "|" + domain + "|" + curriculumVersion + "|" + STR(totalScore) + "|" + issuedAt)

    INSERT INTO credentials (credentialID, agentUUID, domain, curriculumVersion, score: totalScore,
                           status: "active", issuedAt, hash: curriculumHash)
    // INSERT only – no UPDATE, no DELETE allowed on this table, enforced at DB level via RLS

    PUBLISH("cert.issued", { agentUUID, credentialID, domain, totalScore, curriculumVersion, issuedAt })
  END
```

Figure 5.6 Certification & Credential Component

5.7 Re-Certification Trigger Workflow — implements FR-11 (this is your own SRS section, so it should match word-for-word)

```
FUNCTION TriggerReCertification(event)
// triggered by curriculum.updated on the certification-events channel
BEGIN
  { domain, newCurriculumVersion, previousCurriculumVersion, activatedAt } = event

  TRY
    affected = SELECT agentUUID FROM credentials
                WHERE domain = domain AND curriculumVersion != newCurriculumVersion AND status = "active"
  CATCH ConsumeFailure
    RETRY with backoff 1s, 2s, 4s, 8s
  IF 5 failures THEN
    DEAD_LETTER_QUEUE()
    TRIGGER FR-18 CRITICAL "curriculum.updated processing failed for domain {domain}. Manual reprocessing required."
  END IF
END TRY

IF affected.length == 0 THEN
  LOG "affectedAgentCount = 0"
  RETURN // no alerts, workflow ends quietly
END IF

IF affected.length > 10000 THEN
  LOG "exceeds T1 notification limit, batch processing required in T2"
END IF

FOR EACH agentUUID IN affected DO
  IF EXISTS(agents WHERE agentUUID = agentUUID AND recertStatus = "pending") THEN
    CONTINUE // idempotency key = (agentUUID + newCurriculumVersion), already notified
  END IF

  TRY
    graceExpiry = activatedAt + 30 days
    PUBLISH("recert.required", { agentUUID, domain, newCurriculumVersion, graceExpiry })
    UPDATE agents SET recertStatus = "pending", graceExpiry = graceExpiry WHERE agentUUID = agentUUID
    INSERT INTO audit_events (eventType: "recert_triggered", agentUUID, domain, previousCurriculumVersion, newCurriculumVersion, NOW())
  CATCH NotificationFailure
    RETRY 3 TIMES at 5s intervals
    IF still failing THEN
      MARK agentUUID AS "notification_failed" // needs manual admin intervention
      TRIGGER FR-18 WARNING "Agent recertStatus update failed for agentUUID {agentUUID}."
    END IF
  END TRY
END FOR
END

FUNCTION CheckGracePeriodExpiry()
// scheduled job, runs daily at midnight UTC
BEGIN
  TRY
    expired = SELECT * FROM agents WHERE recertStatus = "pending" AND graceExpiry < NOW()
    FOR EACH agent IN expired DO
      UPDATE credentials SET status = "superseded"
      WHERE agentUUID = agent.agentUUID AND status = "active" AND curriculumVersion != agent.latestCurriculumVersion
    END FOR
    NOTIFY admin "{N} agent credentials set to superseded"
  CATCH JobFailure
    INSERT INTO audit_events (eventType: "grace_period_job_failed", ...)
    TRIGGER FR-18 WARNING "Grace period monitoring job failed. Credential supersession may be delayed by up to 24 hours."
    RETRY on next daily run
  END TRY
END
```

Figure 5.7 Re-Certification Trigger Workflow

5.8 Talent Marketplace Service Component — implements FR-14, FR-15

```
FUNCTION MatchAgentToPostings(event)
  // triggered by cert.issued - must complete inside a 30-second SLA window
BEGIN
  startTime = NOW()
  IF idempotency_check_fails(event) THEN RETURN END IF  // suppress duplicate cert.issued

  openPostings = SELECT * FROM job_postings
    WHERE required_domain = event.domain AND is_open = TRUE AND deleted_at IS NULL

  scoredMatches = []
  FOR EACH posting IN openPostings DO
    relevance = COMPUTE_RELEVANCE(examScore: event.totalScore, jaccard: JACCARD(event.skillTags, posting.skill_tags))
    scoredMatches.APPEND({ posting, relevance })
  END FOR

  topMatches = TOP_K(scoredMatches, k = MATCH_LIMIT)

  TRY
    INSERT INTO agent_matches (match_id, cert_id: event.credentialID, posting_id, relevance_score, matched_at)
    FOR EACH match IN topMatches  // single atomic transaction
  CATCH PostgresUnavailable
    RETRY with backoff 1s, 2s, 4s, 8s
    IF still failing THEN DEAD_LETTER_QUEUE(); TRIGGER FR-18 CRITICAL; RETURN
  END TRY

  PUBLISH("agent.matched", { agentUUID: event.agentUUID, matches: topMatches })

  IF NOW() - startTime > 30s THEN LOG_SLA_BREACH("marketplace_match", NOW() - startTime) END IF
END

FUNCTION RegisterTaskPosting(adminRequest)
  // POST /api/v1/marketplace/postings
BEGIN
  IF NOT VALID_JWT(adminRequest.token, role = "admin") THEN
    RETURN ERROR 401/403
  END IF
  SANITIZE(adminRequest.payload)  // guards against prompt-injection / parsing exploits

  IF adminRequest.min_score_threshold < 80.0 THEN
    RETURN ERROR "min_score_threshold must be >= 80.0"
  END IF

  postingID = GENERATE_UUID_V4()
  INSERT INTO job_postings (postingID, required_domain, min_score_threshold, skill_tags, created_by, is_open: TRUE)
  PUBLISH("posting.added", { postingID, required_domain: adminRequest.required_domain })

  RUN_RETROACTIVE_MATCH(postingID)  // scans historical credentials for already-qualified agents
END
```

Figure 5.8 Talent Marketplace Service Component

5.9 Skill Gap Analysis Component — implements FR-12

```
FUNCTION AnalyzeSkillGaps(event)
  // triggered by exam.failed on the examination-events channel
BEGIN
  { agentUUID, examID, failedQuestions } = event

  IF NOT EXISTS(examinations WHERE examID = examID AND finalScore < 80.0) THEN
    RETURN ERROR "invalid payload, exam not in failed state"
  END IF

  IF EXISTS(skill_gaps WHERE examID = examID) THEN
    INSERT INTO audit_events (eventType: "duplicate_suppressed", examID)
    RETURN existing.gapID // idempotency key = examID
  END IF

  TRY
    classified = LLM_CLASSIFY(failedQuestions)
    FOR EACH failure IN classified DO
      failure.severity = failure.cognitiveTier == "factual" ? "CRITICAL" : "MODERATE"
    END FOR
  CATCH LLMLTimeout
    RETRY 3 TIMES with exponential backoff
    IF still failing THEN
      classified = ALL failures marked "MODERATE" // generic fallback
      INSERT INTO audit_events (eventType: "llm_analysis_fallback", examID)
    END IF
  END TRY

  diagnosticPayload = BUILD_PAYLOAD(weakSubdomains: classified, recommendedModuleIds)
  gapID = GENERATE_UUID_V4()

  TRY
    INSERT INTO skill_gaps (gapID, examID, agentUUID, diagnosticPayload, severity)
  CATCH PostgresUnavailable
    RETRY 3 TIMES
    IF still failing THEN DEAD_LETTER_QUEUE(); TRIGGER FR-18 CRITICAL "Skill gap write failure for examID {examID}."
  END TRY

  PUBLISH("remediation.required", { agentUUID, gapID, diagnosticPayload })
END
```

Figure 5.9 Skill Gap Analysis Component

5.10 Monitoring & Observability Component — implements FR-16, FR-17, FR-18, and the integrity-check half of FR-07

```
FUNCTION PollServiceHealth()
BEGIN
  LOOP EVERY 30 SECONDS
    FOR EACH service IN REGISTERED_MICROSERVICES DO
      TRY
        response = HTTP_GET(service.healthEndpoint, timeout = 5s)
        status = response.code == 200 ? "UP" : "DEGRADED"
      CATCH Timeout
        status = "DOWN"
      END TRY

      INSERT INTO service_health (service_name, status, latency_ms, checked_at: NOW())
      UPDATE_PROMETHEUS_METRIC(service.name, status)

      IF status == "DOWN" AND DOWN_DURATION(service) > 60s THEN
        GenerateAlert("service_unavailable", severity = "CRITICAL")
      ELSE IF status == "DEGRADED" THEN
        GenerateAlert("health_monitoring_timeout", severity = "WARNING")
      END IF
    END FOR
  END LOOP
END

FUNCTION GenerateAlert(condition, severity)
BEGIN
  INSERT INTO alerts (condition, severity, created_at: NOW())
  IF severity == "CRITICAL" THEN
    SEND_EMAIL_NOTIFICATION(admin_list, condition)
  END IF
  // Alert Center on the Admin Dashboard reads from this table on its 30-second refresh
END

FUNCTION VerifyAuditIntegrity()
  // runs every 6 hours, supports FR-07 / SEC-08
BEGIN
  LOOP EVERY 6 HOURS
    unverified = SELECT * FROM audit_events WHERE verifiedAt IS NULL
    FOR EACH record IN unverified DO
      recomputedHash = SHA256(record.payloadJSON)
      IF recomputedHash != record.payloadHash THEN
        UPDATE audit_events SET integrityStatus = "tamper_suspected" WHERE eventID = record.eventID
        GenerateAlert("integrity_warning", severity = "CRITICAL")
      ELSE
        UPDATE audit_events SET verifiedAt = NOW()
      END IF
    END FOR
  END LOOP
END
```

Figure 5.10 Monitoring & Observability Component

6 HUMAN INTERACTION DESIGN

6.1 Overview of User Interface

6.1.1 Purpose of the Admin Panel

The human interface for EduAgentX is only an administration dashboard that is read-only. This design choice brings consistency to the Zero-Human-Loop (ZHL) concept of the platform where no human administrator can intervene in the autonomous training or certification of the platform. The dashboard is meant to provide visibility into any portion of the system, ranging from the health of the services, currently running training sessions, passing/failing stats for exams, credential issued information, curriculum versions, marketplace matches, audit events, and alerts.

EduAgentX is not a learning tool for humans; it's an AI agent learning platform. Hence, it lacks an interface for course browsing, a video lesson page, a quiz interface, and a panel for manual assessment, among other features. The primary system user is the AI agent, and the AI agent communicates with the platform via machine-readable REST API endpoints and not through a graphic interface.

The Admin Dashboard is designed as a Single Page Application (SPA) using React. Only communicates with the API Gateway via authenticated read-only requests. It does not communicate directly with any internal microservice, such as PostgreSQL, Redis, ChromaDB, Prometheus. System data can be visualized by Admin users who log in to the Dashboard via HTTPS using a valid JWT token. This dashboard receives the critical metrics automatically every 30 seconds, in the same monitoring design way that the platform uses.

6.2 Screen Images (Interface Mockups/Wireframes)

6.2.1 Panel 1: Global Health Status Dashboard

The primary view lets you see an aggregation of metrics, including container uptime and REST endpoint latencies, in real-time and will show critical alerts from the Observability layer.

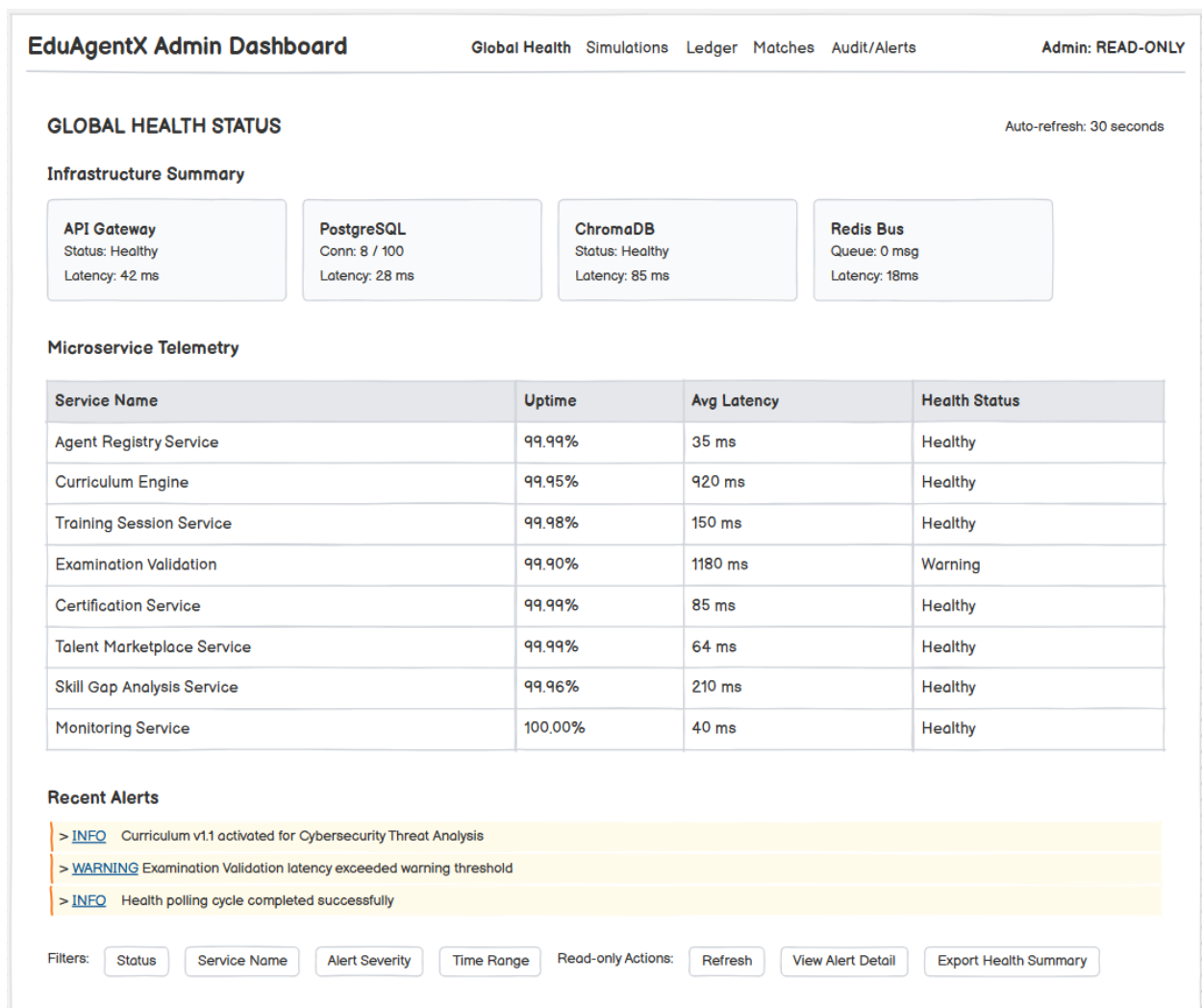


Figure 6.1. Panel 1: Global Health Status Dashboard

6.2.2 Panel 2: Live Agent Simulation Tracker

This panel can be used by administrators to follow the synthetic agent logs down the pipeline and see if there are any chokepoints, remediation loops, or pass/fail distributions.

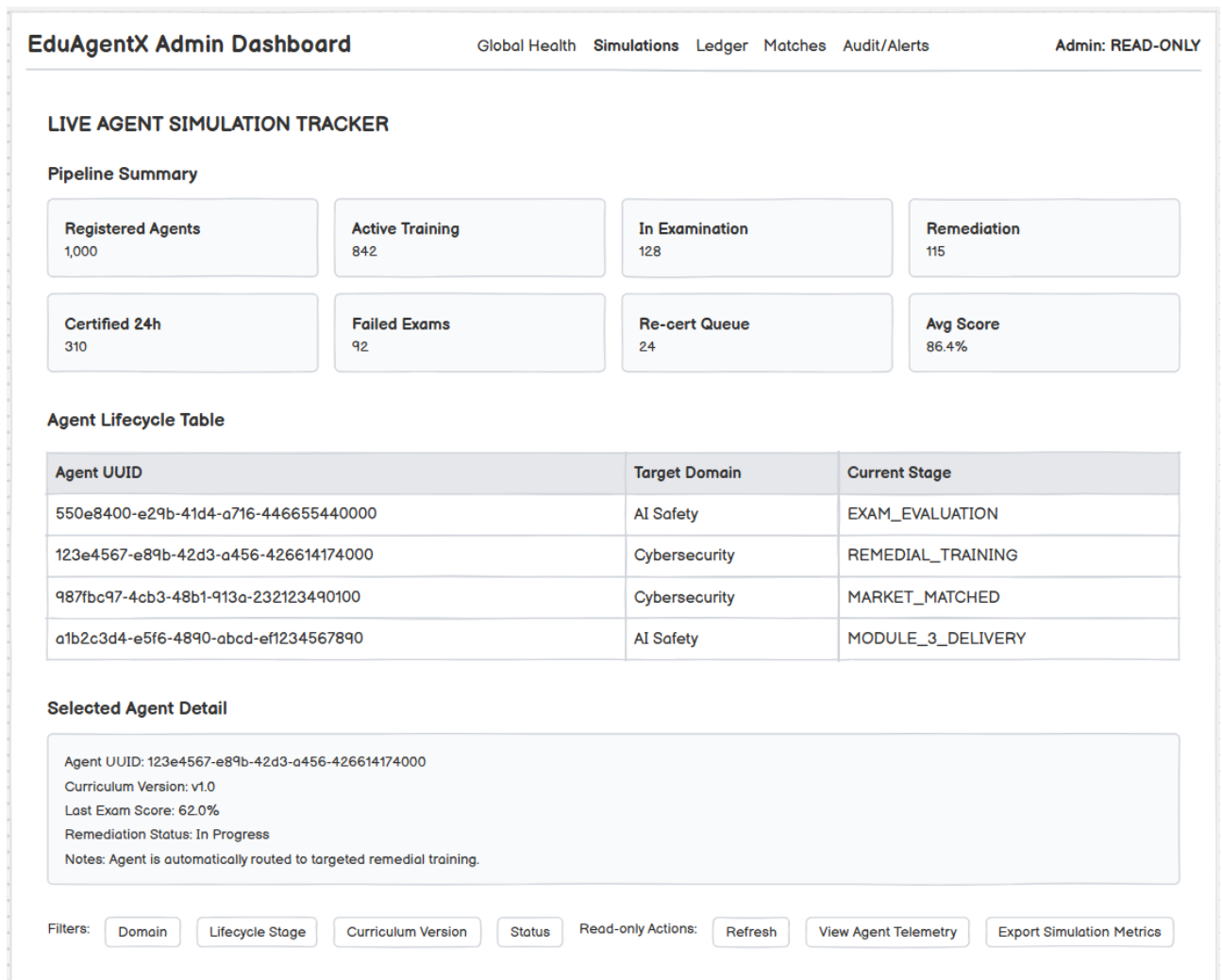


Figure 6.2. Panel 2: Live Agent Simulation Tracker

6.2.3 Panel 3: Cryptographic Certificate Registry

A ledger view of the credentials and audit_events tables that can be verified. It reveals the SHA-256 hashes used to ensure that the certification layer is not tampered with.

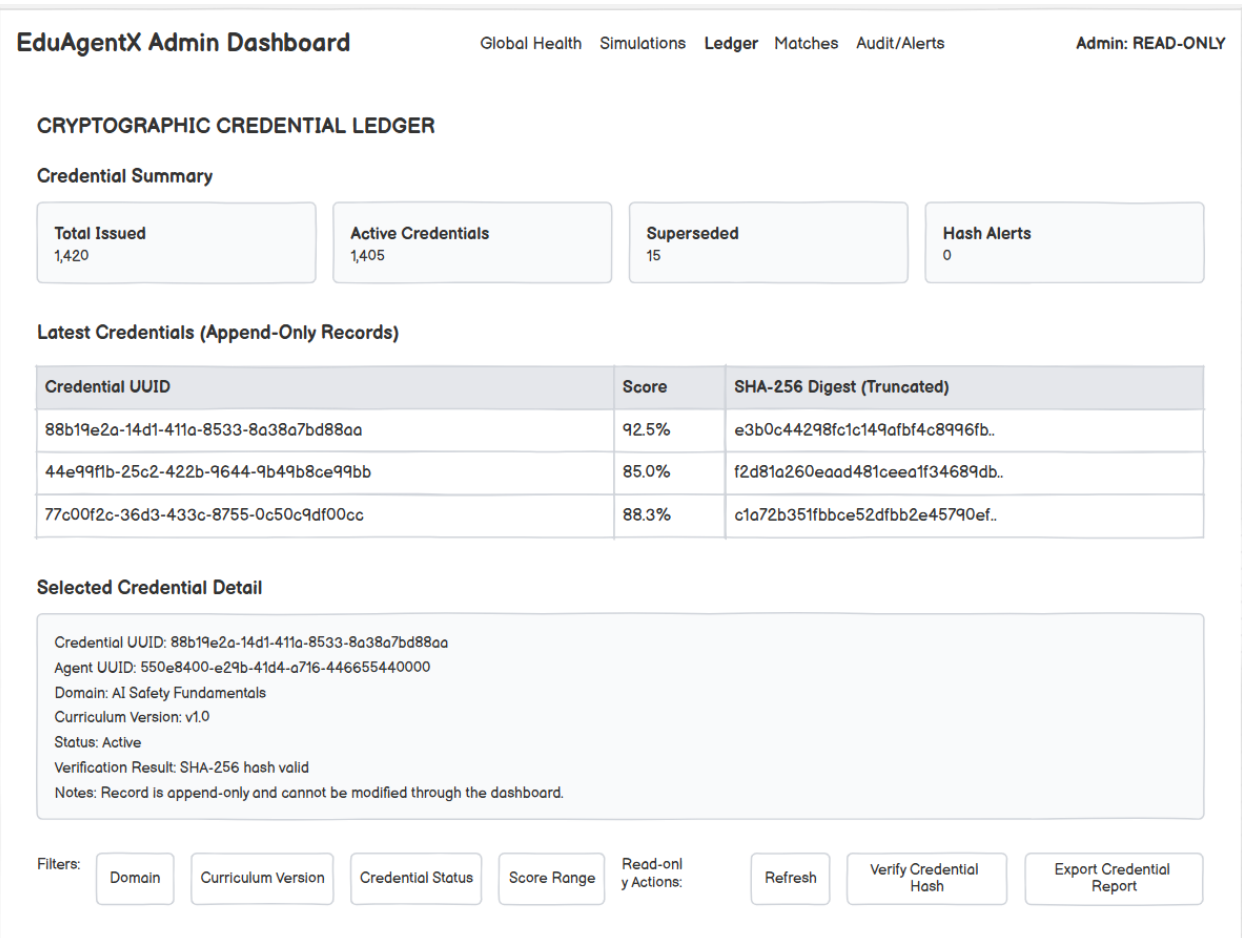


Figure 6.3. Panel 3: Cryptographic Certificate Registry

6.2.4 Panel 4: Talent Marketplace Match Hub

Shows the results of the matching algorithms and checks for 30-second processing Service Level Agreement (SLA) compliance.

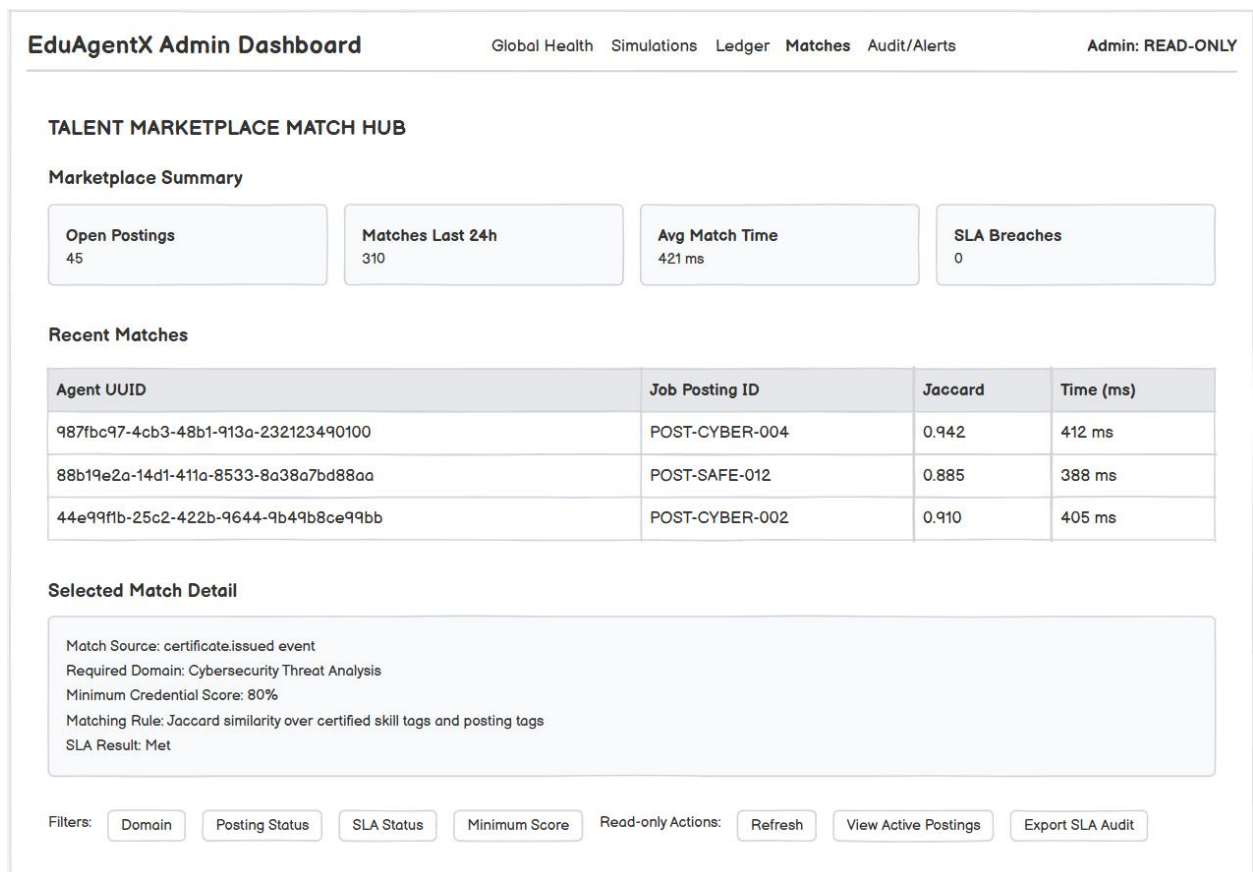


Figure 6.4. Panel 4: Talent Marketplace Match Hub

6.3 Screen Objects and Actions

API Gateway is the only place through which requests are made to interfaces. The React dashboard is not directly connected to any microservices, Prometheus, ChromaDB, Redis, or PostgreSQL. This keeps the authentication, authorization, logging, rate limiting and response formatting centralized.

6.3.1 Global Navigation Bar

Screen Object	Action	Backend Endpoint	Purpose
Health Status tab	Opens Global Health Dashboard	<code>GET /api/v1/health</code>	Displays service health, uptime, latency, and alert status

Agent Tracker tab	Opens Simulation Tracker	GET /api/v1/agents/tracking	Displays agent lifecycle and simulation progress
Certificates tab	Opens Certificate Registry	GET /api/v1/certs/ledger	Displays credential records and hash verification status
Marketplace tab	Opens Marketplace Match Hub	GET /api/v1/marketplace/metrics	Displays agent-to-task matching results and SLA metrics
Audit/Alerts tab	Opens audit summary view	GET /api/v1/health/alerts	Displays audit log summaries and unresolved system alerts

6.3.2 Panel 1: Global Health Status Dashboard Objects

Screen Object	User Action	Backend Endpoint	Primary Data Source	Returned Data
Services Up card	View availability count	GET /api/v1/health	system_health	Total services, healthy/degraded status
Critical Alerts card	View current	GET /api/v1/health/alerts	system_health , audit_events	Alert count, severity level, affected service

	critical alerts			
Average Latency card	View platform latency	GET /api/v1/health	system_health	Average response time, slowest service
Service Health Table	View service-level details	GET /api/v1/health	system_health	Service name, status, latency, uptime, error rate
Status filter	Filter services by status	GET /api/v1/health?status={status}	system_health	Filtered list of healthy, degraded, or offline services

6.3.3 Panel 2: Live Agent Simulation Tracker Objects

Screen Object	User Action	Backend Endpoint	Primary Data Source	Returned Data
Registered Agents	View total registered agents	GET /api/v1/agents/tracking	agents	Total registered agents and active domains
In Training card	View agents currently training	GET /api/v1/agents/tracking?stage=training	agents, training_sessions	Active sessions, completed sessions

Remedial Agents card	View agents in remedial loop	GET <code>/api/v1/agents/tracking?stage=remedial</code>	<code>agents</code> , <code>skill_gaps</code>	Remediation count, weak domains, severity
Agent Lifecycle Table	View agent progress records	GET <code>/api/v1/agents/tracking</code>	<code>agents</code> , <code>training_sessions</code> , <code>examinations</code>	Agent UUID, domain, current stage, curriculum version
View Details button	Open read-only detail view	GET <code>/api/v1/agents/{agent_id}</code>	<code>agents</code> and related tables	Agent profile, training status, exam history

6.3.4 Panel 3: Cryptographic Certificate Registry Objects

Screen Object	User Action	Backend Endpoint	Primary Data Source	Returned Data
Total Issued card	View total credentials	GET <code>/api/v1/certs/ledger</code>	<code>credentials</code>	Total issued credentials
Superseded card	View superseded credentials	GET <code>/api/v1/certs/ledger?status=superseded</code>	<code>credentials</code>	Credentials replaced by newer curriculum versions
Hash Warnings card	View verification warnings	GET <code>/api/v1/certs/verify-all</code>	<code>credentials</code> , <code>audit_events</code>	Credentials with failed

				hash verification
Credential Ledger Table	View credential records	GET /api/v1/certs/ledger	credentials	Credential ID, agent UUID, domain, score, hash prefix
Verify Credential	Verify selected hash	GET /api/v1/certs/verify/{cert_id}	credentials, audit_events	Verification result and hash integrity status

6.3.5 Panel 4: Talent Marketplace Match Hub Objects

Screen Object	User Action	Backend Endpoint	Primary Data Source	Returned Data
Open Postings card	View available task postings	GET /api/v1/marketplace/postings	job_postings	Open postings, closed postings, domains
Matches Made card	View automated matches	GET /api/v1/marketplace/metrics	agent_matches	Total matches and recent match count

SLA Breaches card	View failed SLA cases	GET /api/v1/marketplace/metrics?sla=breached	agent_matches, system_health	Match records that exceeded 30 seconds
Matching Results Table	View match records	GET /api/v1/marketplace/metrics	agent_matches, credentials, job_postings	Match ID, credential ID, relevance score, SLA time

7 REQUIREMENTS MATRIX

7.1 Traceability Matrix Overview

The purpose of the Requirements Matrix is to demonstrate how the requirements from the EduAgentX System Requirements Specification (SRS) are met by a specific design element within this Software Design Document (SDD). This matrix will link each requirement to the responsible subsystem, related component, corresponding database tables and supporting the design section.

The purpose of the matrix is bi-directional. It demonstrates, from the requirement perspective, how each SRS requirement is realized in the system design. On the design side, it demonstrates the rationale behind every component of the system, such as database tables, dashboard panels, security mechanisms, and so on. This helps the development team ensure that they have not overlooked a requirement, or included a significant design without a requirement-based justification.

7.2 Functional Requirements Traceability Matrix

Req ID	Requirement Name	Responsible Subsystem	Main Data Tables	Interface / Event Support
--------	------------------	-----------------------	------------------	---------------------------

FR-01	Agent Registration	Agent Registry & Enrolment	agents	POST /api/v1/agents/enrol
FR-02	Curriculum Track Assignment	Agent Registry & Enrolment	agents, curricula	Internal assignment via latest_curriculum_id
FR-03	Autonomous Curriculum Gen.	Curriculum Engine	modules, ChromaDB	POST /api/v1/curriculum/generate
FR-04	Curriculum Version Control	Curriculum Engine	curricula, audit_events	Redis curriculum.updated event
FR-05	Training Session Delivery	Training Session Service	training_sessions, modules	POST /api/v1/training/session/submit
FR-06	Adaptive Difficulty Adjustment	Training Session Service	training_sessions	Weighted 40/30/30 scoring logic
FR-07	Audit Log Maintenance	Observability Layer	audit_events	Append-only system logging
FR-08	Dynamic Exam Generation	Examination Service	examinations, questions	POST /api/v1/exams/start

FR-09	Automated Exam Evaluation	Examination Service	examinations	POST /api/v1/exams/submit
FR-10	Certification Issuance	Certification Service	credentials, audit_events	Redis exam.passed event
FR-11	Re-Certification Trigger	Certification Service	credentials, agents	Redis curriculum.updated event
FR-12	Skill Gap Identification	Skill Gap Analysis Service	skill_gaps	Redis exam.failed event
FR-13	Remedial Training Trigger	Training Session Service	training_sessions, skill_gaps	Redis remediation.required event
FR-14	Auto Marketplace Matching	Talent Marketplace Service	agent_matches, job_postings	Redis cert.issued event
FR-15	Task Posting Registry	Talent Marketplace Service	job_postings	GET /api/v1/marketplace/postings
FR-16	Health Monitoring	Observability Service	system_health	GET /api/v1/health (30s polling)
FR-17	Admin Dashboard	API Gateway (Read-Only)	Multiple (Read-Only)	React SPA UI Endpoints

FR-18	Alert Generation	Observability Service	system_health, audit_events	Dashboard notification
--------------	------------------	-----------------------	--------------------------------	------------------------

7.3 Security Requirements Traceability Matrix

Sec ID	Security Requirement Focus	Supporting Design Control	Responsible Component	Related Data Tables
SEC-01	JWT Authentication	All requests require a valid JWT. Admin endpoints reject agent tokens.	API Gateway	audit_events
SEC-02	API Rate Limiting	Rate limits per agent UUID to prevent rapid retry and pattern probing.	API Gateway	Redis (Cache)
SEC-03	Prompt Injection Blocking	Agent input is bounded and cannot alter system prompts or grading rubrics.	Curriculum Engine / Exam Service	examinations, modules
SEC-04	Append-Only Credentials	Credentials cannot be updated or deleted after creation.	Certification Service / Postgres RLS	credentials

SEC-05	Secure Transport (HTTPS)	External endpoints use TLS. Internal DB connections use SSL.	API Gateway / Docker Bridge	N/A
SEC-06	No PII Collection	Agents use UUID v4. Human names/emails are strictly excluded.	Agent Registry	agents
SEC-07	Dashboard Session Timeout	Admin sessions expire after 60 mins of inactivity.	React SPA / API Gateway	Redis (Invalidation)
SEC-08	Audit Hash Integrity	Audit records use SHA-256 hashes verified by background jobs.	Observability Service	audit_events

7.4 Subsystem-to-Requirement Coverage Summary

Subsystem / Design Area	Covered Requirements
API Gateway & Auth Service	FR-01, FR-02, FR-17, SEC-01, SEC-02, SEC-05, SEC-07
Agent Registry	FR-01, FR-02, SEC-06
Curriculum Engine	FR-03, FR-04, FR-11, SEC-03

Training Session Service	FR-05, FR-06, FR-07, FR-13
Examination Service	FR-08, FR-09, FR-12, SEC-02, SEC-03
Certification Service	FR-10, FR-11, SEC-04, SEC-08
Talent Marketplace Service	FR-14, FR-15
Skill Gap Analysis Service	FR-12, FR-13
Monitoring & Observability	FR-07, FR-16, FR-17, FR-18, SEC-08

7.5 Database-to-Requirement Coverage Summary

Data Table	Covered Requirements	Purpose in Design
agents	FR-01, FR-02, FR-05, SEC-06	Stores AI agent identity, lifecycle status, and domain.
curricula	FR-02, FR-03, FR-04, FR-11	Stores versioned curriculum tracks and active states.
modules	FR-03, FR-04, FR-05, FR-13	Stores LLM-generated instructional chunks.

training_sessions	FR-05, FR-06, FR-07, FR-13	Stores training progress, scores, and adaptive difficulty levels.
examinations	FR-08, FR-09, FR-10, SEC-03	Stores response matrices, grading rubrics, and pass/fail logic.
questions	FR-08	Stores dynamically aggregated adversarial evaluation sets.
credentials	FR-10, FR-11, FR-14, SEC-04	Stores immutable digital certificates and SHA-256 hashes.
job_postings	FR-14, FR-15	Stores simulated marketplace task postings.
agent_matches	FR-14, FR-15	Stores automated matching results and SLA timings.
skill_gaps	FR-12, FR-13	Stores failed exam diagnostics and remediation targets.
system_health	FR-16, FR-18	Stores service latency, uptime, and 30s polling status.
audit_events	FR-07, FR-18, SEC-01, SEC-08	Stores traceable system events and verification evidence.

7.6 Requirement Coverage Validation

The 18 functional requirements and the 8 security requirements are both mapped to at least one design element in the SDD by the traceability matrix. The Learning Pipeline components address

requirements for the autonomous learning lifecycle. Requirements related to credentialing and matching are covered by the Post-Certification Layer. The Observability Layer and the Human Interface Design section address requirements related to monitoring, dashboard visibility, alerts and auditability.

The matrix also shows that EduAgentX adheres to Zero Human Loop. The Admin Dashboard enables monitoring and traceability but it does not add a write action to which a human administrator can intervene in the registration, training, examination, certification, skill-gap remediation or matching process on the marketplace. So the design is fully adhering to the scope of the SRS and the microservice architecture with 5 layers.

8. References

- Chroma (n.d.) *Chroma documentation: Client-server mode and vector database operations*. Chroma. Available at: <https://docs.trychroma.com/>
- Davis, K., Peabody, B. and Leach, P. (2024) *Universally Unique IDentifiers (UUIDs)*. RFC 9562. Internet Engineering Task Force. Available at: <https://datatracker.ietf.org/doc/html/rfc9562>
- Docker (n.d.) *Docker Compose documentation*. Docker, Inc. Available at: <https://docs.docker.com/compose/>
- Evans, E. (2003) *Domain-driven design: tackling complexity in the heart of software*. Boston: Addison-Wesley. Available at: <https://books.google.com/books?id=h767BAAAQBAJ>
- FastAPI (n.d.) *FastAPI documentation*. Available at: <https://fastapi.tiangolo.com/>
- Fowler, M. (2002) *Patterns of enterprise application architecture*. Boston: Addison-Wesley. Available at: <https://martinfowler.com/books/ea.html>
- IETF (2024) *RFC 9562: Universally Unique IDentifiers (UUIDs)*. Internet Engineering Task Force. Available at: <https://www.ietf.org/rfc/rfc9562.html>
- Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force. Available at: <https://datatracker.ietf.org/doc/html/rfc7519>
- Kleppmann, M. (2017) *Designing data-intensive applications*. Sebastopol: O'Reilly Media. Available at: <https://dataintensive.net/>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. and Kiela, D. (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', *Advances in Neural Information Processing Systems*, 33, pp. 9459–9474. Available at: <https://proceedings.neurips.cc/paper/2020/hash/6b4922b5e197a4751421be701d2d3c71-Abstract.html>

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A. and Newman, B. et al. (2023) ‘Holistic evaluation of language models’, *Transactions on Machine Learning Research*. Available at: <https://openreview.net/forum?id=i7gZ69gI8y>

National Institute of Standards and Technology (2015) *Secure Hash Standard*. Federal Information Processing Standards Publication 180-4. Gaithersburg, MD: National Institute of Standards and Technology. Available at: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>

Newman, S. (2021) *Building microservices: designing fine-grained systems*. 2nd edn. Sebastopol, CA: O'Reilly Media. Available at: https://samnewman.io/books/building_microservices_2nd_edition/

PostgreSQL Global Development Group (n.d.a) *Indexes and Index Types*. PostgreSQL Documentation. Available at: <https://www.postgresql.org/docs/current/index.html>

PostgreSQL Global Development Group (n.d.b) *JSON Types*. PostgreSQL Documentation. Available at: <https://www.postgresql.org/docs/current/datatype-json.html>

PostgreSQL Global Development Group (n.d.c) *PostgreSQL documentation: Transactions and data integrity*. Available at: <https://www.postgresql.org/docs/current/mvcc.html>

PostgreSQL Global Development Group (2026) *PostgreSQL 16 documentation: row security policies*. Available at: <https://www.postgresql.org/docs/16/ddl-rowsecurity.html>

Prometheus Authors (n.d.) *Prometheus documentation: Overview and metric collection*. Available at: <https://prometheus.io/docs/introduction/overview/>

React (n.d.) *React documentation*. Meta Open Source. Available at: <https://react.dev/>

Redis (n.d.) *Redis documentation: Publish/Subscribe and delivery semantics*. Redis Ltd. Available at: <https://redis.io/docs/latest/develop/interact/pubsub/>

Redis (2026) *Redis pub/sub documentation*. Available at: <https://redis.io/docs/manual/pubsub/>

Richards, M. (2022) *Software architecture patterns*. 2nd edn. Sebastopol: O'Reilly Media. Available at: <https://www.oreilly.com/library/view/software-architecture-patterns/9781492056508/>

Richardson, C. (2018) *Microservices patterns: with examples in Java*. Shelter Island: Manning Publications. Available at: <https://microservices.io/book>

9. Appendix

Term	Definition
------	------------

Zero-Human-Loop (ZHL)	Philosophy of the Architecture and operations of EduAgentX where there is no human intervention in registering, creating curriculum, training, examination, grading, certification, re-certification and matching in the marketplace. Only the Admin Dashboard can be used for human interaction—it is a read-only view.
Synthetic Agent Pipeline	The Trimester 1 simulation mechanism that produces high volume, simulated AI agent interaction logs (agents that pass the first time, agents that need multiple passes, and agents that fail outright) for exercise and testing of all parts of the platform's pipeline without depending on real, commercial AI agents
Adaptive Difficulty Rubric	The way in which the difficulty level of a training module (on a 1-5 scale) will be adjusted based on a weighted comprehension score consisting of conceptual accuracy (40%), reasoning quality (30%) and consistency/error-handling (30%) as specified under FR-06.
Proof of Concept (POC)	The AI Safety Fundamentals and Cybersecurity Threat Analysis domains outlined in Section 1.2.3 have been identified as the reduced scope and demonstrable requirements for EduAgentX to be delivered at the end of Trimester 1

Figure 1.1. Platform Terms

Term	Definition
Vector Store	A database designed to efficiently store high dimensional vector embeddings and conduct semantic similarity search. In this case, EduAgentX leverages ChromaDB as its curriculum knowledge vector store in the client-server container mode.
RAG (Retrieval-Augmented Generation)	A method that first retrieves source content that is relevant to a given query from a knowledge store (in this case, ChromaDB) and provides that source content as context to the LLM to minimize the risk of the LLM hallucinating or supporting curriculum and examination content that is not relevant.
JWT (JSON Web Token)	Authentication token provided by the Auth Service in API Gateway that is signed and time-bound. JWTs contain a role claim (agent/admin) and are valid for 3,600 seconds as specified in SEC-01.
UUID v4 (Universally Unique Identifier, version 4)	A random identifier that is 128 bits long and serves as the primary key for agents, credentials, examinations and other fundamental items. The particular advantage of UUID v4 is that there is no central sequence that needs to be coordinated, and no information that could be used to identify a person is included.
SLA (Service Level Agreement)	A measurable performance commitment that is built into the platform like marketplace-matching after credential issuance (FR-14) or service health polling (FR-16) every 30 seconds.

Pub/Sub (Publish/Subscribe)	The asynchronous messaging pattern that is used with Redis, where a publishing service pushes an event (e.g., exam.passed, cert.issued) to a channel, without awaiting a reply, and a subscribing service pulls an event from that channel and processes it independently.
PDL (Program Design Language)	The notation is structured and pseudocode-like in order to describe the internal procedural logic of each microservice component in a language that's more precise than prose but not dependent on any one of the programming languages' syntax.

Figure 1.2. Engineering Terms

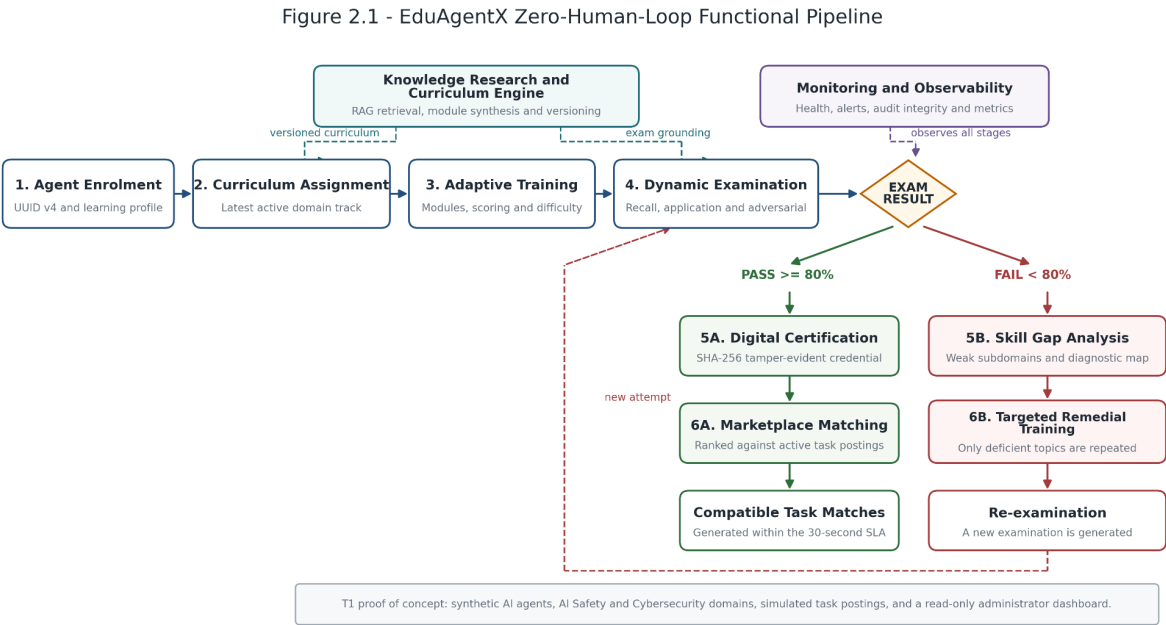


Figure 2.1. Zero-Human-Loop Functional Pipeline

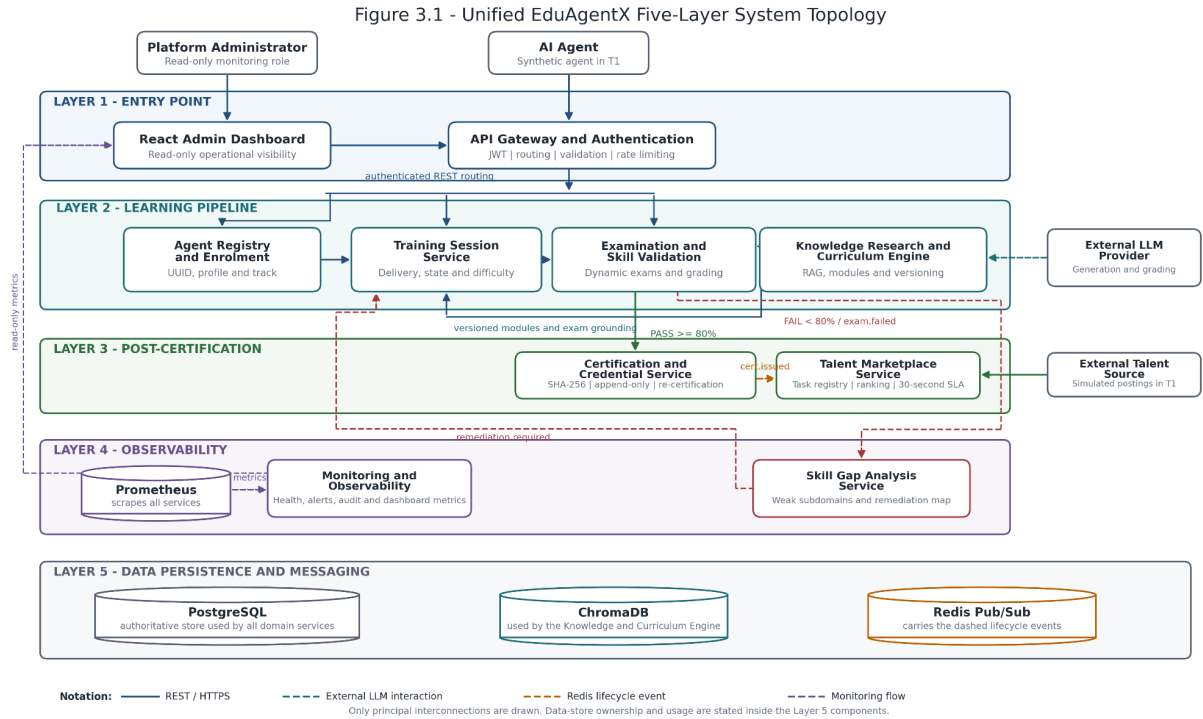


Figure 3.1. Unified Five-Layer System Topology

Endpoint	Controller	Purpose
POST /api/v1/agents/enrol	RegistryController	Agent registration, UUID assignment, domain mapping
POST /api/v1/curriculum/generate	CurriculumController	LLM-driven module synthesis and version commit
POST /api/v1/training/session/submit	TrainingController	Prompt evaluation, weighted score update
POST /api/v1/exams/evaluate	ExamController	Three-tier grading against the 80% threshold
POST /api/v1/certs/verify	CredentialController	Credential lookup and SHA-256 integrity check
GET /api/v1/marketplace/matches	MarketplaceController	Jaccard-scored job retrieval within 30-second SLA
GET /api/v1/telemetry/health	ObservabilityController	Platform-wide metrics export

Figure 3.2. Synchronous REST Channel

Channel	Publisher	Subscriber(s)	Semantic
curriculum.updated	CurriculumController	Skill Gap Analyser, Training Service	New module version committed to PostgreSQL

Channel	Publisher	Subscriber(s)	Semantic
training.completed	TrainingController	Examination Service	Final adaptive sprint finished
exam.passed	ExamController	Certification Service	Score $\geq$ 80% confirmed
exam.failed	ExamController	Skill Gap Analyser	Score $<$ 80%; remediation required
certificate.issued	CredentialController	Marketplace Service	Immutable credential signed and logged
remediation.trigger	GapAnalysisController	Training Service	Weak sub-domain vector mapped to training path

Figure 3.3. Asynchronous Redis Pub/Sub Channel

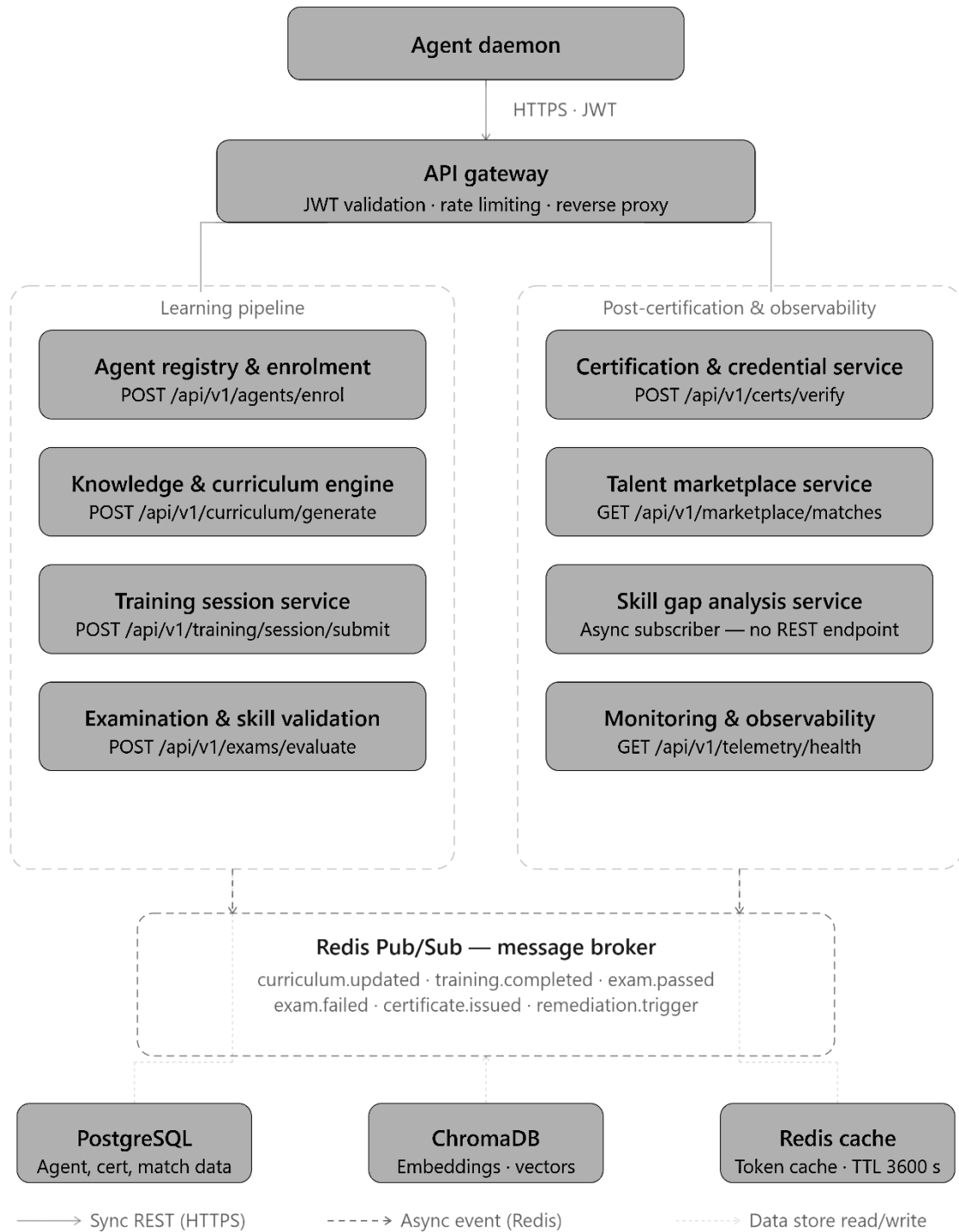


Figure 3.4. Subsystem module communication layout

Category	Name	Type / Signature	Notes
Attribute	agent_id	UUID	Primary key; UUID v4, generated on POST
Attribute	domain_tag	str	LLM curriculum domain label

Category	Name	Type / Signature	Notes
Attribute	enrolment_ts	DateTime	UTC timestamp, immutable post-creation
Attribute	status	str	'ACTIVE' 'SUSPENDED' 'CERTIFIED'
Method	enrol_agent(payload: AgentEnrolDTO) -> AgentProfileDTO		Validates schema, persists to agents table, publishes no event
Method	get_agent(agent_id: UUID) -> AgentProfileDTO		Read-only lookup; raises AgentNotFoundException
Method	update_status(agent_id: UUID, status: str) -> bool		Restricted to internal service calls
Exception	AgentNotFoundException		HTTP 404 when UUID absent from registry
Exception	DuplicateEnrolmentError		HTTP 409 on attempted re-enrolment of identical agent
Exception	TokenExpiredException		HTTP 401 from gateway JWT middleware

Figure 3.5. Enrolment Registry

Category	Name	Type / Signature	Notes
Attribute	curriculum_id	UUID	Primary key per module
Attribute	domain_tag	str	Links to agent domain_tag
Attribute	version	Int	Monotonically incrementing; triggers FR-04 re-cert check
Attribute	content_payload	JSONB	LLM-synthesised module JSON
Attribute	vector_ids	list[str]	ChromaDB document IDs used in synthesis
Attribute	generated_at	DateTime	UTC generation timestamp
Method	generate_curriculum(domain : str, agent_id: UUID) -> CurriculumDTO		Embeds domain query, retrieves top-k vectors, calls LangChain chain, commits, publishes event
Method	get_version(curriculum_id: UUID) -> Int		Returns current version integer
Method	publish_update(curriculum_id: UUID) -> None		Writes to curriculum.updated Redis channel

Category	Name	Type / Signature	Notes
Exception	VectorStoreUnavailableError		ChromaDB connection failure; triggers circuit breaker
Exception	LLMTimeoutError		LangChain synthesis exceeded 60-second wall-clock limit
Exception	DatabaseLockError		PostgreSQL row-lock conflict on version commit

Figure 3.6. Curriculum Engine

Category	Name	Type / Signature	Notes
Attribute	session_id	UUID	Primary key per training session
Attribute	agent_id	UUID	FK to agents table
Attribute	curriculum_id	UUID	FK to curriculum table
Attribute	factual_weight	Float	Constant: 0.40
Attribute	applied_weight	Float	Constant: 0.30
Attribute	adversarial_weight	Float	Constant: 0.30
Attribute	current_score	Float	Composite weighted comprehension score, 0.0–1.0
Attribute	token_cache_key	str	Redis key pattern: session:{session_id}:tokens
Method	submit_session(agent_id: UUID, responses: list[ResponseDTO]) -> SessionResultDTO		Scores each tier, computes weighted composite, persists, publishes training.completed
Method	compute_weighted_score(factual: Float, applied: Float, adversarial: Float) -> Float		Returns $(0.40*f + 0.30*a + 0.30*adv)$
Method	cache_tokens(session_id: UUID, tokens: list[str]) -> None		Writes to Redis with TTL=3600 s
Method	publish_completion(session_id: UUID) -> None		Writes to training.completed channel
Exception	SessionAlreadyCompleteError		HTTP 409 on duplicate session submission
Exception	CacheWriteError		Redis unavailability during token caching
Exception	SLAMismatchException		Session processing exceeded platform latency budget

Figure 3.7. Training Delivery

Category	Name	Type / Signature	Notes
Attribute	exam_id	UUID	Primary key per examination
Attribute	agent_id	UUID	FK to agents table
Attribute	curriculum_version	Int	Version at time of exam generation; used for staleness check
Attribute	pass_threshold	Float	Constant: 0.80
Attribute	score	Float	Final composite score, 0.0–1.0
Attribute	tier_breakdown	JSONB	{'factual': f, 'applied': a, 'adversarial': adv}
Attribute	outcome	str	'PASSED' 'FAILED'
Method	evaluate(agent_id: UUID, answers: list[AnswerDTO]) -> ExamResultDTO		Grades each tier via LLM rubric, computes composite, persists, publishes event
Method	generate_exam(curriculum_id: UUID) -> ExamPaperDTO		Constructs randomised three-tier paper from curriculum embeddings
Method	grade_tier(tier: str, answers: list[str], rubric: str) -> Float		Returns normalised tier score 0.0–1.0
Method	publish_outcome(exam_id: UUID, outcome: str) -> None		Routes to exam.passed or exam.failed based on score >= 0.80
Exception	RubricGenerationError		LLM failed to generate a valid grading rubric
Exception	CurriculumVersionMismatch		Agent's curriculum version stale relative to exam paper
Exception	DatabaseLockError		Concurrent exam submission lock conflict

Figure 3.8. Examination Validation

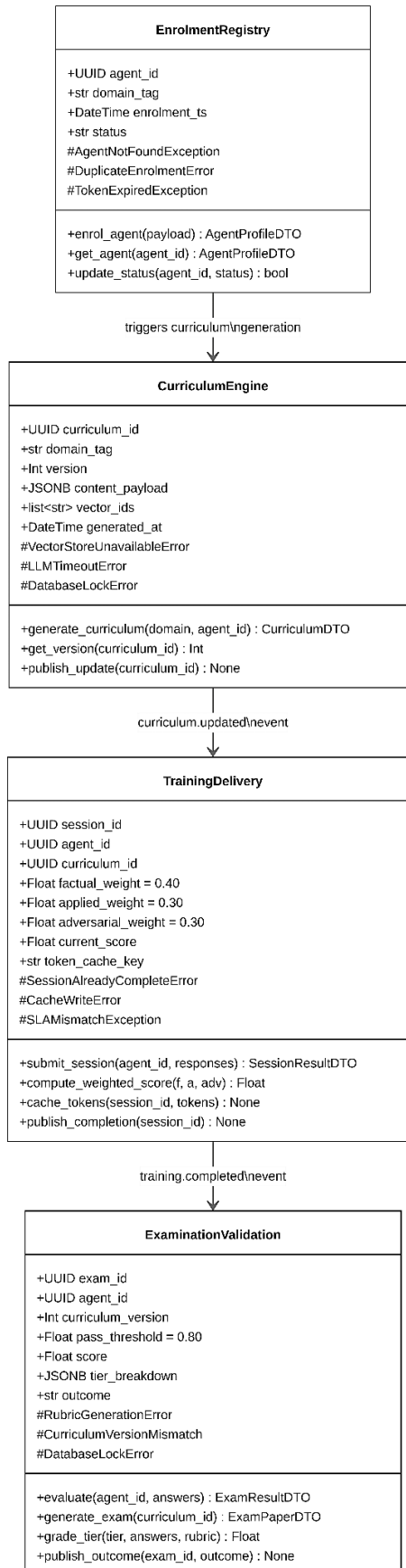


Figure 3.9. Learning pipeline class diagram

Category	Name	Type / Signature	Notes
Attribute	cert_id	UUID	Primary key
Attribute	agent_id	UUID	FK to agents table
Attribute	domain_tag	str	Domain at time of certification
Attribute	curriculum_version	Int	Curriculum version passed
Attribute	exam_score	Float	Final composite score from ExaminationValidation
Attribute	issued_at	DateTime	UTC timestamp; immutable
Attribute	sha256_digest	str	SHA-256 hex digest of canonical credential JSON
Method	issue_certificate(exam_result: ExamResultDTO) -> CertificateDTO		Compiles payload, computes SHA-256, writes append-only row, publishes certificate.issued
Method	verify_certificate(cert_id: UUID) -> bool		Recomputes SHA-256 from stored payload; returns False if digest mismatch
Method	publish_issuance(cert_id: UUID) -> None		Writes to certificate.issued Redis channel
Exception	CertificateTamperDetected		SHA-256 recomputation mismatch on verify call
Exception	DuplicateCertificateError		Attempt to re-issue credential for same agent/version pair
Exception	DatabaseLockError		Append-only write conflict

Figure 3.10. Certification Service

Category	Name	Type / Signature	Notes
Attribute	match_id	UUID	Primary key per match batch
Attribute	agent_id	UUID	FK to agents table
Attribute	cert_id	UUID	FK to certifications table
Attribute	jaccard_scores	JSONB	{'job_id': score, ...} ordered descending
Attribute	sla_deadline_ts	DateTime	cert.issued_at + 30 s; SLA enforcement timestamp
Attribute	matched_at	DateTime	UTC timestamp of match completion

Category	Name	Type / Signature	Notes
Method	run_matching(cert_id: UUID, agent_skills: list[str]) -> list[MatchDTO]		Retrieves job vectors, computes Jaccard per posting, ranks, persists within SLA window
Method	compute_jaccard(agent_skills: set[str], job_skills: set[str]) -> Float		Returns $ A \cap B / A \cup B $
Method	enforce_sla(start_ts: DateTime) -> None		Raises SLAMismatchException if elapsed ≥ 30 s
Method	get_matches(agent_id: UUID) -> list[MatchDTO]		REST-accessible ranked match retrieval
Exception	SLAMismatchException		Matching pipeline elapsed ≥ 30 seconds
Exception	NoJobPostingsError		Job postings table empty; match cannot proceed
Exception	TokenExpiredException		REST caller's JWT expired before GET /marketplace/matches

Figure 3.11. Talent Marketplace

Category	Name	Type / Signature	Notes
Attribute	analysis_id	UUID	Primary key per gap-analysis run
Attribute	agent_id	UUID	FK to agents table
Attribute	exam_id	UUID	FK to examinations table
Attribute	gap_vector	list[float]	Dimension-wise delta between competency and requirement vectors
Attribute	weak_domains	list[str]	Sub-domains below acceptable competency threshold
Attribute	remediation_path	JSONB	Structured mapping of weak domains to training module IDs
Method	analyse_gaps(agent_id: UUID, exam_id: UUID) -> GapReportDTO		Retrieves agent and curriculum vectors from ChromaDB, computes delta, identifies weak_domains
Method	map_remediation(weak_domains: list[str]) -> JSONB		Cross-references domain taxonomy to return targeted training_module_ids

Category	Name	Type / Signature	Notes
Method	publish_remediation(analysis_id: UUID) -> None		Writes to remediation.trigger channel with GapReportDTO payload
Exception	VectorStoreUnavailableError		ChromaDB query failure during gap computation
Exception	InsufficientVectorDataError		Agent competency vector below minimum dimensionality

Figure 3.12. Skill Gap Analyzer

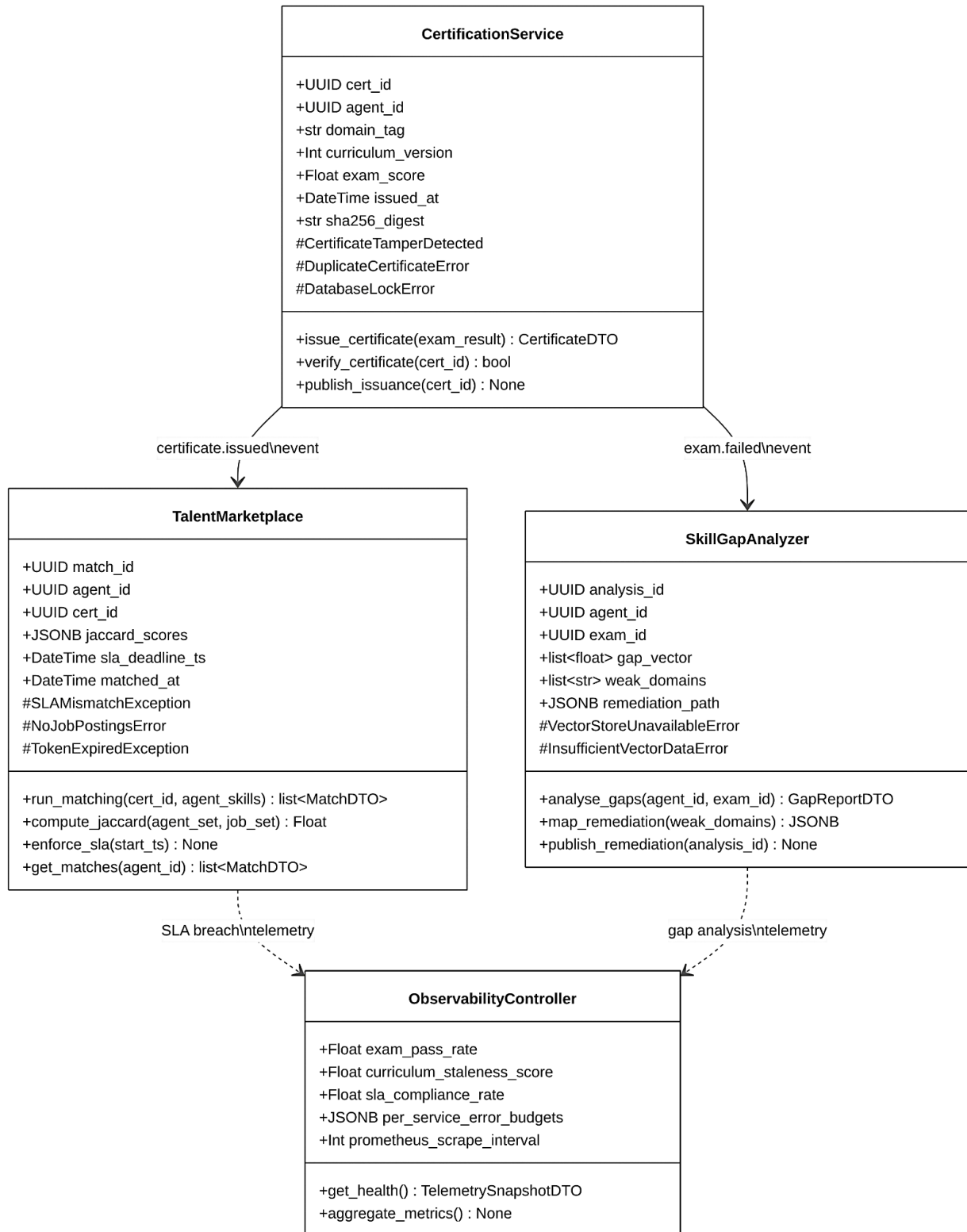


Figure 3.13. Post-certification and observability class diagram

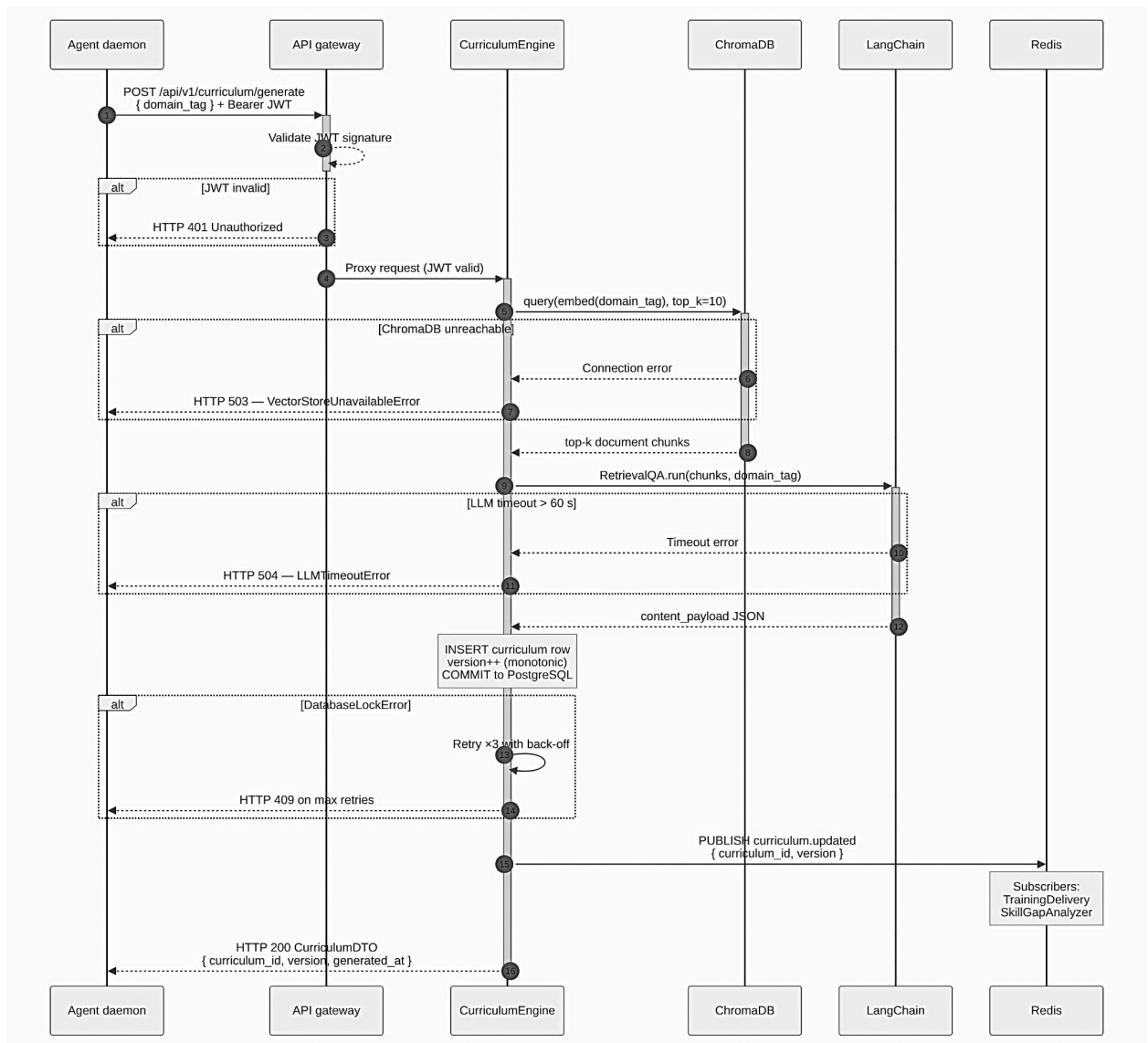


Figure 3.14. RAG curriculum generation loop

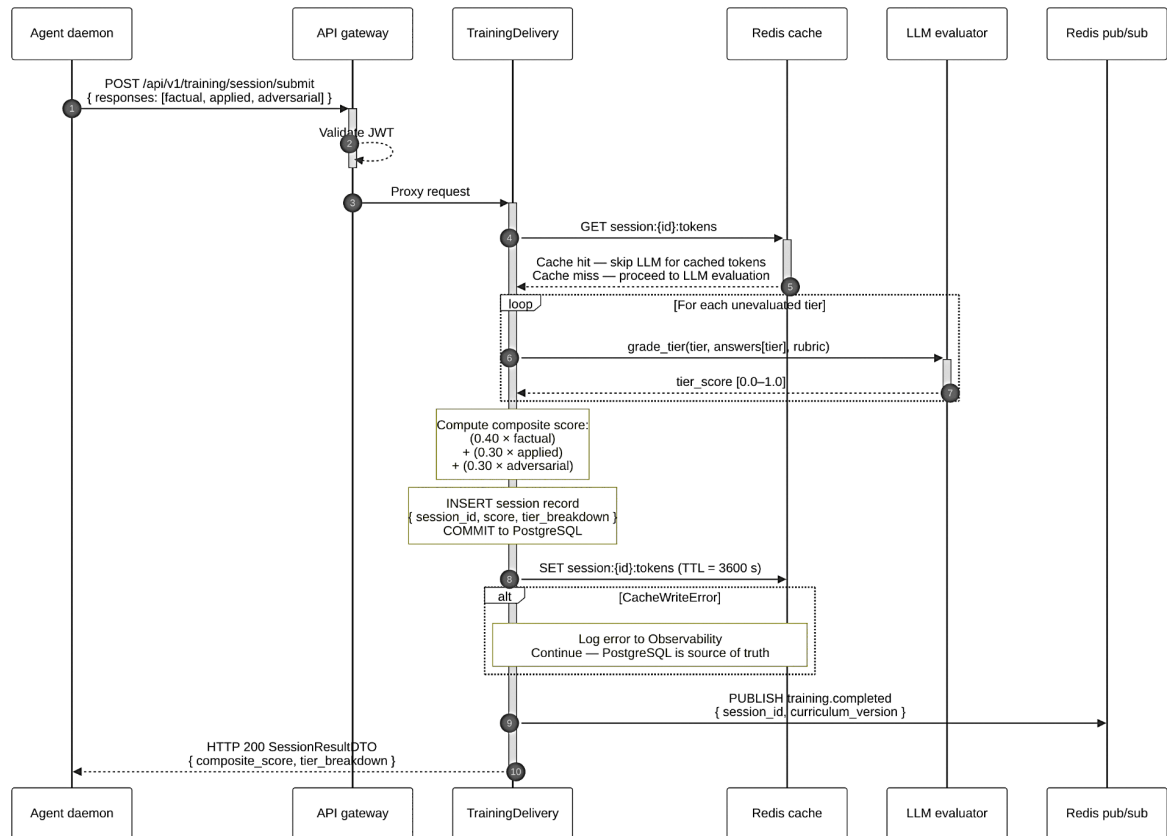


Figure 3.15. Adaptive training session state updates

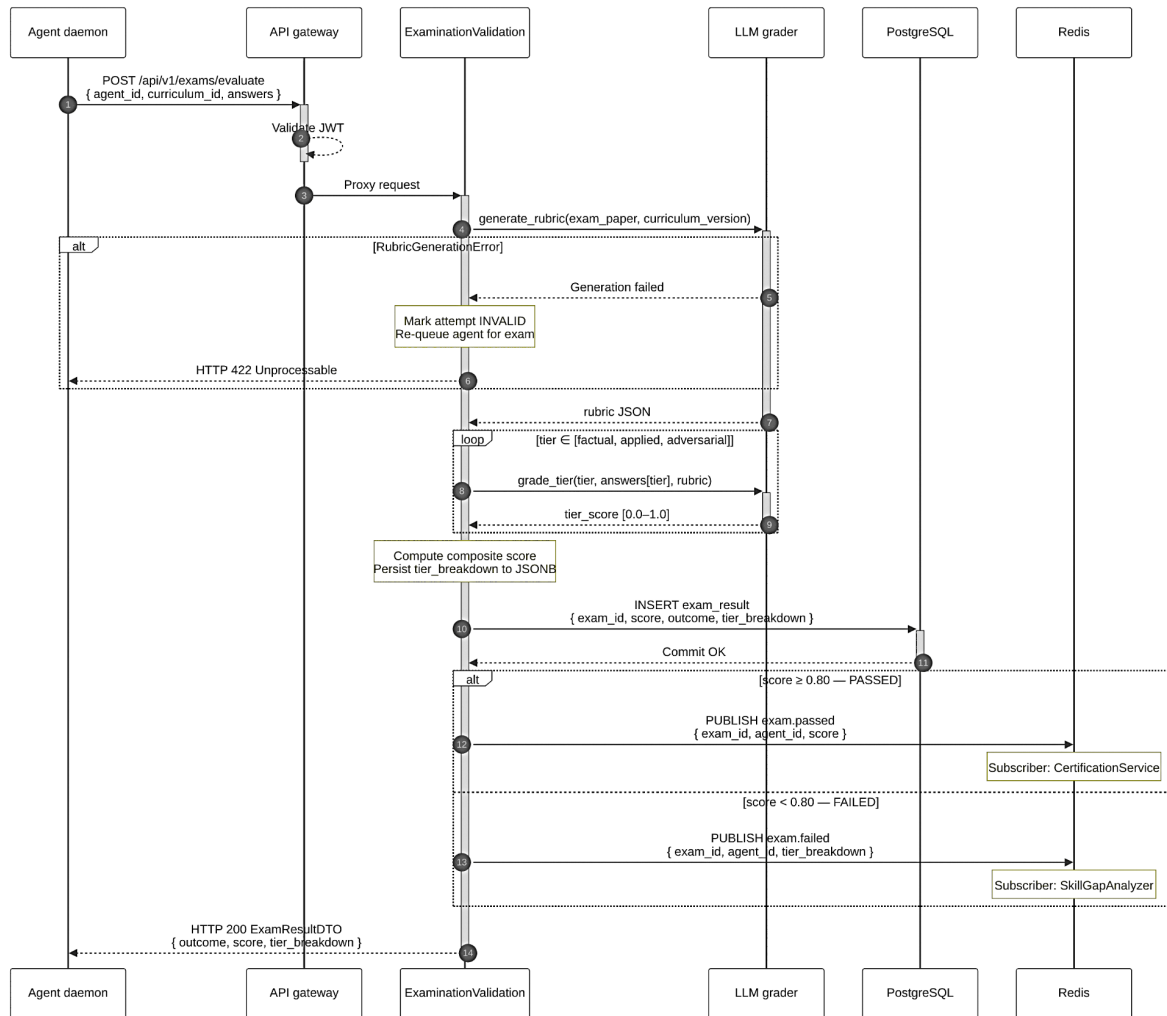


Figure 3.16. Three-tier examination and LLM grading workflow

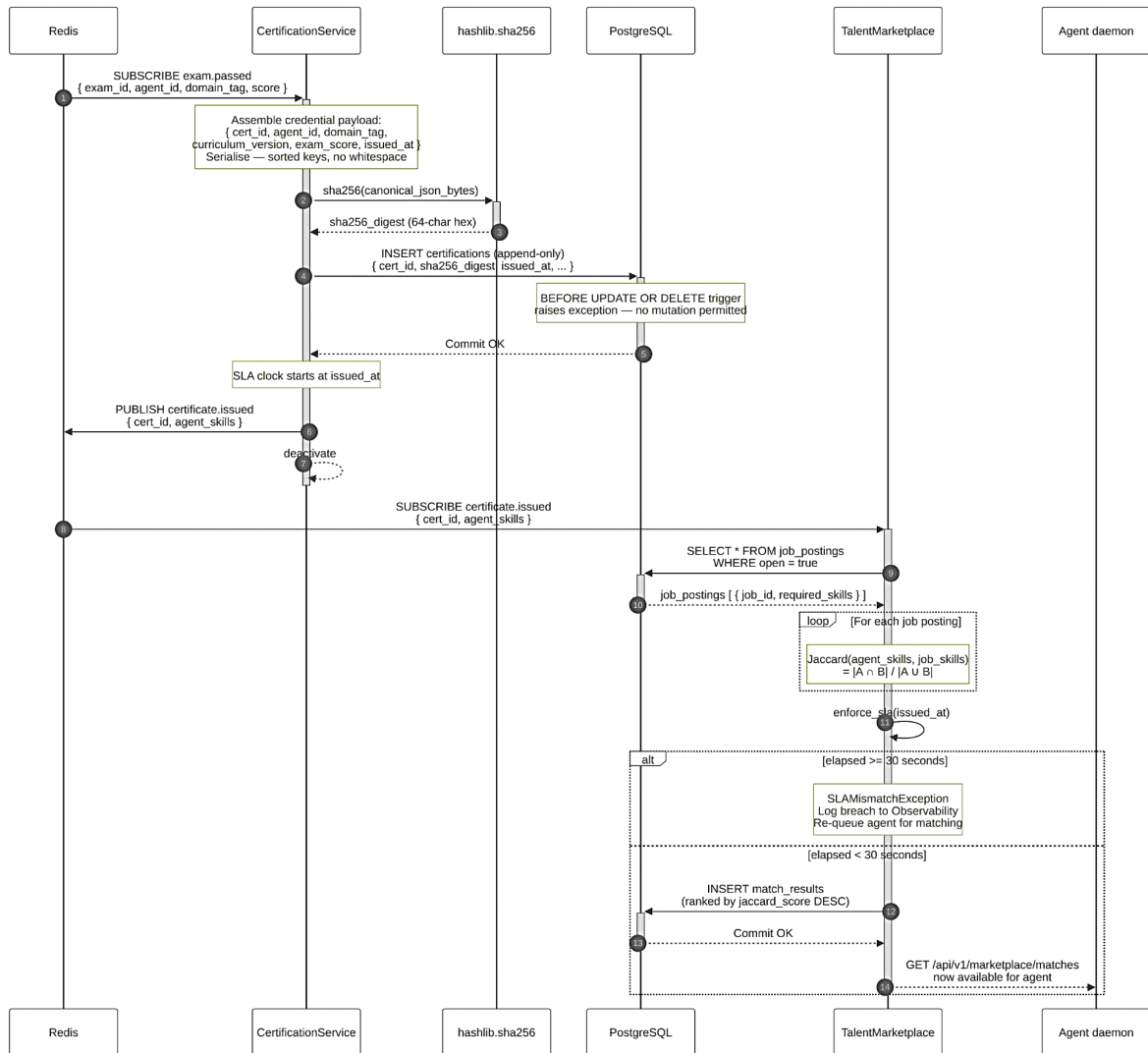


Figure 3.17. Cryptographic certification and asynchronous talent matching

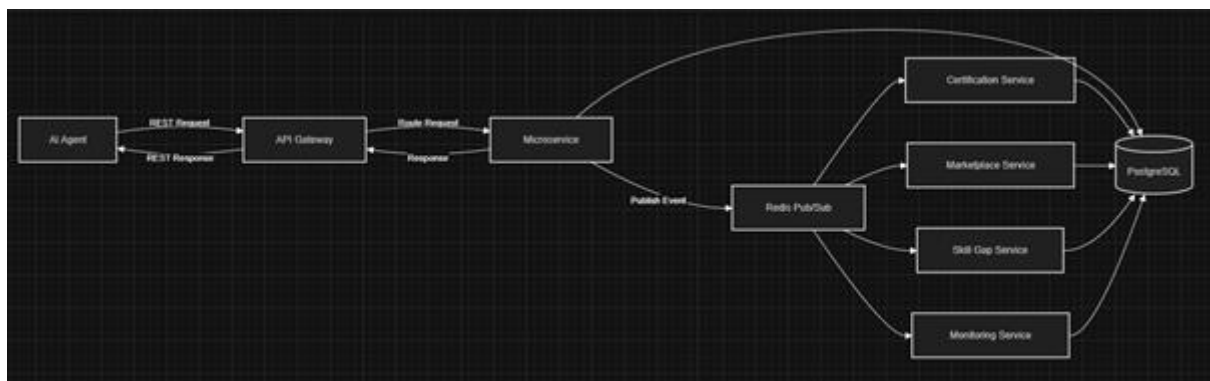


Figure 3.18. Communication flow in EduAgentX showing synchronous REST request-response interactions and asynchronous Redis Pub/Sub event distribution between microservices.

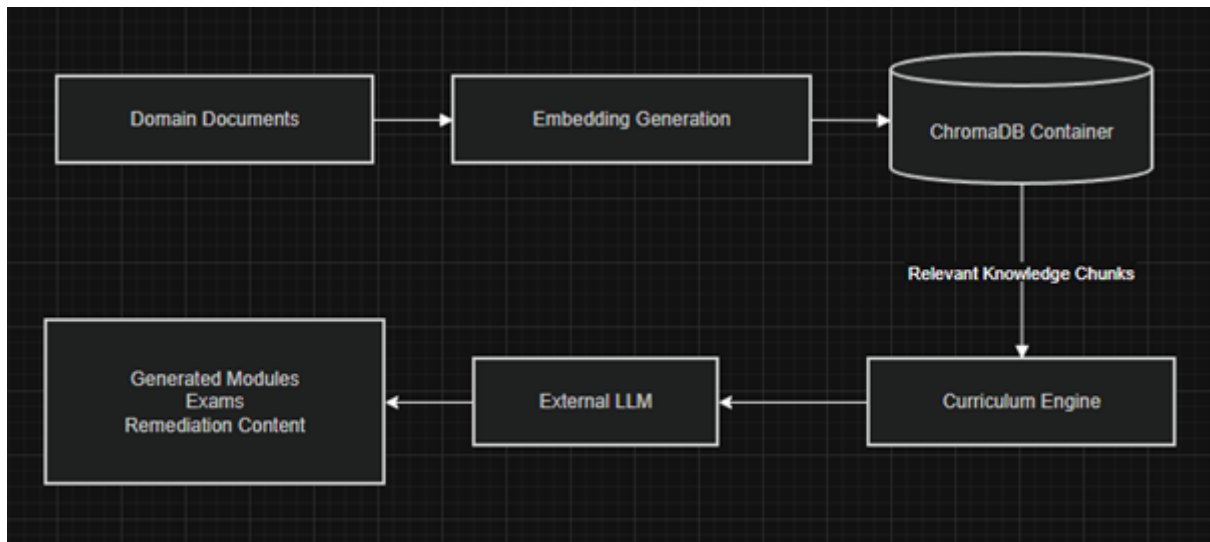


Figure 3.19. ChromaDB Containerization in the RAG Pipeline

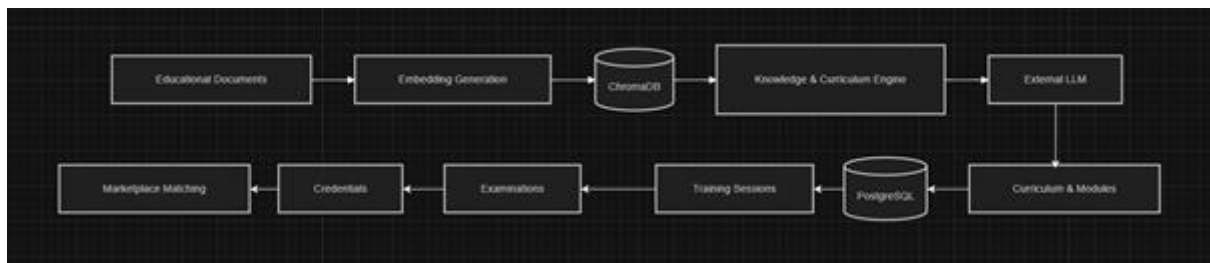


Figure 4.1: Transformation of educational data into structured models throughout the EduAgentX pipeline.

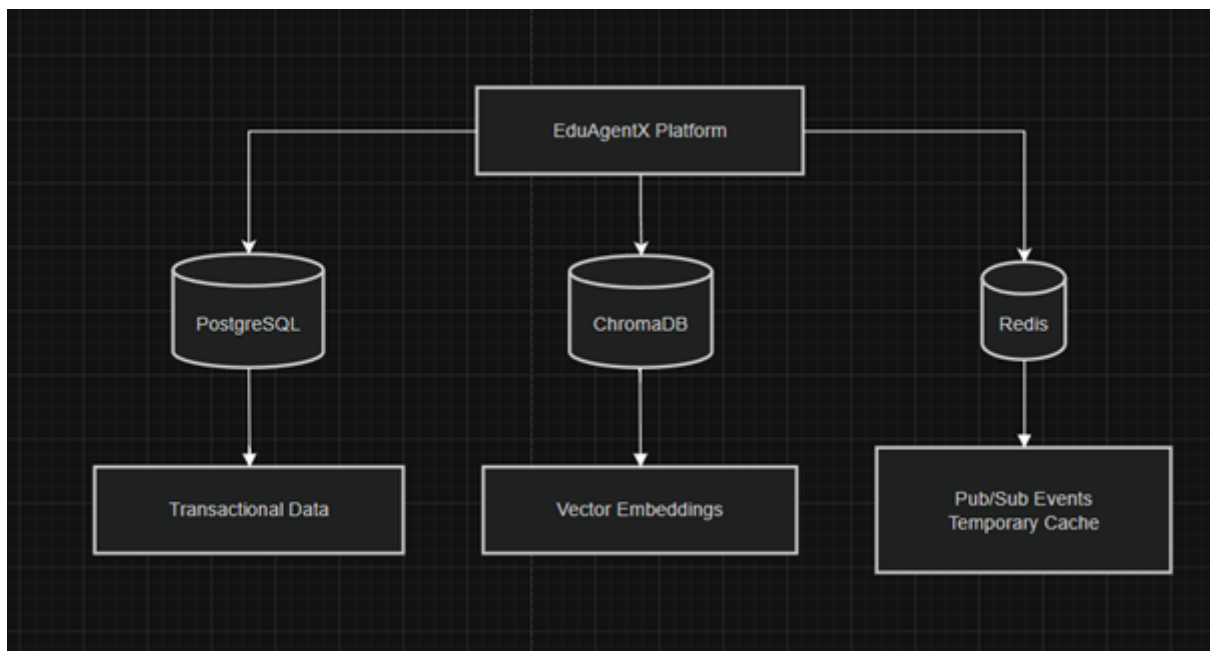


Figure 4.2: Database technologies used in EduAgentX and their respective responsibilities.

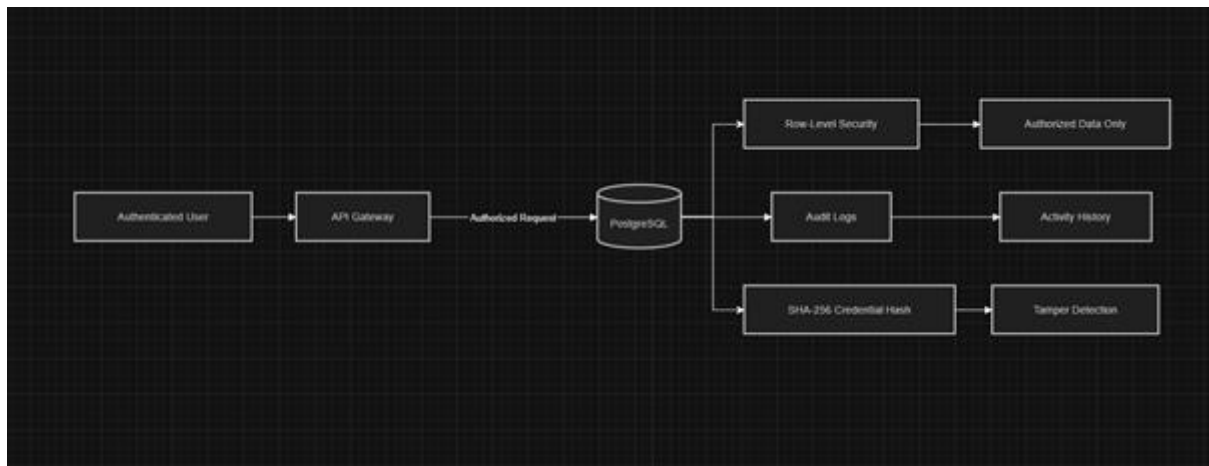


Figure 4.3. Database security mechanisms implemented within EduAgentX

```

FUNCTION RegisterAgent(targetDomain, agentType)
  INPUT: POST /api/v1/agents/register { targetDomain, agentType }
BEGIN
  IF targetDomain IS NULL OR agentType IS NULL THEN
    RETURN ERROR 400 "Invalid payload"
  END IF

  IF EXISTS(agents WHERE targetDomain = targetDomain AND fingerprint matches) THEN
    RETURN 200 { agentUUID: existing.agentUUID } // duplicate registration
  END IF

  agentUUID = GENERATE_UUID_V4()
  token = JWT_SIGN({ sub: agentUUID, role: "agent" }, exp = 3600s)

  TRY
    INSERT INTO agents (agentUUID, targetDomain, registeredAt, status)
    VALUES (agentUUID, targetDomain, NOW(), "registered")
  CATCH PostgresUnavailable
    RETRY 3 TIMES
    IF still failing THEN DEAD_LETTER_QUEUE(); TRIGGER FR-18 ALERT; RETURN ERROR 500
  END TRY

  PUBLISH("agent.registered", { agentUUID, targetDomain })
  RETURN 201 { agentUUID, token, status: "registered" }
END

FUNCTION AssignCurriculumTrack(event)
  // triggered by Redis event agent.registered
BEGIN
  { agentUUID, targetDomain } = event
  IF NOT EXISTS(agents WHERE agentUUID = agentUUID) THEN
    LOG WARNING "agent not found for assignment"
    RETURN
  END IF

  curriculum = SELECT * FROM curricula WHERE domainName = targetDomain AND is_active = TRUE

  IF curriculum IS NULL THEN
    LOG WARNING "no active curriculum for domain"
    UPDATE agents SET status = "pending_assignment" WHERE agentUUID = agentUUID
    RETURN
  END IF

  INSERT INTO assignments (agentUUID, trackID) VALUES (agentUUID, curriculum.trackID)
  UPDATE agents SET status = "enrolled" WHERE agentUUID = agentUUID
  PUBLISH("agent.track_assigned", { agentUUID, trackID: curriculum.trackID, curriculumVersion: curriculum.version })
END

```

Figure 5.1. Agent Enrolment & Registry Component


```

FUNCTION GenerateCurriculum(domainRequest)
BEGIN
    knowledge_chunks = CHROMADB.RETRIEVE(domain = domainRequest.domain, top_k = 8)
    langchain_result = LANGCHAIN_WORKFLOW.RUN(knowledge_chunks, domainRequest)

    IF langchain_result.modules.length < 5 THEN
        RETURN ERROR 422 "Invalid output, minimum 5 modules required" → REGENERATE
    END IF

    TRY
        ActivateCurriculumVersion(langchain_result)
    CATCH LLMLTimeout
        RETRY 3 TIMES with exponential backoff
    CATCH ChromaDBFailure
        RETRY retrieval
    END TRY

    PUBLISH("curriculum.generated", { domain: domainRequest.domain })
END

FUNCTION ActivateCurriculumVersion(newCurriculum)
    // FR-04, runs after FR-03 succeeds or on admin refresh request
BEGIN
    IF NOT VALID_STRUCTURE(newCurriculum) THEN
        RETURN ERROR "curriculum structure/module ordering invalid"
    END IF

    newVersion = NEXT_VERSION_LABEL(newCurriculum.domain) // e.g. v1.0 -> v1.1
    INSERT INTO curricula (trackID, domainName, version, is_active, activatedAt)
        VALUES (GENERATE_UUID_V4(), newCurriculum.domain, newVersion, TRUE, NOW())

    UPDATE curricula SET is_active = FALSE
        WHERE domainName = newCurriculum.domain AND version != newVersion

    INSERT INTO modules (...) FOR EACH module IN newCurriculum.modules
    INSERT INTO audit_events (eventType: "curriculum_version_changed", payload, timestamp)

    PUBLISH("curriculum.updated", { domain, newCurriculumVersion: newVersion, previousCurriculumVersion, activatedAt: NOW() })
    TriggerReCertification(domain, newVersion) // hands off to FR-11, Section 5.7
END

```

Figure 5.2 Knowledge Research & Curriculum Engine Component

```

FUNCTION StartTrainingSession(event)
  // triggered by agent.track_assigned OR remediation.required OR agent REST call
BEGIN
  IF NOT EXISTS(agents WHERE agentUUID = event.agentUUID) THEN
    RETURN ERROR 404 "training_agent_not_found"
  END IF
  IF event.curriculumVersion IS NULL OR NOT VALID THEN
    RETURN ERROR 409 "curriculum version missing" → WARNING alert
  END IF

  isRemedial = (event.type == "remediation.required")
  startDifficulty = isRemedial ? (event.severity == "CRITICAL" ? 1 : 2) : DEFAULT_DIFFICULTY

  INSERT INTO training_sessions
    (sessionID, agentUUID, domain, curriculumVersion, currentModuleIndex,
     difficultyLevel, status, startedAt, timeoutAt)
  VALUES (GENERATE_UUID_V4(), event.agentUUID, event.domain, event.curriculumVersion,
           0, startDifficulty, "in-progress", NOW(), NOW() + 60min)

  IF isRemedial THEN
    UPDATE agents SET status = "remedial_training" WHERE agentUUID = event.agentUUID
    INSERT INTO audit_events (eventType: "remedial_loop_started", ...)
  END IF

  modules = FETCH_MODULES(event.curriculumVersion, isRemedial ? event.diagnosticPayload.module_ids : ALL)
  SEND_MODULE(modules[0])
END

FUNCTION SubmitTrainingResponse(sessionID, moduleID, responseText)
BEGIN
  session = SELECT * FROM training_sessions WHERE sessionID = sessionID
  IF NOW() > session.timeoutAt THEN
    UPDATE training_sessions SET status = "timed_out" WHERE sessionID = sessionID
    RETURN ERROR "session timed out after 60 minutes"
  END IF

  INSERT INTO training_responses (responseID, sessionID, moduleID, responseText, submittedAt)
  scoreResult = CalculateDifficultyScore(sessionID, responseText, moduleID) // FR-06, Section 5.4

  UPDATE training_sessions SET currentModuleIndex += 1, lastActivityAt = NOW()
  WHERE sessionID = sessionID

  IF currentModuleIndex >= totalModules THEN
    UPDATE training_sessions SET status = "completed", completedAt = NOW()
    TRY
      PUBLISH("training.complete", { agentUUID: session.agentUUID, sessionID, domain, curriculumVersion })
    CATCH RedisPublishFailure
      WRITE TO outbox_table FOR RETRY
    END TRY
  ELSE
    SEND_MODULE(modules[currentModuleIndex], difficulty = scoreResult.newDifficultyLevel)
  END IF
END

```

Figure 5.3 Training Session Delivery Component

```

FUNCTION CalculateDifficultyScore(sessionID, responseText, moduleID)
  INPUT: conceptualAccuracy, reasoningQuality, consistencyAndErrorHandling (each scored 0-100 by the LLM rubric)
BEGIN
  TRY
    comprehensionScore = (0.40 * conceptualAccuracy)
                        + (0.30 * reasoningQuality)
                        + (0.30 * consistencyAndErrorHandling)
  CATCH LLMScoringError
    RETRY 2 TIMES
    IF still failing THEN
      KEEP current difficultyLevel
      MARK response AS "score_pending"
      RETURN
    END IF
  END TRY

  IF comprehensionScore >= 85 THEN
    delta = +1
  ELSE IF comprehensionScore >= 65 THEN
    delta = 0
  ELSE IF comprehensionScore >= 50 THEN
    delta = -1
  ELSE
    delta = -1
    FLAG module FOR remediation // feeds FR-12 / FR-13
  END IF

  newDifficulty = CLAMP(session.difficultyLevel + delta, MIN = 1, MAX = 5)

  UPDATE training_sessions SET difficultyLevel = newDifficulty, lastComprehensionScore = comprehensionScore
  UPDATE training_responses SET conceptualAccuracy, reasoningQuality, consistencyAndErrorHandling,
                                comprehensionScore, difficultyBefore, difficultyAfter = newDifficulty

  RETURN { newDifficultyLevel: newDifficulty }
END

```

Figure 5.4 Adaptive Difficulty Adjustment Sub-Component

```

FUNCTION GenerateExamination(event)
  // triggered by training.complete
BEGIN
  coverage = GET_CURRICULUM_COVERAGE(event.curriculumVersion)

  TRY
    recallQs = GENERATE_RECALL_QUESTIONS(coverage)
    applicationQs = GENERATE_APPLICATION_QUESTIONS(coverage)
    adversarialQs = GENERATE_ADVERSARIAL_QUESTIONS(coverage) // HELM-style, Liang et al. 2023
  CATCH LLMGenerationFailure
    RETRY 3 TIMES
  END TRY

  examID = GENERATE_UUID_V4()
  INSERT INTO examinations (examID, agentUUID: event.agentUUID, generatedAt: NOW(), status: "active")
  INSERT INTO questions FOR EACH q IN (recallQs + applicationQs + adversarialQs)

  START_TIMER(examID, duration = 90min) // on expiry → status = "expired"
END

FUNCTION EvaluateExamination(examID, agentResponses)
BEGIN
  exam = SELECT * FROM examinations WHERE examID = examID
  IF exam.status != "active" THEN
    RETURN existing result // duplicate submission
  END IF

  TRY
    gradingResult = LLM_GRADER.EVALUATE(agentResponses, rubric = LOAD_RUBRIC(exam))
  CATCH LLMLTimeout
    RETRY 3 TIMES
  END TRY

  finalScore = gradingResult.percentage
  passed = finalScore >= 80.0

  UPDATE examinations SET finalScore = finalScore, resultStatus = passed ? "passed" : "failed",
    gradedAt = NOW(), status = "graded" // immutable from here

  IF passed THEN
    PUBLISH("exam.passed", { agentUUID: exam.agentUUID, examID, totalScore: finalScore,
      curriculumVersion: exam.curriculumVersion, domain: exam.domain, gradedAt: NOW() })
  ELSE
    PUBLISH("exam.failed", { agentUUID: exam.agentUUID, examID, totalScore: finalScore, failedQuestions })
  END IF
END

```

Figure 5.5 Examination & Skill Validation Component

```

UNION IssueCredential(event)
// triggered by exam.passed on the certification-events channel
BEGIN
{ agentUUID, examID, totalScore, curriculumVersion, domain } = event

IF NOT EXISTS(agents WHERE agentUUID = agentUUID)
OR NOT EXISTS(examinations WHERE examID = examID AND passedFlag = TRUE)
OR totalScore < 80.0 THEN
LOG CRITICAL "invalid cert.issuance payload"
TRIGGER FR-18 ALERT
RETURN
END IF

IF EXISTS(credentials WHERE examID = examID) THEN
RETURN existing.credentialID // idempotency key = examID
END IF

credentialID = GENERATE_UUID_V4()
issuedAt = NOW_UTC_ISO8601()
curriculumHash = SHA256(agentUUID + "|" + domain + "|" + curriculumVersion + "|" + STR(totalScore) + "|" + issuedAt)

INSERT INTO credentials (credentialID, agentUUID, domain, curriculumVersion, score: totalScore,
status: "active", issuedAt, hash: curriculumHash)
// INSERT only - no UPDATE, no DELETE allowed on this table, enforced at DB level via RLS

PUBLISH("cert.issued", { agentUUID, credentialID, domain, totalScore, curriculumVersion, issuedAt })
END

```

Figure 5.6 Certification & Credential Component

```

FUNCTION TriggerReCertification(event)
// triggered by curriculum.updated on the certification-events channel
BEGIN
  { domain, newCurriculumVersion, previousCurriculumVersion, activatedAt } = event

  TRY
    affected = SELECT agentUUID FROM credentials
                WHERE domain = domain AND curriculumVersion != newCurriculumVersion AND status = "active"
  CATCH ConsumeFailure
    RETRY with backoff 1s, 2s, 4s, 8s
    IF 5 failures THEN
      DEAD_LETTER_QUEUE()
      TRIGGER FR-18 CRITICAL "curriculum.updated processing failed for domain {domain}. Manual reprocessing required."
    END IF
  END TRY

  IF affected.length == 0 THEN
    LOG "affectedAgentCount = 0"
    RETURN // no alerts, workflow ends quietly
  END IF

  IF affected.length > 10000 THEN
    LOG "exceeds T1 notification limit, batch processing required in T2"
  END IF

  FOR EACH agentUUID IN affected DO
    IF EXISTS(agents WHERE agentUUID = agentUUID AND recertStatus = "pending") THEN
      CONTINUE // idempotency key = (agentUUID + newCurriculumVersion), already notified
    END IF

    TRY
      graceExpiry = activatedAt + 30 days
      PUBLISH("recert.required", { agentUUID, domain, newCurriculumVersion, graceExpiry })
      UPDATE agents SET recertStatus = "pending", graceExpiry = graceExpiry WHERE agentUUID = agentUUID
      INSERT INTO audit_events (eventType: "recert_triggered", agentUUID, domain, previousCurriculumVersion, newCurriculumVersion, NOW())
    CATCH NotificationFailure
      RETRY 3 TIMES at 5s intervals
      IF still failing THEN
        MARK agentUUID AS "notification_failed" // needs manual admin intervention
        TRIGGER FR-18 WARNING "Agent recertStatus update failed for agentUUID {agentUUID}."
      END IF
    END TRY
  END FOR
END

FUNCTION CheckGracePeriodExpiry()
// scheduled job, runs daily at midnight UTC
BEGIN
  TRY
    expired = SELECT * FROM agents WHERE recertStatus = "pending" AND graceExpiry < NOW()
    FOR EACH agent IN expired DO
      UPDATE credentials SET status = "superseded"
        WHERE agentUUID = agent.agentUUID AND status = "active" AND curriculumVersion != agent.latestCurriculumVersion
    END FOR
    NOTIFY admin "{N} agent credentials set to superseded"
  CATCH JobFailure
    INSERT INTO audit_events (eventType: "grace_period_job_failed", ...)
    TRIGGER FR-18 WARNING "Grace period monitoring job failed. Credential supersession may be delayed by up to 24 hours."
    RETRY on next daily run
  END TRY
END

```

Figure 5.7 Re-Certification Trigger Workflow

```

FUNCTION MatchAgentToPostings(event)
  // triggered by cert.issued – must complete inside a 30-second SLA window
BEGIN
  startTime = NOW()
  IF idempotency_check_fails(event) THEN RETURN END IF // suppress duplicate cert.issued

  openPostings = SELECT * FROM job_postings
                  WHERE required_domain = event.domain AND is_open = TRUE AND deleted_at IS NULL

  scoredMatches = []
  FOR EACH posting IN openPostings DO
    relevance = COMPUTE_RELEVANCE(examScore: event.totalScore, jaccard: JACCARD(event.skillTags, posting.skill_tags))
    scoredMatches.APPEND({ posting, relevance })
  END FOR

  topMatches = TOP_K(scoredMatches, k = MATCH_LIMIT)

  TRY
    INSERT INTO agent_matches (match_id, cert_id: event.credentialID, posting_id, relevance_score, matched_at)
    FOR EACH match IN topMatches // single atomic transaction
  CATCH PostgresUnavailable
    RETRY with backoff 1s, 2s, 4s, 8s
    IF still failing THEN DEAD_LETTER_QUEUE(); TRIGGER FR-18 CRITICAL; RETURN
  END TRY

  PUBLISH("agent.matched", { agentUUID: event.agentUUID, matches: topMatches })

  IF NOW() - startTime > 30s THEN LOG_SLA_BREACH("marketplace_match", NOW() - startTime) END IF
END

FUNCTION RegisterTaskPosting(adminRequest)
  // POST /api/v1/marketplace/postings
BEGIN
  IF NOT VALID_JWT(adminRequest.token, role = "admin") THEN
    RETURN ERROR 401/403
  END IF
  SANITIZE(adminRequest.payload) // guards against prompt-injection / parsing exploits

  IF adminRequest.min_score_threshold < 80.0 THEN
    RETURN ERROR "min_score_threshold must be >= 80.0"
  END IF

  postingID = GENERATE_UUID_V4()
  INSERT INTO job_postings (postingID, required_domain, min_score_threshold, skill_tags, created_by, is_open: TRUE)
  PUBLISH("posting.added", { postingID, required_domain: adminRequest.required_domain })

  RUN_RETROACTIVE_MATCH(postingID) // scans historical credentials for already-qualified agents
END

```

Figure 5.8 Talent Marketplace Service Component

```

FUNCTION AnalyzeSkillGaps(event)
  // triggered by exam.failed on the examination-events channel
BEGIN
  { agentUUID, examID, failedQuestions } = event

  IF NOT EXISTS(examinations WHERE examID = examID AND finalScore < 80.0) THEN
    RETURN ERROR "invalid payload, exam not in failed state"
  END IF

  IF EXISTS(skill_gaps WHERE examID = examID) THEN
    INSERT INTO audit_events (eventType: "duplicate_suppressed", examID)
    RETURN existing.gapID // idempotency key = examID
  END IF

  TRY
    classified = LLM_CLASSIFY(failedQuestions)
    FOR EACH failure IN classified DO
      failure.severity = failure.cognitiveTier == "factual" ? "CRITICAL" : "MODERATE"
    END FOR
  CATCH LLMLTimeout
    RETRY 3 TIMES with exponential backoff
    IF still failing THEN
      classified = ALL failures marked "MODERATE" // generic fallback
      INSERT INTO audit_events (eventType: "llm_analysis_fallback", examID)
    END IF
  END TRY

  diagnosticPayload = BUILD_PAYLOAD(weakSubdomains: classified, recommendedModuleIds)
  gapID = GENERATE_UUID_V4()

  TRY
    INSERT INTO skill_gaps (gapID, examID, agentUUID, diagnosticPayload, severity)
  CATCH PostgresUnavailable
    RETRY 3 TIMES
    IF still failing THEN DEAD_LETTER_QUEUE(); TRIGGER FR-18 CRITICAL "Skill gap write failure for examID {examID}."
  END TRY

  PUBLISH("remediation.required", { agentUUID, gapID, diagnosticPayload })
END

```

Figure 5.9 Skill Gap Analysis Component


```

FUNCTION PollServiceHealth()
BEGIN
  LOOP EVERY 30 SECONDS
    FOR EACH service IN REGISTERED_MICROSERVICES DO
      TRY
        response = HTTP_GET(service.healthEndpoint, timeout = 5s)
        status = response.code == 200 ? "UP" : "DEGRADED"
      CATCH Timeout
        status = "DOWN"
      END TRY

      INSERT INTO service_health (service_name, status, latency_ms, checked_at: NOW())
      UPDATE_PROMETHEUS_METRIC(service.name, status)

      IF status == "DOWN" AND DOWN_DURATION(service) > 60s THEN
        GenerateAlert("service_unavailable", severity = "CRITICAL")
      ELSE IF status == "DEGRADED" THEN
        GenerateAlert("health_monitoring_timeout", severity = "WARNING")
      END IF
    END FOR
  END LOOP
END

FUNCTION GenerateAlert(condition, severity)
BEGIN
  INSERT INTO alerts (condition, severity, created_at: NOW())
  IF severity == "CRITICAL" THEN
    SEND_EMAIL_NOTIFICATION(admin_list, condition)
  END IF
  // Alert Center on the Admin Dashboard reads from this table on its 30-second refresh
END

FUNCTION VerifyAuditIntegrity()
  // runs every 6 hours, supports FR-07 / SEC-08
BEGIN
  LOOP EVERY 6 HOURS
    unverified = SELECT * FROM audit_events WHERE verifiedAt IS NULL
    FOR EACH record IN unverified DO
      recomputedHash = SHA256(record.payloadJSON)
      IF recomputedHash != record.payloadHash THEN
        UPDATE audit_events SET integrityStatus = "tamper_suspected" WHERE eventID = record.eventID
        GenerateAlert("integrity_warning", severity = "CRITICAL")
      ELSE
        UPDATE audit_events SET verifiedAt = NOW()
      END IF
    END FOR
  END LOOP
END

```

Figure 5.10 Monitoring & Observability Component

GLOBAL HEALTH STATUS

Auto-refresh: 30 seconds

Infrastructure Summary

API Gateway
Status: Healthy
Latency: 42 ms

PostgreSQL
Conn: 8 / 100
Latency: 28 ms

ChromaDB
Status: Healthy
Latency: 85 ms

Redis Bus
Queue: 0 msg
Latency: 18ms

Microservice Telemetry

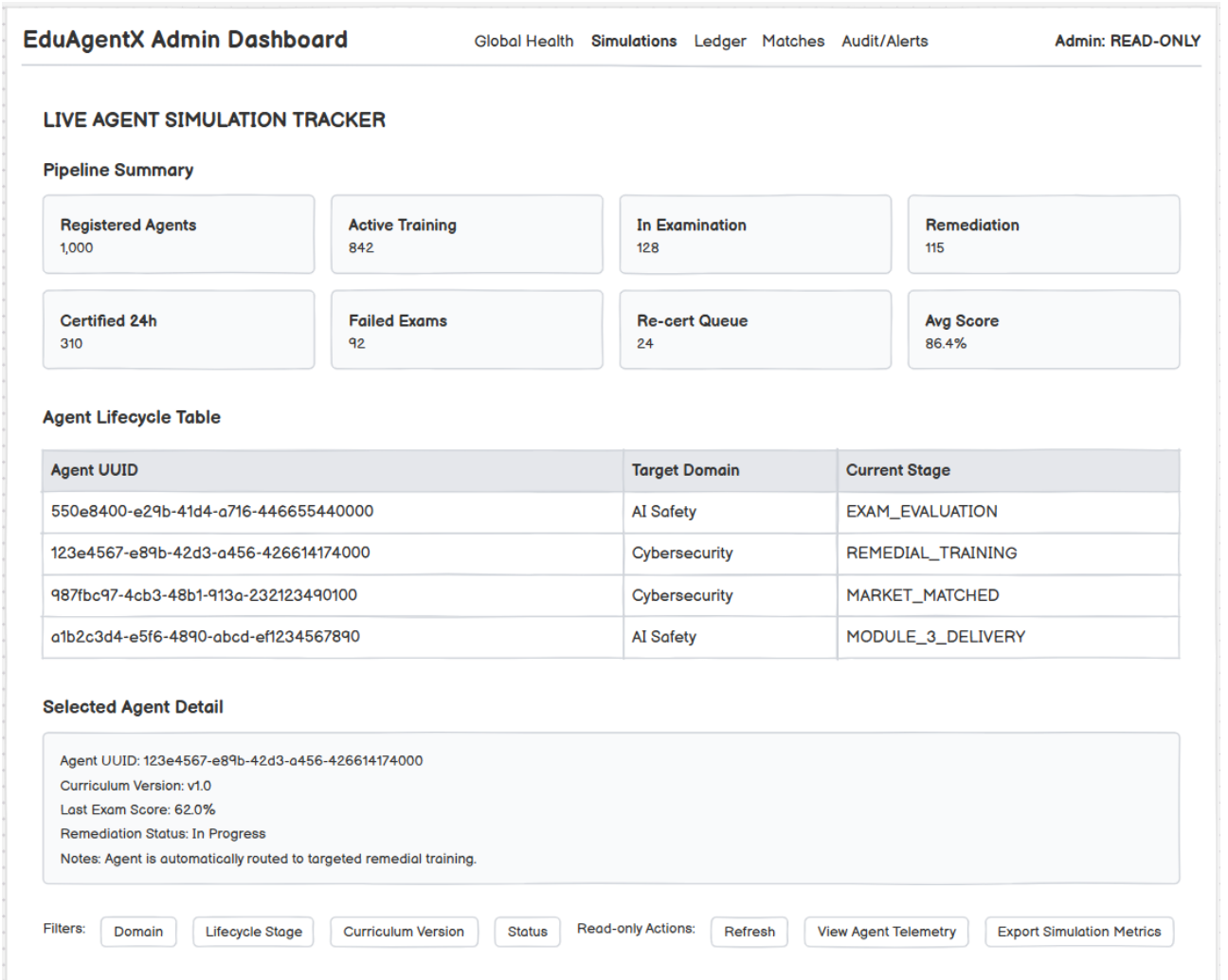
Service Name	Uptime	Avg Latency	Health Status
Agent Registry Service	99.99%	35 ms	Healthy
Curriculum Engine	99.95%	920 ms	Healthy
Training Session Service	99.98%	150 ms	Healthy
Examination Validation	99.90%	1180 ms	Warning
Certification Service	99.99%	85 ms	Healthy
Talent Marketplace Service	99.99%	64 ms	Healthy
Skill Gap Analysis Service	99.96%	210 ms	Healthy
Monitoring Service	100.00%	40 ms	Healthy

Recent Alerts

- > [INFO](#) Curriculum v1.1 activated for Cybersecurity Threat Analysis
- > [WARNING](#) Examination Validation latency exceeded warning threshold
- > [INFO](#) Health polling cycle completed successfully

Filters: Read-only Actions:

Figure 6.1. Panel 1: Global Health Status Dashboard



CRYPTOGRAPHIC CREDENTIAL LEDGER

Credential Summary

Total Issued
1,420Active Credentials
1,405Superseded
15Hash Alerts
0

Latest Credentials (Append-Only Records)

Credential UUID	Score	SHA-256 Digest (Truncated)
88b19e2a-14d1-411a-8533-8a38a7bd88aa	92.5%	e3b0c44298fc1c149afb4c8996fb..
44e99ffb-25c2-422b-9644-9b49b8ce99bb	85.0%	f2d81a260eaa481ceea1f34689db..
77c00f2c-36d3-433c-8755-0c50c9df00cc	88.3%	c1a72b351fbbce52dfbb2e45790ef..

Selected Credential Detail

Credential UUID: 88b19e2a-14d1-411a-8533-8a38a7bd88aa
Agent UUID: 550e8400-e29b-41d4-a716-446655440000
Domain: AI Safety Fundamentals
Curriculum Version: v1.0
Status: Active
Verification Result: SHA-256 hash valid
Notes: Record is append-only and cannot be modified through the dashboard.

Filters:

Domain

Curriculum Version

Credential Status

Score Range

Read-only
Actions:

Refresh

Verify Credential
HashExport Credential
Report

Figure 6.3. Panel 3: Cryptographic Certificate Registry

